Escuela Técnica Superior de Ingeniería

Grado en Ingeniería de las Tecnologías Industriales

TRABAJO DE FIN DE GRADO

Determinación de Etapas del Sueño mediante Redes Neuronales Híbridas

Marta Turias Romero

Tutor: Juan José Perea Rodríguez

Curso académico 2024-25

Convocatoria Junio

Resumen

La monitorización del sueño mediante tecnologías accesibles se ha consolidado como una necesidad urgente, dada la creciente prevalencia de trastornos del sueño y la escasez de especialistas. En este contexto, el presente trabajo propone el desarrollo de un modelo híbrido *CNN+BiLSTM* para la clasificación automática de las etapas del sueño (Vigilia, NREM y REM) a partir de señales fisiológicas captadas con dispositivos de bajo coste. Mediante un preprocesamiento basado en Análisis de Componentes Principales (PCA), se minimizan los artefactos y se concentran las características más relevantes de la señal, optimizando la carga computacional y facilitando la extracción de patrones significativos. A continuación, una *CNN* jerárquica capta las estructuras locales de la señal en el dominio temporal–espectral, mientras que una *BiLSTM* modela las dependencias secuenciales inherentes a las transiciones entre etapas de sueño.

El objetivo central de este trabajo es demostrar que un enfoque de aprendizaje profundo no solo permite automatizar por completo el proceso de clasificación, sino que además supera a las metodologías numéricas de clasificación categórica basadas en regresiones logarítmicas, habitualmente empleadas en dispositivos económicos. Para ello, se incorpora una validación cruzada y ajuste de hiperparámetros, garantizando que el modelo generalice correctamente sin requerir altos recursos de hardware. De este modo, se ofrece una solución *end-to-end* que reduce drásticamente el tiempo de inferencia y la intervención manual en la etapa de etiquetado, facilitando diagnósticos tempranos y precisos en entornos de atención primaria o investigación ambulatoria.

Asimismo, se asientan las bases para futuras líneas de desarrollo, tales como la incorporación de arquitecturas basadas en Transformers con mecanismos de atención, la fusión de señales multimodales y la implementación en plataformas de *edge computing* para procesamiento en tiempo real. En definitiva, este trabajo aspira a democratizar el acceso a herramientas avanzadas de diagnóstico de trastornos del sueño, ofreciendo un sistema altamente escalable y de bajo coste que contribuye a mejorar la atención clínica y la calidad de vida de las personas, especialmente en contextos con recursos limitados.

Palabras clave: *Monitorización del sueño; CNN-BiLSTM; Clasificación automática; Etapas del sueño.*

Abstract

Sleep monitoring through accessible technologies has become an urgent necessity, given the rising prevalence of sleep disorders and the shortage of specialists. In this context, the present study proposes the development of a hybrid CNN–BiLSTM model for the automatic classification of sleep stages (Wake, NREM and REM) from physiological signals acquired with low-cost devices. By employing pre-processing based on Principal Component Analysis (PCA), artefacts are minimised and the most relevant signal features are concentrated, thereby optimising computational load and facilitating the extraction of meaningful patterns. A hierarchical CNN then captures local structures in the temporal–spectral domain, while a BiLSTM models the sequential dependencies inherent in transitions between sleep stages.

The central aim of this research is to demonstrate that a deep-learning approach not only fully automates the classification process but also outperforms categorical classification methods based on logistic regressions, which are commonly used in economical devices. To achieve this, a cross-validation and hyperparameter optimisation has been incorporated, ensuring that the model generalises correctly without requiring extensive hardware resources. In this way, an end-to-end solution is offered that drastically reduces inference time and manual intervention during labelling, thereby facilitating early and precise diagnoses in primary care or ambulatory research settings.

Moreover, this work lays the groundwork for future developments, such as the incorporation of Transformer-based architectures with attention mechanisms, the fusion of multimodal signals and deployment on edge-computing platforms for real-time processing. Ultimately, this thesis aspires to democratise access to advanced diagnostic tools for sleep disorders by providing a highly scalable, low-cost system that contributes to improving clinical care and quality of life, especially in contexts with limited resources.

***Keywords**: Sleep monitoring; CNN-BiLSTM networks; Automatic classification; Sleep stages.*

Agradecimientos

Índice general

1 Introducción y Justificación **15**
1.1 Importancia del Estudio 15
1.2 El Sueño: Concepto, Funcionamiento y Patologías 15
1.2.1 Definición de Sueño 15
1.2.2 Función del Sueño 16
1.2.3 Principales Patologías del Sueño 16
1.3 Etapas del Sueño 17
1.4 Redes Neuronales y Predicción de Datos Secuenciales 18
1.4.1 Redes Neuronales Artificiales 18
1.4.2 Predicción de Datos Secuenciales 19
1.5 Objetivo del Estudio 20
1.6 Organización de la Memoria 21

2 Estado del Arte **23**
2.1 Introducción 23
2.2 Métodos Tradicionales y Automáticos para la Clasificación de las Etapas del Sueño 24
2.3 Uso de Redes Neuronales Recurrentes (RNN) y Redes BiLSTM en Series Temporales Biomédicas 25
2.4 Otras Arquitecturas Relevantes 27
2.5 Ventajas y Limitaciones 28

3 Descripción de los datos **31**
3.1 Introducción 31
3.2 Sensores y Mediciones Realizadas 32
3.3 Estructura del Dataset 33
3.4 Proceso de Etiquetado de las Fases del Sueño 34

4 Metodología **37**
4.1 Introducción 37
4.2 Ventajas del Modelado con RNN 38
4.2.1 BiLSTM 39
4.2.2 Ventajas del Modelado con CNN 39
4.3 CNN + BiLSTM 41
4.4 Configuración de Hiperparámetros 43
4.5 Desbalanceo de Clases 45

5 Descripción de Python y TensorFlow **49**
5.1 Introducción 49

5.2 Python como Lenguaje de Desarrollo 49
5.3 TensorFlow: Infraestructura para Aprendizaje Profundo 50
5.4 Justificación del Uso de Python y TensorFlow en este Trabajo 50

6 Preprocesamiento de los Datos 53
6.1 Introducción 53
6.2 Limpieza y Normalización de los Datos 54
6.2.1 Selección de Estrategia de Normalización 54
6.2.2 Análisis de la Distribución de los Datos 55
6.2.3 Comparación de Métodos de Normalización 56
6.3 Análisis de Variables y Reducción Dimensional 60
6.3.1 Evaluación de la Contribución Individual de las Variables 60
6.3.2 Análisis de Redundancia y Justificación del Uso de PCA 62
6.3.3 Resultados del Análisis PCA 63

7 Métricas de Cuantificación de la Capacidad Predictiva 67
7.1 Introducción 67
7.2 Error Medio Absoluto (MAE) 68
7.3 Error Cuadrático Medio (MSE) 68
7.4 Raíz del Error Cuadrático Medio (RMSE) 69
7.5 Coeficiente de Correlación 69
7.6 El Coeficiente de Determinación (R^2) 70
7.7 Matriz de Confusión 71
7.7.1 Accuracy (Precisión Global) 72
7.7.2 Precision (Valor Predictivo Positivo) 73
7.7.3 Recall (Sensibilidad o Tasa de Verdaderos Positivos) . . 74
7.7.4 Specificity (Especificidad o Tasa de Verdaderos Negativos) 75
7.7.5 F1-score 75
7.7.6 Coeficiente de Correlación de Matthews (MCC) 76
7.7.7 Coeficiente de Kappa de Cohen 77

8 Diseño de la Red Neuronal Implementada 81
8.1 Introducción 81
8.2 Preprocesamiento de Datos y Codificación de Etiquetas 81
8.2.1 One-hot Encoding de Etiquetas 82
8.2.2 Expansión de Dimensiones de la Señal 82
8.2.3 División Entrenamiento/Validación y Pesos de Clase . 82
8.3 Bloque CNN de Entrada (Conv1D) 82
8.3.1 Justificación de Cada Componente 83
8.3.2 Configuración Concreta de las Tres Capas 83
8.3.3 Resumen del Flujo CNN 83
8.4 Bloque BiLSTM (Modelo de Dependencias Temporales) 83
8.4.1 Primera Capa BiLSTM 83
8.4.2 Segunda Capa BiLSTM 84
8.4.3 Dropout(0.4) 84
8.5 Capas Densas Finales y Predicción 84
8.5.1 Capa Densa Intermedia 84
8.5.2 Dropout(0.4) 84

8.5.3 Capa de Salida (softmax) 84
8.6 Compilación del Modelo y Estrategia de Entrenamiento 85
8.6.1 Función de Pérdida: Focal Loss 85
8.6.2 Optimizador: Adam con OneCycleLR 85
8.6.3 Callbacks Adicionales 85
8.6.4 Pesos de clase 85
8.7 Resumen de Entradas y Salidas de Cada Capa 86

9 Evaluación, Resultados y Discusión del Modelo 87
9.1 Introducción 87
9.2 Métricas Numéricas de Aproximación Continua 89
9.3 Matriz de Confusión 91
9.3.1 Métricas Asociadas a la Matriz de Confusión 92
9.4 Interpretación de los Resultados Asociados a la Matriz de Confusión 93

10 Conclusiones y Trabajos Futuros 97
10.1 Conclusiones 97
10.2 Trabajos Futuros 99

A Anexo Red Neuronal Implementada 103
A.1 Introducción 103
A.2 Bloque CNN de entrada (Conv1D) 103
A.2.1 Justificación Teórico-Práctica de Emplear Convoluciones en Datos Secuenciales 103
A.2.2 Elección de Tres Capas Conv1D y Progresión de Filtros (64 →128 →256) 104
A.2.3 Kernel size = 3 y padding = "same": Equilibrio entre Contexto Local y Preservación de la Dimensión 104
A.2.4 BatchNormalization y Dropout(0.3) tras cada Conv1D: Estabilización y Regularización 105
A.2.5 Regularización L2 (`kernel_regularizer = 0.001`) . . 106
A.2.6 Código de Referencia para el Bloque CNN 106
A.3 Bloque BiLSTM (Modelo de Dependencias Temporales) 107
A.3.1 Motivación para Usar LSTM Bidireccional 107
A.3.2 Primera capa BiLSTM: 64 unidades por dirección y return_sequences=True 108
A.3.3 Segunda capa BiLSTM: 32 Unidades por Dirección y return_sequences=False 109
A.4 Capas densas Finales y Predicción 110
A.4.1 Capa Densa Intermedia (Dense(64, ReLU) y Dropout(0.4)) 110
A.4.2 Capa de Salida (Dense(3, softmax)): Clasificación en tres Clases 111
A.5 Compilación del Modelo y Estrategia de Entrenamiento 111
A.5.1 Función de pérdida: Focal Loss ($\alpha = 0{,}25$, $\gamma = 2{,}0$) . . 111
A.5.2 Optimizador Adam con OneCycle Learning Rate: Rationale y Beneficios 112

A.5.3 Callbacks Adicionales: EarlyStopping, ModelCheckpoint y LambdaCallback para Registro de Métricas 114
A.5.4 Código de referencia para compilación 115
A.6 Preprocesamiento de Datos y Cálculo de Pesos de Clase 116
A.6.1 División Entrenamiento/Validación 116
A.6.2 Cálculo de Pesos de Clase con `compute_class_weight` 116
A.6.3 One-hot Encoding de Etiquetas y Expansión de Dimensiones de Entrada 117

Índice de figuras

Figura 1: Porcentaje de encuestados que padecieron trastornos del sueño en los últimos doce meses en distintos países. Elaboración propia a partir de datos de Statista [8]. 16

Figura 2: Distribución de las variables de entrada antes de la normalización. Se observa una fuerte asimetría en BM y VIN, una distribución bimodal en TSDM y una distribución aproximadamente normal en DA. 56

Figura 3: Comparación de métodos de normalización. Se observa que *RobustScaler* es más estable frente a outliers en BM y VIN, mientras que *MinMaxScaler* conserva la distribución de TSDM y DA. 57

Figura 4: Gráfico de sedimentación del análisis PCA donde se muestra la proporción de varianza explicada por cada componente principal. 64

Figura 5: Matrices de confusión para cada uno de los cuatro pacientes a estudio (etapas de sueño 1, 2 y 3). 92

Índice de Tablas

Tabla 1: Efecto de las técnicas de reponderación sobre el rendimiento de la red implementada 47

Tabla 2: Análisis estadístico de las variables tras aplicar diferentes métodos de normalización 59
Tabla 3: Comparativa de métricas obtenidas con diferentes configuraciones de variables. 62
Tabla 4: Coeficientes del PCA para cada variable original. 65
Tabla 5: Varianza explicada y acumulada por cada componente principal. 65

Tabla 6: Métricas numéricas continuas por paciente. 90
Tabla 7: Métricas de asociadas a las matrices de confusión mostradas en la Figura 5 por paciente 93

Capítulo 1

Introducción y Justificación

1.1. Importancia del Estudio

Según el National Heart, Lung, and Blood Institute [1], los estudios del sueño, o polisomnografías, son pruebas indoloras que evalúan la calidad del descanso y cómo el cuerpo reacciona ante posibles trastornos del sueño. Estos estudios registran actividad cerebral, frecuencia cardíaca, respiración y niveles de oxígeno en la sangre durante toda la noche, ayudando a los médicos a diagnosticar problemas del sueño.

La evaluación manual de las etapas del sueño es un proceso laborioso para los especialistas y puede variar entre diferentes evaluadores, lo que limita su precisión y consistencia. Por ello, es fundamental implementar redes neuronales para predecir de manera más precisa y automatizada las distintas etapas del sueño, mejorando así la eficiencia y fiabilidad del proceso, tal como destacan Mousavi et al. [2].

El análisis automatizado permite un diagnóstico más preciso y la elaboración de tratamientos específicos. Tratar adecuadamente los trastornos del sueño mejora la calidad de vida y reduce significativamente el riesgo de desarrollar problemas cardíacos, accidentes cerebrovasculares, obesidad, diabetes, cáncer y depresión. Además, una mejor calidad del sueño contribuye a prevenir accidentes y lesiones asociadas a la somnolencia diurna, resaltando así la importancia de estos estudios para la salud pública, según destaca la American Academy of Sleep Medicine [3].

1.2. El Sueño: Concepto, Funcionamiento y Patologías

1.2.1. Definición de Sueño

El sueño es un estado fisiológico complejo, caracterizado por una reducción temporal del nivel de conciencia, indispensable para la recuperación física y mental. Según la Real Academia Española (RAE, 2024 [4]), el sueño se define sencillamente como el “acto de dormir”. Sin embargo, va más allá de un simple descanso; es una necesidad biológica esencial que permite al organismo restaurar funciones vitales, facilitando

un rendimiento diario óptimo. Los estudios sobre privación del sueño han mostrado que su ausencia completa puede resultar letal, mientras que la falta parcial desencadena un aumento compensatorio del sueño en días posteriores (conocido como deuda de sueño), confirmando la importancia del sueño en el mantenimiento del equilibrio fisiológico [5].

1.2.2. Función del Sueño

Históricamente, el sueño ha sido un fenómeno poco comprendido por la ciencia, y todavía existen numerosos aspectos sin aclarar. En un principio, se interpretaba como un estado pasivo durante el cual el cerebro permanecía inactivo [6]. Sin embargo, avances recientes en la medición de la actividad cerebral han demostrado que el sueño es un proceso dinámico. Durante esta etapa, el cerebro muestra una actividad comparable a la vigilia y ocurren cambios fisiológicos importantes, incluyendo fluctuaciones en la presión arterial, la frecuencia cardiaca y respiratoria, la temperatura corporal y la secreción hormonal.

El sueño cumple funciones esenciales en procesos cerebrales específicos como la memoria, la concentración, la regulación emocional y la percepción del dolor. Además, facilita la eliminación de toxinas del cerebro, asegurando su óptimo funcionamiento. Por otra parte, actúa como un regulador térmico corporal, adaptando la temperatura del organismo según sus necesidades. Por lo tanto, está directamente vinculado con la regulación de diversos procesos corporales vitales, como el metabolismo, el control del apetito, las funciones inmunológicas y hormonales, así como la salud cardiovascular [3].

1.2.3. Principales Patologías del Sueño

El American College of Cardiology [7] estima que hasta un 8 % de las muertes podría estar asociado a patrones de sueño deficientes. Las investigaciones señalan que mantener hábitos de sueño saludables reduce significativamente el riesgo de mortalidad prematura.

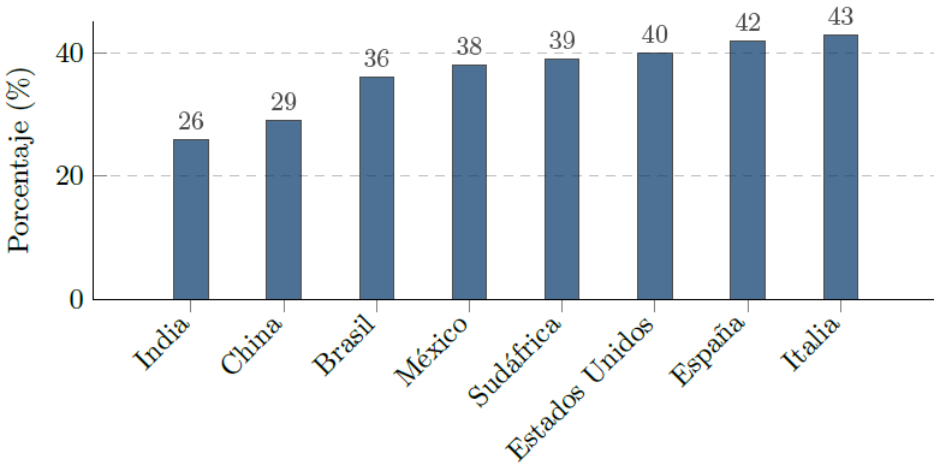


Figura 1: Porcentaje de encuestados que padecieron trastornos del sueño en los últimos doce meses en distintos países. Elaboración propia a partir de datos de Statista [8].

Una encuesta de Statista Consumer Insights [8], realizada entre enero y diciembre de 2023, reveló que Italia (43 %) y España (42 %) fueron los países con mayor porcentaje de personas que manifestaron haber padecido trastornos del sueño recientemente. En contraste, China (29 %) e India (26 %) reportaron los niveles más bajos, como se observa en la Figura 1.

La Sociedad Española de Neurología (SEN) estima que entre el 20 % y el 48 % de los adultos sufre en algún momento dificultades para iniciar o mantener el sueño, y al menos un 10 % presenta trastornos crónicos graves, muchas veces sin diagnóstico [9].

Entre las patologías del sueño más relevantes destacan las siguientes [10]:

- **Apnea del sueño**: Interrupciones repetidas de la respiración durante el sueño, que reducen los niveles de oxígeno y alteran el descanso. Sus síntomas incluyen ronquidos, jadeos al dormir y somnolencia diurna excesiva [11].

- **Privación y deficiencia de sueño**: La privación implica una duración de sueño insuficiente, mientras que la deficiencia abarca tanto el tiempo, como la calidad o el momento del sueño, afectando su eficacia reparadora [12].

- **Narcolepsia**: Trastorno neurológico crónico caracterizado por somnolencia extrema y episodios súbitos de sueño durante actividades cotidianas [13].

- **Insomnio**: Dificultad persistente para conciliar o mantener el sueño, o para lograr un descanso reparador, incluso con condiciones adecuadas. Afecta el rendimiento y la calidad de vida [14].

1.3. Etapas del Sueño

Los estudios científicos han demostrado que el sueño no es un estado uniforme, sino que se compone de distintas etapas organizadas en ciclos que se repiten varias veces a lo largo de la noche. La calidad del descanso y el rendimiento funcional no dependen únicamente del tiempo total dormido, sino también de cómo se distribuyen estas etapas y del momento en que ocurren [15].

Las etapas del sueño se dividen en tres grandes categorías: vigilia, sueño no REM (NREM) y sueño REM (Rapid Eye Movement), tal como describen Morokuma et al. [16]:

- **Vigilia:**
  - Fase en la que el cerebro se encuentra en un estado de alerta y actividad consciente, caracterizada por la percepción del entorno, la atención dirigida y la capacidad de respuesta voluntaria. La vigilia es el estado opuesto al sueño y constituye la base de la experiencia consciente cotidiana.

- **Sueño No REM (NREM):**
  - **N1 (sueño ligero)**: Fase inicial en la que disminuye la actividad cerebral y el cuerpo comienza a relajarse.
  - **N2 (sueño ligero)**: Etapa más prolongada del sueño. Se caracteriza por una reducción del ritmo cardíaco y de la temperatura corporal.
  - **N3 (sueño profundo)**: Fase más restauradora, en la que se produce la recuperación física, se refuerza el sistema inmunológico y se consolida la memoria.
- **Sueño REM:**
  - Fase en la que se producen los sueños más intensos. La actividad cerebral es similar a la de la vigilia, y el cuerpo experimenta una parálisis muscular temporal (excepto en ojos y diafragma). Esta etapa está relacionada con el procesamiento emocional y la consolidación de la memoria.

La proporción y duración de estas fases varían con la edad. Durante la infancia, el sueño profundo (N3) es más prolongado, lo que explica por qué los niños pueden dormir profundamente incluso en ambientes ruidosos. Con el paso de los años, el patrón del sueño cambia: las personas mayores tienden a acostarse y despertarse más temprano, presentan un sueño más fragmentado y experimentan una disminución en las etapas de sueño profundo y REM. Aun así, los porcentajes relativos de cada etapa suelen estabilizarse tras la infancia [15].

1.4. Redes Neuronales y Predicción de Datos Secuenciales

1.4.1. Redes Neuronales Artificiales

Las Redes Neuronales Artificiales (Artificial Neural Networks, ANNs) son modelos matemáticos inspirados en el funcionamiento del sistema nervioso central. Consisten en una red de elementos interconectados entre sí, denominados "neuronas artificiales", que se organizan típicamente en capas: capa de entrada, capas ocultas o intermedias, y capa de salida. Cada capa se conecta con la siguiente mediante conexiones ponderadas por pesos, junto con sesgos y funciones de activación [17].

Desde el punto de vista de entradas y salidas, una red neuronal puede interpretarse como una caja negra: no se conoce explícitamente la función que transforma las entradas en salidas, pero se dispone de un mapeo observable entre ambas. La cantidad de neuronas, el número de capas y los valores de los pesos dependerán de la complejidad de los datos de entrada que la red debe aprender.

Durante la etapa de entrenamiento, la red se "alimenta" con datos de entrada y ajusta los pesos de las conexiones entre neuronas para que la

salida generada se aproxime lo máximo posible a la deseada. Es decir, la red aprende a modelar una correspondencia entre entradas y salidas a través de un proceso iterativo de optimización que minimiza una función de error.

Para ello, se utiliza comúnmente el algoritmo de retropropagación de errores (backpropagation algorithm), como se describe en [17]. Las capacidades de las ANNs se detallan en la investigación desarrollada por Hornik et al. (1988) [18], donde se demuestra que las redes multicapa con una sola capa oculta y conexiones hacia adelante (feedforward) son aproximadores universales: pueden reproducir con suficiente precisión funciones arbitrarias, siempre que se disponga de suficientes datos y una arquitectura adecuada. A este tipo de redes se les denomina redes neuronales artificiales o shallow ANNs. No obstante, es imposible determinar a priori cuántas neuronas ocultas serán necesarias para modelar adecuadamente un mapeo entre variables de entrada y salida.

Las redes neuronales recurrentes (Recurrent Neural Networks, RNNs) constituyen una evolución de las ANNs específicamente orientada al modelado de series temporales.

1.4.2. Predicción de Datos Secuenciales

El uso de redes neuronales en la predicción de datos secuenciales, como las etapas del sueño, ha revolucionado el análisis y diagnóstico de trastornos del sueño, debido a su capacidad para modelar dependencias temporales, procesar grandes volúmenes de datos y mejorar continuamente su precisión.

Las principales ventajas que presentan son:

- **Capacidad para modelar secuencias temporales:** Las redes neuronales, y en particular las redes neuronales recurrentes (RNN) y las redes de memoria a largo plazo (LSTM), desarrolladas por Hochreiter & Schmidhuber [19], se han destacado en tareas de análisis de secuencias temporales debido a su capacidad para modelar dependencias a lo largo del tiempo. Este tipo de redes son especialmente útiles en aplicaciones donde los datos son secuencias, como las etapas del sueño, que siguen patrones cíclicos y progresivos.

- **Precisión en la clasificación de las etapas:** Las redes neuronales pueden aprender de manera efectiva las características de las distintas etapas del sueño a partir de señales biométricas como el electroencefalograma (EEG), respiración, ritmo cardíaco y movimientos corporales. Esto permite la clasificación precisa de las etapas del sueño, como se describe por Hassan & Kumar [20].

- **Mejora del diagnóstico de trastornos del sueño:** El uso de redes neuronales en el análisis del sueño también mejora el diagnóstico de trastornos como la apnea del sueño, el insomnio y otros trastornos relacionados, ya que estas redes pueden identificar patrones complejos y sutiles en los datos de sueño que pueden ser pasados por alto en los análisis tradicionales, como se destaca en Zhang et al.[21].

- **Manejo de grandes volúmenes de datos:** Las redes neuronales pueden procesar y analizar grandes volúmenes de datos, lo que es crucial en el contexto de los estudios del sueño donde se recopilan múltiples señales a lo largo de la noche. Esto permite que el sistema maneje eficazmente los datos y los traduzca en diagnósticos significativos, tal como se desarrolla en Rashid et al. [22].

- **Automatización y reducción de costos:** Al automatizar la predicción de las etapas del sueño, las redes neuronales ayudan a reducir la dependencia de especialistas altamente capacitados. Esto puede ser de gran ayuda en entornos con limitados recursos, donde el uso de tecnologías de bajo costo para el monitoreo del sueño se vuelve más accesible, como describen Liu et al. [23] .

- **Mejora continua:** Las redes neuronales tienen la capacidad de mejorar su rendimiento a medida que se entrenan con más datos. Este proceso de aprendizaje continuo contribuye a la precisión y confiabilidad de los diagnósticos, lo que hace que los modelos sean cada vez más efectivos para detectar patrones de sueño y posibles trastornos, que se deduce del estudio de Schirrmeister et al. [24].

1.5. Objetivo del Estudio

El objetivo principal de este trabajo es desarrollar un modelo eficiente, basado en redes neuronales, que permita predecir, de forma automática, a partir de señales fisiológicas comprimidas mediante componentes principales, las diferentes etapas del sueño y detectar posibles trastornos asociados,

Este objetivo general se desglosa en los siguientes objetivos específicos:

- **Proporcionar una solución eficaz** frente al creciente número de personas que padecen trastornos del sueño, teniendo en cuenta la limitada disponibilidad de especialistas en esta área.

- **Facilitar diagnósticos más rápidos y precisos** mediante el uso de modelos predictivos, con el fin de asistir no solo a especialistas, sino también a médicos generales en la identificación de patrones anómalos del sueño.

- **Fomentar el uso de tecnologías accesibles**, promoviendo el empleo de dispositivos de bajo coste que permitan implementar soluciones de análisis del sueño en entornos con recursos limitados.

- **Maximizar la precisión en la clasificación de etapas del sueño**, optimizando tanto el diseño del modelo como las estrategias de preprocesamiento, con el fin de mejorar la detección y tratamiento de alteraciones del sueño.

1.6. Organización de la Memoria

Además de este capítulo inicial, donde se expone la introducción y justificación del tema elegido, la memoria se compone de los siguientes capítulos, estructurados para facilitar una comprensión progresiva del trabajo realizado:

- **Capítulo 2:** expone un estado del arte sobre las principales técnicas utilizadas para la clasificación de las etapas del sueño. Se revisan tanto métodos tradicionales como enfoques basados en redes neuronales recurrentes (RNN) y arquitecturas BiLSTM, así como otras arquitecturas relevantes. También se analizan sus ventajas y limitaciones.

- **Capítulo 3:** describe el conjunto de datos empleado, incluyendo los sensores y mediciones realizadas, la estructura del dataset y el proceso de etiquetado de las fases del sueño.

- **Capítulo 4:** describe la metodología seguida, incluyendo el diseño de la arquitectura CNN + BiLSTM, la configuración de los hiperparámetros, el tratamiento del desbalanceo entre clases y la comparación con otras arquitecturas alternativas.

- **Capítulo 5:** justifica la elección de Python y TensorFlow como herramientas de desarrollo, explicando sus ventajas en el contexto del aprendizaje profundo y la manipulación de datos.

- **Capítulo 6:** detalla el proceso de preprocesamiento de los datos, que incluye la limpieza y normalización, el análisis de la distribución y la reducción dimensional.

- **Capítulo 7:** expone las métricas utilizadas para cuantificar la capacidad predictiva del modelo, tales como el error medio absoluto (MAE), el error cuadrático medio (MSE), la raíz del error cuadrático medio (RMSE), los coeficientes de correlación y determinación (R^2), el F1 Score y la matriz de confusión.

- **Capítulo 8:** presenta el modelo híbrido propuesto CNN + BiLSTM para la clasificación de las etapas del sueño. Se usa regularización, focal loss para clases desbalanceadas, optimizador Adam con OneCycleLR y callbacks para optimizar el entrenamiento. El preprocesamiento adapta etiquetas y dimensiones para la red.

- **Capítulo 9:** presenta los resultados obtenidos tras la evaluación del modelo. Se incluyen métricas como precisión, recall y F1-score. Además, se realiza una interpretación crítica de los resultados y los principales desafíos identicados.

- **Capítulo 10:** expone las conclusiones del estudio, resumiendo los hallazgos principales, reflexionando sobre las limitaciones del trabajo y proponiendo recomendaciones para futuras investigaciones.

Capítulo 2

Estado del Arte

2.1. Introducción

Clasificar de forma fiable las etapas del sueño es una piedra angular tanto para la investigación neurofisiológica como para la práctica clínica, ya que permite diagnosticar trastornos como la apnea obstructiva o el insomnio y evaluar la eficacia de intervenciones terapéuticas, tal como se esbozó en el capítulo anterior. Tradicionalmente, esta tarea se ha realizado de manera manual a partir de polisomnografía (PSG), el "patrón oro" —es decir, el método de referencia más preciso y confiable para una evaluación clínica— que combina el electroencefalograma (EEG), el electrooculograma (EOG), el electromiograma (EMG) y señales cardiorrespiratorias, siguiendo las guías de la American Academy of Sleep Medicine (AASM) [3].

En este capítulo se realiza una "*traza*" de la evolución desde esos "métodos tradicionales" hasta los "sistemas automáticos de última generación". En primer lugar, se describirán los enfoques basados en ingeniería de características y clasificadores lineales (regresión logística, random forest...), detallando las variables del dominio del sueño (potencia δ, densidad de husos, variabilidad de la frecuencia cardíaca (HRV)) y sus limitaciones para capturar la dinámica temporal. A continuación, se revisará la irrupción de las redes neuronales, empezando por los perceptrones multicapa (MLP) y justificando la transición necesaria hacia modelos con memoria: RNN (*Recurrent Neural Network*), LSTM (*Long Short-Term Memory*) y BiLSTM (*Bidirectional Long Short-Term Memory*) capaces de modelar las dependencias secuenciales que caracterizan la transición NREM–REM. El capítulo finaliza con las arquitecturas híbridas y los Transformer, subrayando sus requisitos de datos, avances en precisión y retos de interpretabilidad.

Esta "panorámica" tiene tres objetivos:

1. **Establecer una perspectiva:** se pretende dar visión global básica, hacerla profunda implicaría, por sí solo, una memoria completa, para situar el ámbito en el que se ubica este proyecto y los estudios previos que lo avalan.

2. **Fundamentar el problema de investigación:** al exponer las carencias de los enfoques clásicos y las brechas actuales (capacidad,

determinación, generalización...), se justifica la necesidad del modelo que será implementado.

3. **Definir el marco metodológico:** al catalogar sensores, algoritmos y métricas empleadas en la literatura, se establecen los criterios de comparación que guiarán la validación experimental del trabajo.

2.2. Métodos Tradicionales y Automáticos para la Clasificación de las Etapas del Sueño

El sueño es un proceso biológico complejo que se organiza en ciclos con distintas etapas. Su clasificación se ha realizado tradicionalmente de forma manual siguiendo las directrices de la AASM [3], mediante la polisomnografía (PSG). Esta prueba diagnóstica registra diversas señales fisiológicas, como el electroencefalograma (EEG), el electrooculograma (EOG), el electrocardiograma (ECG) y el electromiograma (EMG), a través de sensores colocados en el cuerpo del paciente.

Si bien este procedimiento constituye el estándar clínico, presenta importantes limitaciones: alto coste, demanda de tiempo y una notable dependencia del criterio del especialista. Estas limitaciones afectan directamente a la consistencia del etiquetado de etapas, lo cual se evidencia en una concordancia variable entre evaluadores. Por ejemplo, el coeficiente kappa de Cohen, utilizado para medir la consistencia entre anotadores, suele situarse entre 0,76 y 0,78 en adultos, pero puede descender a 0,57–0,63 en centros internacionales, y ser aún menor en estudios pediátricos, debido a la mayor variabilidad del EEG en la infancia.

Entre las señales electrofisiológicas, el EEG y el EOG siguen siendo fundamentales para diferenciar con precisión las etapas del sueño. No obstante, el ECG ha emergido como una alternativa prometedora, especialmente en contextos de monitorización domiciliaria. Investigaciones recientes han demostrado que es posible estimar las etapas del sueño a partir de variables derivadas del ECG, como la frecuencia cardíaca (HR), el intervalo entre latidos (RR) y la variabilidad de la frecuencia cardíaca (HRV) [25]. Dichas señales comenzaron a ser utilizadas en los primeros sistemas automáticos que se construyeron en torno a pipelines de *feature-engineering* clásico [6]: potencia espectral en bandas δ y θ, densidad de husos, complejidad no lineal, etc.

En esta línea diversos autores han propuesto enfoques automáticos basados en estas variables. Adnane et al. [26] utilizaron el análisis de fluctuación desentrenada de la HRV para distinguir entre sueño y vigilia, y para estimar la eficiencia del sueño. Xiao et al. [27] aplicaron un algoritmo de bosque aleatorio sobre características extraídas de la HRV, mientras que Singh et al. [28] utilizaron el intervalo RR para diferenciar entre sueño REM y NREM. Por su parte, Yucelbas et al. [29] desarrollaron un sistema que identifica las fases: Vigilia, NREM y REM utilizando características no lineales y morfológicas del ECG.

Ante estos desafíos, se han desarrollado sistemas automáticos de clasificación del sueño. Algunos de ellos ya están integrados en software comercial de PSG, proporcionando anotaciones preliminares que posteriormente son revisadas por especialistas. Estos métodos automáticos han evolucionado desde enfoques tradicionales, basados en reglas fijas y extracción manual de características, hacia modelos más avanzados que emplean algoritmos de aprendizaje profundo capaces de detectar patrones complejos directamente desde las señales.

Otro enfoque de aprendizaje automático es el basado en la regresión logística (RL) multinomial que presenta la ventaja de poder realizar un análisis global gracias a su interpretación probabilística y entrenamiento ultrarrápido. Gaiduk et al. [30] usaron parámetros derivados de respiración, frecuencia cardíaca y acelerometría para lograr una exactitud del 72 % en Vigilia/NREM/REM con sólo treinta grabaciones domiciliarias. Wang et al. [31] demostraron que, aun siendo superada por *desbordamiento del gradiente* y LSTM, la RL mantiene valor como línea base explicativa al incorporar EEG del cerebro. La principal fortaleza de estos métodos es su transparencia, los coeficientes vinculan la potencia δ o el índice θ/α con la probabilidad de encontrarse en cada fase. Sin embargo, parten de la hipótesis de que la separación entre cada fase es lineal, que en muchos casos no se cumple, además de su dependencia del tipo de sensor y la población sobre la que se realiza el estudio.

Los enfoques basados en el aprendizaje profundo han mostrado mejoras notables en la precisión, alcanzando valores de kappa entre 0,67 y 0,87, según Erdenebayar et al. [25]. Sin embargo, la mayoría de estos estudios se han centrado en adultos, lo cual limita su aplicabilidad a otras poblaciones. En el caso de niños y lactantes, se ha optado a menudo por dividir el sueño en solo dos o tres etapas y, en algunos casos, por utilizar señales distintas al EEG, empleando configuraciones de bajo número de canales para facilitar el cribado.

No obstante, estos enfoques requieren una fase previa de ingeniería de características: extraer variables intermedias del ECG, aplicar transformaciones de alta dimensión y seleccionar los atributos más discriminativos para optimizar el entrenamiento de los clasificadores. Esta dependencia de procesamiento previo limita en parte la escalabilidad y adaptabilidad de dichos modelos frente a los sistemas basados en aprendizaje profundo end-to-end.

2.3. Uso de Redes Neuronales Recurrentes (RNN) y Redes BiLSTM en Series Temporales Biomédicas

Las redes neuronales recurrentes (RNN) han demostrado ser herramientas eficaces para el análisis de datos biomédicos secuenciales, especialmente en el procesamiento de señales fisiológicas como el EEG, el ECG y la HRV. A diferencia de los modelos clásicos de aprendizaje automático, las RNN son capaces de modelar dependencias temporales, lo que las convierte en

una opción especialmente adecuada para el estudio de fenómenos biológicos dinámicos como la arquitectura del sueño [32].

Los primeros intentos de aplicar redes neuronales a la clasificación automática del sueño recurrieron a perceptrones multicapa (MLP) entrenados con vectores de características extraídas de épocas de 30 s de EEG. Fraiwan et al. [33] emplearon un MLP *feed-forward* para distinguir los estados de sueño en neonatos y sólo alcanzaron un $\kappa = 0{,}65$, en parte porque cada época se analizaba de forma aislada y se ignoraban las transiciones fisiológicas a lo largo de la noche. Una década después, la revisión de Aboalayon et al. [34] señaló que esa falta de contexto temporal era la principal causa de la alta confusión en las fronteras entre las fases Vigilia/NREM y REM/Vigilia y recomendó, explícitamente, incorporar modelos con memoria, que considerasen la dependencia temporal de los datos.

La llegada masiva de GPUs (*Graphics Processing Units*) de relativo bajo coste catalizó arquitecturas *end-to-end*. DeepSleepNet unió redes convolucionales (CNN) y BiLSTM para alcanzar, y en ocasiones superar, el 82 % de precisión en Sleep-EDF, una base de datos pública ampliamente utilizada en la investigación del sueño que sigue el formato europeo de datos de sueño (*Sleep European Data Format*), utilizando EEG monocanal y eliminando por completo la extracción manual de rasgos [35]. Bresch et al. [36] exploraron 58 variantes de CNN+LSTM y demostraron su viabilidad de uso con dispositivos portátiles de bajo consumo y clasificación en tiempo real. AttnSleep introdujo atención *multi-head* para capturar dependencias largas y mejorar la detección de la fase REM con un solo canal [37]. Por su parte, Tezuka et al. [38] validaron un conjunto CNN+LSTM que mostró un elevado potencial, aunque con margen de mejora.

Una variante particularmente robusta de las RNN es la previamente mencionada red de memoria a largo plazo bidireccional (BiLSTM), que amplía las capacidades de las LSTM tradicionales al procesar las secuencias en ambas direcciones, hacia adelante y hacia atrás. Esta bidireccionalidad permite capturar de forma más completa la estructura temporal de los datos, ya que considera tanto el contexto pasado como el futuro de cada instante. En aplicaciones biomédicas, donde se dispone del registro completo de la señal, por ejemplo, una noche completa de sueño, este tipo de modelado resulta especialmente ventajoso, como se destaca en [35], [39].

Estudios recientes han mostrado que las BiLSTM superan en rendimiento a sus versiones unidireccionales en tareas como la clasificación de etapas del sueño o la detección de eventos fisiológicos anómalos en señales EEG. Un ejemplo representativo es el trabajo de Supratak et al. [35], que introdujeron el modelo DeepSleepNet, antes mencionado. Con esta propuesta lograron resultados superiores a modelos previos basados únicamente en LSTM, demostrando la eficacia de combinar extracción jerárquica de características con modelado temporal bidireccional.

Del mismo modo, Zhang et al. [39] emplearon una red BiLSTM sobre señales respiratorias y ECG para mejorar la detección de apnea del sueño, alcanzando niveles de precisión y sensibilidad significativamente más altos que modelos convencionales. La capacidad del modelo para identificar patrones dependientes del contexto temporal resultó clave para discriminar

episodios respiratorios complejos.

En el caso de la transición entre vigilia y sueño, particularmente la etapa NREM, caracterizada por su ambigüedad electrofisiológica, las redes BiLSTM también han demostrado ventajas relevantes. Al considerar simultáneamente el contexto previo y posterior de un segmento de señal, estas redes pueden capturar de forma más precisa las transiciones sutiles entre estados, lo que contribuye a una mejor identificación de dicha etapa, frecuentemente subrepresentada en los registros [40].

El éxito de las redes BiLSTM en este dominio se debe a su capacidad para modelar secuencias de datos de forma dinámica y contextual, adaptándose a las particularidades fisiológicas del sujeto y del entorno. Esta propiedad ha facilitado el desarrollo de herramientas automáticas que complementan —e incluso superan en ciertos casos— la evaluación manual de especialistas. Su integración en sistemas portátiles, aplicaciones clínicas y entornos de monitorización en tiempo real representa un avance significativo hacia el diagnóstico automatizado y la personalización del tratamiento en medicina del sueño [35], [39], [40].

2.4. Otras Arquitecturas Relevantes

Además de las redes LSTM y BiLSTM, se han desarrollado múltiples arquitecturas de aprendizaje profundo con resultados prometedores en el análisis de series temporales biomédicas. Entre estas destacan las unidades recurrentes simplificadas, como las redes GRU (*Gated Recurrent Units*), las CNN, los modelos basados en mecanismos de atención como Transformer, y las arquitecturas híbridas que combinan distintas técnicas para aprovechar sus fortalezas complementarias.

Las redes GRU han sido ampliamente empleadas como una alternativa computacionalmente más ligera a las LSTM. Aunque comparten la capacidad de modelar dependencias temporales, las GRU utilizan una estructura más simple con menos puertas, lo que reduce el tiempo de entrenamiento y los requisitos de memoria. Esta eficiencia las convierte en una opción atractiva para dispositivos portátiles o aplicaciones embebidas que requieren procesamiento en tiempo real. Por ejemplo, Zhang et al. [41] aplicaron una arquitectura basada en GRU para la detección de arritmias a partir de señales ECG, obteniendo resultados comparables a los obtenidos con LSTM, pero con una reducción significativa en el coste computacional.

Las redes CNN, aunque originalmente desarrolladas para visión por computadora, han evidenciado un rendimiento notable en tareas de análisis de señales fisiológicas. Su capacidad para extraer características espaciales y patrones locales las hace útiles como etapa de preprocesamiento antes del modelado secuencial. En particular, al aplicarlas sobre ventanas temporales de señales como EEG o ECG, las CNN permiten identificar estructuras relevantes sin necesidad de ingeniería manual de características. En este sentido, el modelo *DeepSleepNet* propuesto por Supratak et al. [35], que combina CNN con BiLSTM, ilustra cómo la integración de componentes convolucionales y recurrentes puede mejorar significativamente la clasificación automática de las etapas del sueño.

Recientemente, los modelos basados en Transformer han ganado atención debido a su capacidad para modelar relaciones de largo alcance sin necesidad de recurrencia. Estos modelos emplean mecanismos de atención que asignan pesos diferenciales a distintas partes de la secuencia, lo que permite capturar dependencias globales de manera eficiente. En aplicaciones biomédicas, el uso de Transformer ha mostrado resultados prometedores. Por ejemplo, Song et al. [42] utilizaron una arquitectura basada en atención para analizar señales multicanal de EEG, logrando mejoras sustanciales en la clasificación de estados de sueño y en la detección de eventos epilépticos, superando en varios casos a modelos convencionales basados en RNN. Sin embargo, su complejididad y requerimiento de una gran cantidad de datos para el entrenamiento, no lo hacen adecuados para conjuntos de datos de entrenamiento inferiores a 10^6 [43], [44].

A partir de estas líneas de investigación, han surgido también modelos híbridos que combinan múltiples arquitecturas con el objetivo de capturar tanto patrones locales como dinámicas secuenciales y relaciones contextuales. Estas arquitecturas integran la capacidad de extracción jerárquica de características de las CNN, la modelación temporal de las RNN (o BiLSTM), y la atención adaptativa de los Transformers. En trabajos recientes como el de Gao et al. [45], este tipo de modelos han demostrado un rendimiento superior en tareas como la clasificación automática de patologías del sueño y la predicción de eventos cardíacos, al aprovechar sinérgicamente los puntos fuertes de cada componente.

En conjunto, estas arquitecturas representan un avance relevante en la evolución de los sistemas inteligentes aplicados a la salud. Su flexibilidad y potencial de integración han abierto nuevas posibilidades tanto en el entorno clínico como en plataformas de monitoreo ambulatorio. En el contexto de este trabajo, estos desarrollos sirven como base conceptual y técnica para justificar el diseño de modelos híbridos que combinen convolución, memoria bidireccional y mecanismos de atención, con el fin de mejorar la precisión y generalización en la clasificación de etapas del sueño.

2.5. Ventajas y Limitaciones

El uso de redes neuronales profundas en la clasificación automática de las etapas del sueño ha supuesto una mejora significativa frente a los métodos manuales tradicionales y aquellos basados en reglas predefinidas. Modelos como las redes convolucionales (CNN), las redes de memoria a largo plazo bidireccionales (BiLSTM) y las arquitecturas híbridas han demostrado un rendimiento notable al integrar múltiples señales fisiológicas y detectar patrones complejos con mayor precisión y consistencia [35], [39], [42].

Una de las principales ventajas de estas arquitecturas es su capacidad para aprender representaciones jerárquicas directamente desde los datos crudos, eliminando la necesidad de realizar una ingeniería de características manual específica del dominio. En particular, enfoques como DeepSleepNet, que combinan CNN y BiLSTM, han alcanzado niveles de precisión cercanos a los obtenidos por especialistas humanos, con coeficientes kappa superiores a 0.80 en bases de datos como Sleep-EDF [35]. Además, estos modelos han

mostrado eficacia en la detección de eventos como microdespertares o apneas, gracias a su capacidad para modelar dependencias a largo plazo y transiciones entre etapas del sueño [39].

Otra ventaja destacable es la escalabilidad de estos modelos, que permite su implementación en dispositivos portátiles, aplicaciones móviles y sistemas de monitorización en tiempo real. Estudios recientes han presentado modelos ligeros basados en CNN y GRU optimizados para su uso en entornos embebidos, donde los recursos computacionales son limitados [46], [47].

No obstante, estos enfoques también presentan limitaciones importantes. Uno de los principales desafíos es la generalización del modelo a diferentes contextos clínicos y poblaciones. Muchos estudios han sido entrenados y validados en bases de datos restringidas, como Sleep-EDF o MASS, lo que limita su aplicabilidad en poblaciones pediátricas o en pacientes con trastornos poco frecuentes [48]. La variabilidad interindividual, las diferencias en los protocolos de adquisición y la calidad de las señales afectan directamente al rendimiento del modelo cuando se aplica a nuevos dominios.

Además, la naturaleza opaca de los modelos de aprendizaje profundo —especialmente aquellos que integran múltiples capas y mecanismos de atención— dificulta su interpretación clínica. Esta falta de explicabilidad representa una barrera para su adopción en entornos médicos, donde se requiere transparencia y trazabilidad en los procesos de decisión [49].

Otra limitación importante radica en los requerimientos computacionales asociados al entrenamiento. Aunque la inferencia puede ser optimizada para su ejecución en tiempo real, la fase de entrenamiento suele requerir grandes volúmenes de datos, recursos computacionales intensivos y un ajuste cuidadoso de los hiperparámetros [46]. Esta demanda puede constituir un obstáculo en entornos clínicos con infraestructura limitada.

Por último, aunque se han alcanzado altos niveles de precisión, aún existen dificultades en la clasificación de etapas complejas como la NREM, así como en la robustez frente a artefactos fisiológicos —es decir, distorsiones o interferencias en la señal que alteran su forma real y pueden afectar su interpretación o análisis posterior—, así como frente al ruido electromecánico. Estos aspectos siguen siendo objeto de investigación activa [48], [50].

Por todo ello, aunque los métodos automáticos basados en redes neuronales profundas representan una alternativa prometedora para el análisis del sueño, su consolidación como herramienta clínica requiere seguir investigando para mejorar su interpretabilidad, adaptabilidad y robustez. La combinación de múltiples señales y el desarrollo de modelos más robustos pueden contribuir a superar las limitaciones actuales y hacer que estos sistemas sean una herramienta viable para el diagnóstico y monitoreo del sueño en entornos clínicos y domiciliarios, tal como se destaca en Gaiduk et al. [51].

Capítulo 3

Descripción de los datos

3.1. Introducción

El diseño y la validación de cualquier sistema clasificación de las etapas del sueño depende de la solidez metodológica con la que se capturen, organicen y anoten las señales fisiológicas de referencia. Por ello, en este capítulo se detalla el *andamiaje experimental* que sustenta el resto de la memoria. En primer lugar, se justificará la selección de un conjunto mínimo de señales no invasivas, estas son: acelerometría torácica y pletismografía inductiva, capaces de preservar la información discriminatoria necesaria para diferenciar las fases, o etapas, del sueño sin recurrir a la polisomnografía completa. A continuación, se describirá la arquitectura del *dataset*: desde el reclutamiento de pacientes y los parámetros de muestreo *32 Hz*, hasta la estrategia de división entre entrenamiento y prueba que garantiza la independencia entre pacientes. El capítulo concluye con el protocolo de anotación clínica, donde se detalla el procedimiento de doble ciego y la cuantificación de la concordancia entre evaluadores mediante el coeficiente κ de Fleiss—exigiéndose $\kappa \geq 0{,}80$ con el objetivo de validar el "patrón oro" (comúnmente conocido por su traducción al inglés *gold standard*), ya definido en la sección 2.1.

En este capítulo se fijan dos objetivos:

1. **Delimitar el dominio de entrada** al definir con precisión las variables BM, TSDM, VIN y el parámetro combinado DA, se establecen los límites de información a los que se ceñirá la arquitectura CNN+BiLSTM propuesta en el presente trabajo. De este modo, cualquier mejora atribuida al modelo puede interpretarse sin ambigüedades como resultado del aprendizaje y no de variaciones instrumentales.

2. **Asegurar la trazabilidad de los resultados** la descripción minuciosa del proceso de etiquetado, la estructura de datos y los criterios de exclusión permiten que otros investigadores reproduzcan los experimentos y comparen nuevos algoritmos sobre una base común, aportando transparencia y rigor científico al conjunto del trabajo.

3.2. Sensores y Mediciones Realizadas

Para seleccionar las señales de entrada se priorizó su carácter no invasivo, su robustez frente a artefactos y su relevancia fisiológica en la distinción de fases de sueño. Se optó por dos modalidades complementarias: movimiento corporal y esfuerzo respiratorio, ampliamente acreditadas en la literatura de monitorización domiciliaria y clínico-portátil [52], [53]. En este proyecto se utilizan cuatro tipos de señales, o datos, estas son: BM, TDSM, VIN y DA,

- **Señal de movimiento corporal (BM).** Se empleó un acelerómetro inercial triaxial (3D) colocado en la región torácica, ya que estudios demuestran su eficacia en la detección de microdespertares y movimientos periódicos durante el sueño [52]. Cada eje (x, y, z) se muestrea a *32 Hz*, y, en cada instante i, se calcula la magnitud instantánea de movimiento:

  $$\text{Body}_i = \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2 + (z_i - z_{i-1})^2}.$$

  Este índice reduce la influencia de la orientación y conserva variaciones finas de la postura. A continuación, a partir de los valores i-ésimo, se calcula el parámetro *BM* como la media de Body_i sobre ventanas de *30 s*, alineadas con la convención AASM para épocas de sueño [54].

- **Señal de esfuerzo respiratorio (TSDM y VIN).** Se registró mediante un cinturón de pletismografía inductiva (RIP) torácico, muestreado también a *32 Hz*. Para mitigar artefactos de movimiento y drifts en la línea de base. Los drifts son desviaciones lentas de la línea de base que aparecen en las señales de esfuerzo respiratorio registradas con cinturones de pletismografía inductiva (RIP). Estas señales no representan información ventilatoria real, si no se corrigen, distorsionan la amplitud de cada ciclo e introducen un sesgo en métricas derivadas como TSDM o VIN. Para reducir estas dos desviaciones, se aplicó un filtrado en dos etapas: filtro de mediana (ventana de $1\,s$) seguido de media móvil (ventana de $0{,}5\,s$) [53]. De la señal procesada se extrajeron dos características principales:

  - *TSDM* (profundidad media de exhalación): definida como la mediana de los picos de exhalación, normalizada por el rango intercuartílico para minimizar la influencia de atípicos [55].
  - *VIN* (volumen mediano de inhalación): calculado como la mediana de las áreas bajo curva de cada inhalación, estimada mediante integración trapezoidal por ciclo respiratorio [55].

  Estas dos métricas seleccionadas mostraron, en pruebas previas, mayor discriminación entre REM y NREM sin aumentar la complejidad del modelo, tal y como destaca en [56].

- **Parámetro combinado (DA).** Para capturar interacciones no lineales entre movimiento y respiración, se definió un coeficiente logarítmico combinado:

  $$\text{DA} = \ln\left(\frac{\text{BM}}{\text{BM+VIN}}\right),$$

que atenúa picos excesivos en BM y homogeneiza las escalas de entrada [56]. Esta fusión mejorará la capacidad de la red para distinguir fases con actividad respiratoria variable, sin necesidad de incrementar parámetros de la CNN+BiLSTM.

Todos los parámetros (BM, TSDM, VIN y DA) se normalizaron por paciente utilizando normalización z-score y se calcularon en ventanas de 30 s para su alineación con el etiquetado manual según la AASM [54].

3.3. Estructura del Dataset

En el conjunto de datos empleado se ha seguido el protocolo estandarizado, y de la alta calidad, establecido por el hospital Charité de Berlín, de reputada categoría, tal como se destaca en los estudios [57], [58]. Para maximizar la validez y generalización que permita obtener la red CNN+BiLSTM desarrollada, se estableció la siguiente organización:

- **Pacientes:** 35 voluntarios sanos (20 hombres, 15 mujeres; edad media 48,6±14,5 años; IMC 24,4±4,9 kg/m^2), seleccionados según criterios de exclusión de trastornos respiratorios y neurológicos, fijados en [57]. Este tamaño de muestra y heterogeneidad demográfica garantizan potencia estadística y representan variabilidad entre diferentes pacientes habitual en estudios de sueño domiciliario [59].

- **Duración y muestreo:** Se registraron aproximadamente 260 h de sueño nocturno, con una tasa de muestreo de *32 Hz* para las señales de movimiento y respiración. Esta frecuencia se eligió tras análisis de la literatura que sitúa el rango óptimo entre *25-50 Hz* para capturar patrones respiratorios y microdespertares sin generar exceso de datos [60].

- **Segmentación en épocas:** Las grabaciones se dividieron en ventanas no solpadas de *30 s*, siguiendo la convención AASM para etiquetado de fases [54]. Este horizonte temporal equilibra resolución temporal y calidad de las etiquetas clínicas, además de facilitar la comparación con otros estudios de la comunidad.

- **Partición entrenamiento / prueba:**

  - *Entrenamiento:* 100 h de datos (15 pacientes), con sobremuestreo de las clases minoritarias (Vigilia y REM) y particionado estratificado para mantener proporción fase–paciente [61]. Este enfoque reduce el sesgo de clases y mejora la estabilidad del entrenamiento.

  - *Prueba:* 160 h de datos (20 pacientes), totalizando 19 324 épocas etiquetadas de forma independiente. Se reservó este subconjunto para la evaluación final, asegurando que ningún paciente del entrenamiento reaparezca en la prueba [59].

- **Metadatos y señales auxiliares:** Además de edad, sexo y el índice de masa corporal (IMC), el dataset incluye las señales tradicionales de PSG (EEG, EOG, EMG, ECG), todo ello empleado, exclusivamente, para el etiquetado de referencia. El sistema trabajará, únicamente, a partir de BM, TSDM, VIN y DA, lo que facilita futuras aplicaciones con hardware reducido [56].

3.4. Proceso de Etiquetado de las Fases del Sueño

El etiquetado de las épocas de *30 s* se llevó a cabo de forma manual por al menos dos expertos en medicina del sueño, siguiendo estrictamente la normativa de la ya mencionada American Academy of Sleep Medicine (AASM) [54]. En este contexto, el término "patrón oro", hace referencia al conjunto de etiquetas clínicas consideradas verdaderas y utilizadas como referencia para validar cualquier método automático. Para que dicho *gold standard* sea fiable se garantizó:

- **Validez clínica:** las anotaciones se basaron en criterios estandarizados AASM, asegurando la identificación precisa de marcadores fisiológicos (ondas específicas en EEG, movimientos oculares en EOG, atonía muscular en EMG y variaciones cardiacas en ECG), como se indica en [58].

- **Consistencia inter-evaluador:** al menos dos especialistas mapearon cada época de forma independiente. Se cuantificó el acuerdo mediante coeficientes κ (Cohen y Fleiss), exigiendo un $\kappa \geq 0{,}8$ para considerar una etiqueta reproducible; las discrepancias por debajo de este umbral se resolvieron en sesiones de consenso documentadas, tal como se recomienda en [62], [63].

Para implementar y garantizar este *gold standard* se siguieron las siguientes fases operativas:

1. **Revisión del entrenamiento experto:** Antes de iniciar el estudio, los técnicos participaron en un programa de calibración con cinco registros de referencia, comparando sus anotaciones con un panel supervisor hasta alcanzar un κ intra-grupo $\geq 0{,}80$.

2. **Análisis de PSG completo:** Cada época se clasificó empleando simultáneamente EEG (mastoidea-central), EOG (canal bilateral), EMG (submentoniano) y ECG, aplicando criterios AASM para distinguir Vigilia, NREM y REM. Se marcaron microdespertares cuando al menos dos señales mostraban cambios concordantes.

3. **Agrupación a tres clases:** Con el objetivo de alinear las etiquetas al modelo CNN+BiLSTM, las subfases N1–N3 se fusionaron en la categoría NREM, práctica que mantiene alta sensibilidad y especificidad [64].

4. **Control de calidad entre evaluadores:** Se evaluó la consistencia entre evaluadores con κ de Fleiss en el 10 % de las épocas; aquellas con κ0,60 fueron revisadas en sesiones conjuntas hasta alcanzar consenso, documentando las causas de discrepancia.

5. **Distribución final de etiquetas (conjunto de prueba, 19 324 épocas):**

   - Vigilia: 2571 épocas (13,3 %)
   - NREM: 13876 épocas (71,8 %)
   - REM: 2726 épocas (14,1 %)

   Estas proporciones reflejan la prevalencia típica en poblaciones sanas de edad similar [59].

Este riguroso protocolo garantiza la validación del *gold standard*, la CNN+BiLSTM se entrenó y evaluó con un referente clínico sólido, minimizando sesgos de etiquetado y mejorando la generalización del modelo.

Capítulo 4

Metodología

4.1. Introducción

La solidez de cualquier estudio basado en aprendizaje profundo depende, de la claridad con la que se exponga su *metodología*. Este capítulo reúne, de forma unificada, las decisiones de diseño que sustentan el sistema automático de clasificación de etapas del sueño desarrollado en este proyecto. Su contenido abarca desde la elección de la arquitectura neuronal hasta las estrategias concretas que permiten entrenarla en presencia de datos desbalanceados y restricciones computacionales reales.

En un primer bloque se analizan las razones que justifican el uso de modelos *secuenciales*. Las Redes Neuronales Recurrentes (RNN) y, en particular, sus variantes de memoria prolongada (LSTM) y bidireccional (BiLSTM) que destacan por su capacidad para capturar la continuidad temporal de las señales fisiológicas. Posteriormente se discute cómo las redes convolucionales (CNN) complementan esta virtud extrayendo, de forma automática, patrones locales robustos a ruido y a desplazamientos de fase. La combinación CNN+BiLSTM, núcleo de la propuesta de este proyecto, se describe con detalle: primero en su vertiente conceptual y después en términos de la arquitectura implementada, de modo que quede patente por qué resulta un sistema mejor al aplicar ambos paradigmas por separado.

Un segundo bloque aborda la *ingeniería de entrenamiento*. Se explican los hiperparámetros críticos (tasa de aprendizaje, tamaño de lote, número de épocas, función de activación...) y los criterios prácticos empleados para ajustarlos con un presupuesto computacional limitado. A continuación, se describe el método seguido para compensar el desbalance de clases, etapas del sueño y se justifica la elección realizada frente a otras técnicas, *a priori* mejores.

El objetivo final de este capítulo es doble. Por un lado, garantizar la trazabilidad de todos los resultados empíricos presentados en los capítulos posteriores; por otro, ofrecer un protocolo completo que pueda replicarse y extenderse en nuevos estudios clínico-portátiles. Con este marco técnico establecido, la siguiente sección —**Ventajas del modelado con RNN**— profundiza en las propiedades que hacen de las redes recurrentes el pilar temporal idóneo para la tarea que nos ocupa.

4.2. Ventajas del Modelado con RNN

Las redes neuronales recurrentes (RNN) constituyen una arquitectura fundamental para el procesamiento de datos secuenciales, debido a su capacidad intrínseca para modelar dependencias temporales en series de datos.
Esta característica las hace particularmente adecuadas para aplicaciones biomédicas donde las señales registradas, como las electroencefalográficas (EEG), electrooculográficas (EOG) y electromiográficas (EMG), exhiben dinámicas temporales complejas y correlaciones a lo largo del tiempo, como ocurre en el análisis polisomnográfico para la clasificación de las etapas del sueño [19].

A diferencia de las redes feedforward tradicionales, las RNN incorporan conexiones recurrentes que permiten mantener un estado oculto actualizado a lo largo de las secuencias temporales, lo cual facilita la retención y utilización de información contextual previa para la toma de decisiones posteriores. Esta capacidad es especialmente relevante en la clasificación de etapas del sueño, ya que dichas etapas no solo dependen de características instantáneas de la señal, sino también del contexto secuencial que las precede y sucede, reflejando patrones fisiológicos de transición entre fases [46].

Sin embargo, las RNN convencionales presentan limitaciones para capturar relaciones a largo plazo debido al problema del desvanecimiento y explosión del gradiente durante el entrenamiento mediante retropropagación a través del tiempo [65]. Para superar estas dificultades, se han desarrollado variantes avanzadas como las Long Short-Term Memory (LSTM) y las redes Bidirectional LSTM (BiLSTM), que emplean mecanismos de puertas para controlar el flujo de información y preservar contextos relevantes en secuencias extensas [19], [66].

La efectividad de las LSTM y BiLSTM en la clasificación de las etapas del sueño ha sido demostrada en diversos estudios. Por ejemplo, Supratak et al. [35] propusieron un modelo basado en LSTM capaz de capturar la evolución temporal de las señales EEG y EOG, logrando una mejora significativa en la precisión de clasificación respecto a métodos tradicionales. Asimismo, las BiLSTM ofrecen la ventaja de procesar las secuencias en ambas direcciones temporales (hacia adelante y hacia atrás), lo que permite una contextualización más completa de la señal, un aspecto crucial dado que la etapa de sueño actual puede estar influenciada tanto por eventos anteriores como posteriores en la secuencia [46].

Otra ventaja importante de las RNN es su capacidad para trabajar directamente con señales temporales crudas o preprocesadas, evitando la necesidad de realizar una extracción manual y compleja de características. Esto reduce el sesgo humano y permite que el modelo aprenda automáticamente las representaciones más discriminativas para la tarea de clasificación [67].

4.2.1. BiLSTM

Las RNN han sido ampliamente utilizadas para modelar datos secuenciales debido a su capacidad para capturar dependencias temporales. No obstante, las RNN tradicionales presentan limitaciones al momento de aprender relaciones de largo plazo debido al conocido problema del desvanecimiento del gradiente [65]. Para mitigar esta dificultad, surgieron las redes LSTM, las cuales introducen bloques de memoria con mecanismos de control que permiten retener y olvidar información de forma selectiva [68].

Cada bloque LSTM incorpora compuertas de entrada, olvido y salida que regulan el flujo de información a través del tiempo. Estas compuertas permiten que la red mantenga un estado de celda, que actúa como una memoria a largo plazo, y un estado oculto, que contiene información a corto plazo. Gracias a este diseño, las LSTM pueden capturar patrones complejos en secuencias largas, siendo adecuadas para tareas como la predicción temporal, el análisis de lenguaje natural o la clasificación de señales fisiológicas [69].

Como una extensión natural de las LSTM, las BiLSTM fueron propuestas para mejorar la capacidad de contextualización al permitir el procesamiento de la información en dos direcciones: de pasado a futuro y de futuro a pasado. En lugar de procesar las secuencias en una sola dirección, como en las LSTM convencionales, las BiLSTM utilizan dos capas ocultas independientes, una que recorre la secuencia hacia adelante y otra hacia atrás. Ambas capas están conectadas a una capa de salida común, lo que permite que la red tenga una visión más completa del contexto [66].

Este enfoque es particularmente ventajoso en tareas donde el significado de un elemento depende tanto del contexto previo como del posterior. Por ejemplo, en clasificación automática de las fases de sueño, una LSTM unidireccional intenta etiquetar la época central de *30 s*, situada justo al inicio de REM, solo contexto precedente (dos épocas NREM con husos y potencia δ elevada); esa información le hace proclive a catalogar la época como NREM. En cambio, una BiLSTM procesa la misma secuencia en ambas direcciones y, además de los rasgos pasados, incorpora el contexto futuro inmediato; al fusionar ambas evidencias, la red identifica correctamente la transición y asigna la etiqueta REM, reduciendo la confusión típica entre NREM y REM en los límites de fase.

En el ámbito del análisis de señales biomédicas, como la clasificación de etapas del sueño, las BiLSTM han demostrado ser altamente efectivas. En este tipo de tareas, la información relevante para clasificar un segmento de señal no siempre está contenida únicamente en el propio segmento, sino también en los patrones que lo rodean. Al aprovechar la bidireccionalidad, las BiLSTM logran extraer dependencias temporales más ricas, lo que contribuye a una mejora en la precisión del modelo [70].

4.2.2. Ventajas del Modelado con CNN

La convolución es una operación matemática fundamental en las redes neuronales convolucionales (CNNs), que combina dos funciones para cuantificar su grado de solapamiento. Esta operación consiste en "deslizar"una

función sobre otra, multiplicar sus valores en cada punto de intersección y sumar los productos resultantes, generando así una nueva función que refleja la interacción entre las funciones originales.

Las CNNs [71] constituyen un tipo de red neuronal profunda ampliamente utilizada en el análisis de imágenes, entre otras aplicaciones. A diferencia de los métodos tradicionales que requieren una extracción manual de características, las CNNs aprenden automáticamente las características relevantes, lo que las hace altamente eficaces para tareas complejas como el reconocimiento de imágenes o el análisis de imágenes médicas. Esta capacidad automática de aprendizaje reduce la dependencia de ingeniería manual y mejora la adaptabilidad del modelo a distintos dominios.

La arquitectura de las CNNs está inspirada en la estructura biológica de la corteza visual, compuesta por agrupaciones de células simples y complejas que responden a subregiones específicas dentro de un campo visual, conocidas como campos receptivos [72]. Siguiendo este principio, las neuronas en una capa convolucional están conectadas únicamente a subregiones localizadas de la capa anterior. Esto limita el número de conexiones y parámetros, haciendo el modelo más eficiente y reduciendo el riesgo de sobreajuste mediante el compartido de pesos y técnicas de submuestreo.

Las capas convolucionales actúan como extractores de características, aplicando filtros específicos (o kernels) que detectan patrones distintivos en los datos de entrada. Matemáticamente, la convolución entre un filtro W y una imagen de entrada X se define como:

$$Y(i,j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X(i+m, j+n)\, W(m,n) \tag{4.1}$$

donde $X(i,j)$ representa el valor del píxel en la posición (i,j), $W(m,n)$ es el filtro de tamaño $M \times N$, y $Y(i,j)$ es el mapa de características resultante. La operación se realiza sobre todos los canales de la imagen, y las salidas se combinan para producir el mapa final. Durante el entrenamiento, la red optimiza los valores de estos filtros para capturar características desde las más simples (bordes, texturas) hasta las más complejas y abstractas en capas profundas.

Las CNNs incluyen además capas ReLU, que introducen no linealidad para modelar representaciones complejas, y capas totalmente conectadas que integran la información extraída para tareas de clasificación o regresión. La arquitectura y los hiperparámetros de la red se ajustan en función de la complejidad del problema, a menudo mediante técnicas de búsqueda como *grid search*.

Aunque las CNNs se popularizaron inicialmente en el análisis de imágenes bidimensionales, su eficacia se ha extendido a dominios con datos unidimensionales (1D) que presentan estructura secuencial o temporal, tales como series temporales, señales biomédicas (ECG, EEG), datos de audio o texto. En este contexto, las CNNs 1D aplican filtros a lo largo de una única dimensión, capturando patrones locales y dependencias temporales de manera automática, similar al modo en que las CNNs 2D operan en imágenes.

Las ventajas descritas hacen de las CNNs una herramienta útil en el aprendizaje profundo para problemas complejos de reconocimiento y análisis

de datos.

4.3. CNN + BiLSTM

La red empleada en este trabajo adopta una topología **híbrida** en dos etapas: (i) un bloque convolucional profundo que actúa como extractor jerárquico de características y (ii) una cadena de capas **BiLSTM** que modela la dinámica temporal bidireccional a partir de las descripciones obtenidas. Esta fusión aprovecha los puntos fuertes de cada paradigma y mitiga sus debilidades cuando se usan de forma aislada.

Bloque convolucional. La primera etapa de la arquitectura aplica *convoluciones unidimensionales* sobre la señal cruda o preprocesada. Cada **filtro convolucional** es un conjunto reducido de pesos que se desplaza (*stride*) por la secuencia y calcula un producto punto en cada posición. Tras el aprendizaje, estos filtros adquieren formas análogas a los *filtros de Gabor*: funciones sinusoidales moduladas por una envolvente gaussiana que actúan como detectores de bandas de frecuencia localizadas; de ese modo extraen motivos característicos —por ejemplo, husos, complejos K o ráfagas rápidas— sin que sea necesario definirlos de forma explícita.

Después de cada convolución se introduce un paso de **max-pooling**. Este operador recorre la señal filtrada con pequeñas ventanas y conserva únicamente el valor máximo de cada una. El efecto combinado de la traslación del filtro y del max-pooling confiere *invariancia por desplazamiento*: un mismo patrón evoca la misma respuesta independientemente de su posición exacta dentro de la ventana de entrada, lo que limita la tendencia de la red a memorizar ubicaciones concretas en lugar de contenido relevante.

Al apilar varias capas, aumenta el **campo receptivo** (*receptive field*) de las neuronas profundas, es decir, el segmento de señal original que puede influir en una activación. Las primeras capas capturan rasgos muy locales; las últimas integran información sobre intervalos temporales más amplios. Así se construye una representación jerárquica que describe la señal en múltiples escalas espectro-temporales, algo que las redes recurrentes puras necesitarían muchas más unidades para alcanzar con la misma eficacia computacional.

Bloque BiLSTM. La segunda etapa procesa la secuencia de *vectores latentes* generados por la CNN con una red **LSTM**. Una LSTM es un tipo de red neuronal recurrente diseñada para modelar dependencias largas: cada unidad mantiene un *estado oculto* $\mathbf{h}_t$ y una *memoria interna* que se actualizan mediante *puertas* (de entrada, olvido y salida). Estas puertas —raíces sigmoides que toman valores entre 0 y 1— controlan cuánta información nueva se incorpora, cuánta se descarta y qué parte se expone al siguiente nivel, evitando la saturación de gradientes que limita a las RNN tradicionales.

En la variante **bidireccional** (BiLSTM) se emplean dos LSTM independientes: una *rama directa* que recorre la secuencia tal como ocurrió ($\mathbf{h}_t^{\rightarrow}$) y una *rama inversa* que la recorre en sentido contrario ($\mathbf{h}_t^{\leftarrow}$). Al

concatenar las salidas de ambas ramas $[\mathbf{h}_t^{\rightarrow}; \mathbf{h}_t^{\leftarrow}]$ cada instante dispone simultáneamente de:

- **Contexto causal** —la información procedente de los vectores anteriores, útil para reconocer la tendencia que lleva la señal, y
- **Contexto anticausal** —la información procedente de los vectores posteriores, que ayuda a confirmar o refinar la interpretación del presente.

Esta doble perspectiva resulta crucial en la clasificación de etapas del sueño: la transición *NREM* $\rightarrow$ *REM*, por ejemplo, suele anticiparse con una disminución progresiva de la amplitud del EEG y confirmarse sólo tras varios segundos de actividad teta; disponer de los dos márgenes temporales permite desambiguar la fase con más acierto que una LSTM unidireccional [35]. Además, al limitarse a procesar la secuencia de representaciones *ya compactadas* por la CNN, la BiLSTM concentra sus recursos computacionales en modelar la dinámica de alto nivel sin verse sobrecargada por detalles de baja frecuencia.

La conjunción de ambas presenta las siguientes ventajas:

1. **Separación de funciones**: la CNN aprende filtros invariantes a fase y robustos al ruido muscular, mientras que la BiLSTM se especializa en la regularidad ultradiana y en las dependencias de largo alcance (>5 min).
2. **Reducción de dimensionalidad temprana**: al comprimir las señales crudas en activaciones convolucionales compactas, se disminuye el número de parámetros que la LSTM debe procesar, evitando la explosión de memoria característica de las RNN cuando operan sobre ventanas crudas de alta resolución.
3. **Gradiente estable**: la retro-propagación atraviesa primero capas convolucionales con *batch-norm*, lo que suaviza la varianza del gradiente antes de llegar a las compuertas LSTM y reduce el riesgo de *vanishing/exploding* grads.
4. **Generalización** frente a modelos monolíticos: estudios en bases Sleep-EDF y MASS muestran mejoras consistentes, al menos, de +3–5 puntos porcentuales (pp) en *F1* global respecto a CNN puras y de +6–8 pp respecto a LSTM unidireccionales, manteniendo igual complejidad computacional [37], [49].

Así pues, la arquitectura **CNN+BiLSTM** captura simultáneamente la riqueza espectral de corto plazo y la estructura temporal de larga duración que caracterizan la arquitectura del sueño, superando la capacidad explicativa de los modelos que aplican convolución o recurrencia por separado.

4.4. Configuración de Hiperparámetros

La expresión *hiperparámetro* designa a cualquier valor que el investigador debe fijar antes de iniciar el entrenamiento de cualquier red neuronal y condiciona la dinámica del algoritmo de optimización y, en consecuencia, el rendimiento final del modelo. A diferencia de los *parámetros*, los pesos y sesgos que la red aprende a partir de los datos, los hiperparámetros no se actualizan durante el proceso de retro-propagación; su ajuste adecuado suele requerir iteraciones sucesivas, validación cruzada y, en ocasiones, una parte sustancial del presupuesto computacional del proyecto desde la perspectiva del coste computacional (complejidad temporal). Lo hiperparámetros que definen una red neuronal son:

1. **Tasa de aprendizaje (*learning rate*).** Es el factor de escala que multiplica el gradiente en cada actualización de pesos. Si la tasa es demasiado alta, los pasos sobre la superficie de error pueden "saltar" el mínimo y provocar inestabilidad numérica; si es demasiado baja, la convergencia se ralentiza y el entrenamiento puede quedar atrapado en mínimos locales. Optimizadores adaptativos como Adam ajustan este coeficiente para cada parámetro en función de los momentos del gradiente y han demostrado una convergencia más rápida y robusta que el descenso por gradiente estocástico clásico [35]. En la práctica se emplea una tasa inicial en el rango $[10^{-4}, 10^{-3}]$ y se decrece de forma programada (*learning-rate scheduling*) cuando el error de validación se estanca.

2. **Tamaño de lote (*batch size*).** Define cuántas muestras se utilizan para estimar un gradiente antes de actualizar los pesos. Lotes pequeños introducen ruido estocástico; ese ruido actúa como regularizador implícito y favorece la generalización, pero puede dificultar la estabilidad del aprendizaje. Lotes grandes producen estimaciones del gradiente más fiables y permiten aprovechar la paralelización de la GPU, aunque requieren más memoria y tienden a sobreajustar (overfitting)[73]. En estudios de clasificación de sueño con señales de bajo coste se emplean típicamente tamaños entre 32 y 128 épocas de *30 s*.

3. **Número de épocas.** Cada época equivale a procesar la totalidad del conjunto de entrenamiento una vez. Aumentar las épocas incrementa la probabilidad de hallar un mínimo profundo, pero también el riesgo de que la red memorice el ruido. Por ello, se monitoriza la pérdida sobre un conjunto de validación y se aplica *parada temprana* (*early stopping*): si el rendimiento de validación no mejora tras un número predefinido de épocas, las denominada épocas de paciencia, el entrenamiento se detiene para evitar sobreajuste.

4. **Arquitectura: número de capas y neuronas.** El ancho (neuronas por capa) controla la capacidad para representar funciones complejas; el fondo (número de capas) determina la profundidad jerárquica de las representaciones. Un modelo infra-parametrizado exhibe alta pérdida

de entrenamiento (*underfitting*); un modelo sobredimensionado necesita regularización adicional (dropout, weight-decay) para no sobreajustar.

5. **Funciones de activación.** Introducen no linealidad en la red. ReLU y sus variantes (Leaky-ReLU, SELU) se imponen en capas convolucionales por su bajo coste computacional y mitigación del problema de gradientes desvanecidos. En unidades recurrentes se emplean *tanh* y σ (sigmoide) dentro de las puertas LSTM/BiLSTM para comprimir el estado en $[-1, 1]$ o $[0, 1]$.

6. **Normalización y regularización.** *Batch Normalization* estandariza la salida de cada capa en mini-lotes, acelerando la convergencia y estabilizando la distribución de activaciones. *Dropout* desactiva aleatoriamente un porcentaje de neuronas en entrenamiento, forzando a la red a aprender representaciones más redundantes y reduciendo el riesgo de sobreajuste [49].

7. **Optimizadores: SGD y Adam.** El descenso por gradiente estocástico (SGD) actualiza los pesos con el gradiente promedio del lote y un único *learning rate* global. Aunque simple y con buena capacidad de generalización, requiere un ajuste fino de la tasa y de un término de inercia (*momentum*). Adam integra momentum de primer y segundo orden para estimar momentos adaptativos; su ajuste automático de la tasa para cada parámetro ofrece convergencia más rápida y menos sensibilidad a la escala del gradiente, ventaja demostrada en clasificación de sueño [35].

8. **Búsqueda de hiperparámetros.** Al no existir una configuración universal de los valores de los hiperparámetros obliga a estrategias sistemáticas de exploración, donde destacan *Grid search* y *Random search*. *Grid search* recorre todas las combinaciones de un espacio discreto; garantiza encontrar la mejor configuración dentro de esa cuadrícula, pero escalona exponencialmente con el número de hiperparámetros. *Random search* toma muestras aleatorias y, para el mismo presupuesto, cubre con mayor probabilidad las regiones más relevantes; demuestra ser más eficiente cuando solo unos pocos hiperparámetros influyen fuertemente en la pérdida. Métodos bayesianos, no utilizados en este estudio por restricciones de tiempo, modelan la función de coste como un proceso gaussiano y eligen la siguiente evaluación donde se espera mayor mejora. En situaciones con hardware limitado se recurre a heurísticas guiadas por la literatura: partir de tasas de aprendizaje y tamaños de lote recomendados, refinar manualmente y validar con *early stopping* [74].

En conjunto, la configuración de hiperparámetros define el comportamiento óptimo de cualquier red. Un correcto ajuste permite conciliar tres objetivos que suelen entrar en tensión: rapidez de convergencia, estabilidad numérica y capacidad de generalización, de predicción.

4.5. Desbalanceo de Clases

En los registros polisomnográficos la distribución temporal de las fases está fuertemente sesgada: la etapa NREM ocupa en adultos sanos entre el 65 % y el 80 % del tiempo total de sueño, mientras que REM apenas alcanza el 20 % y los breves despertares (Vigilia) no superan el 5 %, como se indica en el estudio de [73]. Cuando se entrena una red para predecir la etapa sobre esa distribución, la minimización de la función de pérdida queda dominada por NREM; el gradiente asociado a REM o Vigilia es demasiado débil y la red aprende un modelo que optimiza la *accuracy* global a costa de sacrificar la sensibilidad hacia las clases minoritarias. El resultado es un sistema clínicamente inútil: detecta con gran acierto el sueño profundo, pero ignora microdespertares o episodios REM críticos para el diagnóstico de trastornos.

Reponderación de la pérdida mediante *class weights* y *focal loss* Las cifras anteriores obligan a reforzar el aprendizaje en las clases minoritarias sin perturbar la fisiología subyacente. La estrategia adoptada combina dos técnicas complementarias, implementadas en el código (véase Apéndice A):

1. **Pesos de clase inversamente proporcionales a la frecuencia**. Se emplea la función `compute_class_weight` de *scikit–learn*, que asigna un factor $w_k = \frac{N}{K\,n_k}$ a la clase k (donde N es el total de muestras, K el número de clases y n_k el conteo de la clase). Al incorporarlos al parámetro `class_weight` de `model.fit`, la pérdida de cada ejemplo se multiplica por w_k, de modo que REM y Vigilia producen un gradiente tan influyente como NREM.

2. **Pérdida focal**. La *focal loss* [75] introduce un término dinámico $(1 - p_t)^\gamma$ (donde p_t es la probabilidad asignada a la clase correcta) que atenúa los ejemplos clasificados con alta confianza y enfatiza los difíciles. Con γ=2 y α=0,25 se observa en las bases Sleep-EDF y MASS una mejora media de 3–4 pp en *F1*-REM sin degradar la precisión de NREM. En nuestro caso, la *focal loss* se implementa como función de pérdida personalizada.

La combinación *class weights + focal loss* actúa en niveles distintos: los pesos corrigen el *prior* de clase a nivel de *dataset*, la clase mayoritaria, mientras que la *focal loss* redistribuye la atención del optimizador dentro de cada mini–batch. Juntas producen un gradiente estable y equilibrado que converge en pocas épocas (menos de 50 épocas), evitando tanto el sobreajuste a NREM como las oscilaciones típicas de pesos mal escalados.

Estrategias consideradas y descartadas Se evaluaron otras vías habituales para mitigar el desbalanceo; a continuación se describen de forma resumida las razones para descartarlas en este trabajo:

- **Reequilibrado de mini–batches (sobre, sub–muestreo, SMOTE).** Debido al sobremuestreo se duplican segmentos REM, incrementando el riesgo de sobreajuste a ruido específico; el submuestreo descarta NREM y compromete la arquitectura global del sueño. SMOTE y variantes interpolan señales, pero al operar en el dominio de características corren el riesgo de generar transiciones fisiológicamente imposibles [76].

- **Modelos generativos (Autoencoders, VAE).** En [77] se indica que, aunque los VAE pueden crear ejemplos sintéticos coherentes, requieren un preentrenamiento separado y un control estricto de artefactos para no contaminar el clasificador. Ensayos preliminares demostraron una ganancia marginal (< 1 pp en *F1*-REM) a costa de duplicar el tiempo de entrenamiento y la complejidad del *pipeline*.

- **Preentrenamiento auto–supervisado (SimCLR, masked modelling).** Descrito en [78], estas técnicas fortalecen el codificador antes de la etapa supervisada, pero necesitan grandes lotes sin etiquetas y maquinaria de datos distribuida. En el ámbito de entorno clínico privado, de relativo bajo coste, el coste de ingeniería superaba la mejora esperada.

- **Mecanismos de atención (Transformer, atención temporal).** Tal como se describe en [79], añadir capas de atención sobre la salida de la BiLSTM introduce millones de parámetros adicionales y exige secuencias más largas para aprender patrones globales. Lo cual es un problema para situaciones en las que el número de datos disponibles es relativamente bajo, ya que se puede inducir sobreajuste.

Ventajas de la solución adoptada La combinación de class weights y focal loss aporta una solución equilibrada entre rigor estadístico y viabilidad operativa. Su implementación es relativamente sencilla, sólo hay que considerar diccionario de pesos en y sustituir la pérdida categórica por la *focal loss*; ello evita etapas externas de síntesis de datos y no exige hardware adicional. Además, al no generar ejemplos artificiales se descarta la aparición de artefactos fisiológicamente inverosímiles, reforzando la robustez clínica, mientras que el coste computacional, añadido por época, resulta despreciable lo que permite el uso de equipos relativamente de bajo coste. A todo ello hay que añadir que los hiperparámetros (α, γ) y los propios pesos de cada clase se pueden analizar fácilmente, lo que salvaguarda la interpretabilidad del procedimiento.

Los efectos de esta estrategia se cuantifican en la Tabla 1 sobre el **conjunto de test**. La **macro–*F1*** aumenta de 79.6 % a 92.1 %, esto es +12.5 pp, mientras que la *F1* específica de REM y Vigilia crece en +14.4 pp y +13.4 pp, respectivamente; incluso la *accuracy* global mejora 3–4 pp. Estas métricas son apropiadas para problemas de clases desbalanceadas porque ponderan por igual la sensibilidad y la precisión de cada fase, o clase, evitando que la preponderancia de NREM oculte los fallos en REM y Vigilia [80]. El capítulo

7 describe en detalle cómo se calculan estas métricas y cómo se emplean para monitorizar la convergencia.

Tabla 1: Efecto de las técnicas de reponderación sobre el rendimiento de la red implementada

Métrica (%)	**Sin reponderar**	**+ Class Weights**	**+ Class Weights + Focal Loss**
Macro–F1	79.6	86.8	**92.1**
F1–REM	76.0	84.3	**90.4**
F1–Vigilia	78.2	85.5	**91.6**
Accuracy global	86.4	89.7	**93.2**

En base a los resultados mostrados en la Tabla 1, se puede afirmar que prescindir de autoencoders, VAE y mecanismos complejos de generación o atención no representa una pérdida de rendimiento significativo.

Capítulo 5

Descripción de Python y TensorFlow

5.1. Introducción

En este capítulo se describen las herramientas tecnológicas fundamentales empleadas en este Trabajo de Fin de Grado (TFG), centrado en la clasificación automática de las etapas del sueño mediante redes neuronales profundas. En concreto, se abordan las características de Python y TensorFlow, así como la justificación de su elección, basada en evidencia científica y en su idoneidad para abordar tareas complejas de aprendizaje profundo sobre señales biomédicas.

5.2. Python como Lenguaje de Desarrollo

Python es actualmente uno de los lenguajes de programación más utilizados en ciencia de datos, inteligencia artificial (IA) y aprendizaje profundo (deep learning). Esta popularidad no es casual: Python ofrece una sintaxis clara y legible que facilita el desarrollo colaborativo y reduce la complejidad de los algoritmos implementados [81]. Gracias a su naturaleza interpretada y de alto nivel, los desarrolladores pueden centrarse en la lógica del modelo sin preocuparse por detalles de bajo nivel, lo cual es esencial en aplicaciones experimentales y de investigación.

El ecosistema de Python se ha expandido de forma significativa, proporcionando bibliotecas especializadas para tareas como manipulación de datos (NumPy, pandas), visualización (Matplotlib, Seaborn), análisis estadístico (SciPy, Statsmodels) y aprendizaje automático (Scikit-learn, TensorFlow, PyTorch). Esta riqueza de herramientas permite abordar todas las fases del ciclo de vida de un modelo de IA, desde la preprocesamiento de datos hasta la evaluación y despliegue, lo cual es particularmente relevante en el contexto del análisis de señales fisiológicas como EEG, EOG y EMG, fundamentales para el estudio del sueño [20].

Asimismo, entornos como Jupyter Notebook y gestores de entornos como Anaconda permiten crear entornos reproducibles y compartir experimentos de forma sencilla, lo que es esencial en la investigación académica. El lenguaje

también destaca por su integración con herramientas de visualización y depuración, facilitando el análisis de resultados y la identificación de errores.

5.3. TensorFlow: Infraestructura para Aprendizaje Profundo

TensorFlow es una plataforma de código abierto desarrollada por Google para implementar algoritmos de machine learning y deep learning de forma eficiente, flexible y escalable [82]. Su arquitectura basada en grafos computacionales permite representar operaciones matemáticas complejas como flujos de datos, optimizando así el uso de recursos computacionales tanto en CPUs como en aceleradores como GPUs y TPUs.

Una característica destacable de TensorFlow es su capacidad para ejecutarse en múltiples entornos, desde ordenadores personales hasta servidores distribuidos, sin necesidad de reescribir el código. Esta escalabilidad lo hace adecuado tanto para prototipos iniciales como para despliegues en producción.

TensorFlow incluye también una API de alto nivel, Keras, que permite construir modelos de redes neuronales de forma modular e intuitiva. Keras ofrece una sintaxis simplificada para definir, compilar y entrenar modelos complejos como CNNs y RNNs, manteniendo al mismo tiempo la capacidad de personalizar cada componente del modelo [83].

En el campo de la neurociencia computacional y el análisis del sueño, TensorFlow ha sido ampliamente utilizado para modelar secuencias fisiológicas mediante arquitecturas como LSTM, GRU o CNN-BiLSTM. Estas redes han demostrado ser eficaces en tareas como la clasificación de etapas del sueño, gracias a su capacidad para capturar relaciones tanto temporales como espaciales en los datos [46], [84].

Además, TensorFlow proporciona herramientas como TensorBoard para monitorizar visualmente el entrenamiento de los modelos, facilitando la interpretación de métricas como la pérdida y la precisión, así como la detección de problemas como el sobreajuste.

5.4. Justificación del Uso de Python y TensorFlow en este Trabajo

La elección de Python y TensorFlow para el desarrollo de este TFG no solo responde a su popularidad, sino también a su adecuación técnica para abordar el problema concreto de la clasificación automática de las etapas del sueño. Esta tarea requiere procesar y analizar grandes volúmenes de señales fisiológicas con estructuras temporales complejas, lo que demanda un entorno flexible, potente y orientado al desarrollo ágil.

Python, como lenguaje principal, ofrece una plataforma robusta para el procesamiento y la exploración de datos biomédicos. Bibliotecas como NumPy y pandas permiten manipular con eficiencia grandes conjuntos de datos polisomnográficos, mientras que Matplotlib y Seaborn facilitan su

visualización para análisis exploratorios. Estas capacidades son esenciales en la fase de preprocesamiento, donde se requiere una inspección cuidadosa de las señales y la detección de artefactos.

TensorFlow complementa este ecosistema al proporcionar las herramientas necesarias para implementar arquitecturas avanzadas de redes neuronales. En particular, este trabajo hace uso de modelos híbridos basados en CNN y BiLSTM, que han demostrado ser eficaces en estudios previos. Por ejemplo, Zhang et al. [84] muestran que la combinación de convoluciones para extraer patrones espaciales y capas recurrentes bidireccionales para modelar dependencias temporales mejora significativamente la precisión de clasificación respecto a modelos tradicionales.

Además, Phan et al. [46] introducen SeqSleepNet, una red jerárquica basada en BiLSTM que considera secuencias largas de señales de sueño, demostrando una alta capacidad para capturar patrones globales. Estos trabajos respaldan el enfoque adoptado en este TFG, tanto en términos de arquitectura como de herramientas empleadas.

Otra razón clave para la elección de TensorFlow es su compatibilidad con aceleradores de hardware como GPUs y TPUs, lo que permite entrenar modelos con mayor rapidez. Esto es especialmente relevante al trabajar con datos fisiológicos, donde la duración de los registros puede ser extensa y la frecuencia de muestreo alta.

Finalmente, el ecosistema de Python proporciona herramientas adicionales para tareas de validación cruzada, ajuste de hiperparámetros y evaluación de resultados mediante bibliotecas como Scikit-learn y SciPy. Como señalan Millman y Aivazis [85], estas herramientas han sido diseñadas específicamente para entornos científicos, garantizando precisión y eficiencia en los cálculos.

En términos generales, la combinación de Python y TensorFlow ofrece una solución integrada y validada por la comunidad científica, permitiendo el desarrollo de modelos precisos, escalables y con potencial de aplicación clínica en el análisis del sueño.

Capítulo 6

Preprocesamiento de los Datos

6.1. Introducción

La calidad del preprocesamiento determina, en gran medida, el techo de rendimiento de cualquier sistema de aprendizaje profundo aplicado a datos biomédicos. En particular, los registros obtenidos de sensores de movimiento durante el sueño presentan una doble complejidad: por un lado, *heterogeneidad métrica* entre variables procedentes de distintas fuentes y, por otro, la aparición frecuente de valores atípicos que sesgan la distribución estadística. Abordar ambos retos —limpieza y normalización— es imprescindible para garantizar la estabilidad numérica en cualquier arquitectura de red en general, y la propuesta CNN+BiLSTM, en particular. Si no se garantiza la estabilidad numérica, el modelo podría aprender patrones espurios, producto de escalas incompatibles o de datos aberrantes, lo que comprometería gravemente su capacidad predictiva.

En este capítulo se describe la estrategia de preprocesamiento seguida basada en tres pilares complementarios:

1. **Análisis exploratorio y detección de outliers**. Se emplea el criterio intercuartílico (IQR) para identificar observaciones extremas en las variables `BM`, `VIN`, `DA` y `TSDM`, minimizando la dependencia de supuestos de normalidad.

2. **Selección adaptativa del método de normalización**. A partir de las propiedades de cada distribución se justifica el uso diferenciado de *RobustScaler*, *StandardScaler* y *MinMaxScaler*, de acuerdo con la evidencia empírica y la literatura reciente sobre la influencia del escalado en el desempeño de los clasificadores [86], [87].

3. **Evaluación cuantitativa del impacto del escalado**. Se analizan métricas de asimetría, curtosis y dispersión antes y después de la transformación, lo que ha demostrado que la estrategia propuesta reduce la varianza no informativa sin distorsionar la semántica de las señales originales.

La aproximación adoptada pone especial énfasis en preservar la estructura latente de los datos: se evita la aplicación indiscriminada de un único método

de normalización y se opta por un enfoque *variable-específico* que respeta tanto la presencia de colas pesadas como la existencia de distribuciones bimodales. Este cuidado resulta crítico en escenarios donde la tarea final —clasificación en tres etapas de sueño— depende de matices dinámicos sutiles que pueden desaparecer tras un escalado inadecuado.

A lo largo de las secciones siguientes se detallan: (1) el razonamiento estadístico detrás de la elección de cada técnica, (2) los resultados comparativos obtenidos y (3) las implicaciones prácticas para el posterior modelado con redes neuronales. De este modo, el capítulo sienta las bases para una representación numérica coherente y robusta, capaz de potenciar la capacidad predictiva del sistema propuesto.

6.2. Limpieza y Normalización de los Datos

6.2.1. Selección de Estrategia de Normalización

En el contexto del análisis de datos procedentes de sensores biomédicos, uno de los desafíos más frecuentes radica en la disparidad existente entre las escalas de las distintas variables que integran el conjunto de datos. Esta heterogeneidad suela afectar negativamente al rendimiento de los modelos de aprendizaje profundo, generando inestabilidad en el proceso de optimización, lo que dificulta la convergencia durante el entrenamiento y compromete la calidad de las representaciones internas, tal como se advierte en [88]. Para mitigar estos efectos, resulta imprescindible aplicar un proceso de normalización que homogeneice las escalas sin distorsionar la estructura de la información original.

Con el fin de diseñar una estrategia de normalización efectiva, se llevó a cabo un análisis de distintas técnicas de normalización considerando las propiedades estadísticas de cada variable. En este proceso se identificó la presencia de valores atípicos —también conocidos como *outliers*—, definidos como aquellas observaciones que se sitúan significativamente alejadas del patrón general. La detección de estos valores se realizó aplicando el criterio clásico basado en el rango intercuartílico (IQR), de uso extendido en entornos biomédicos por su robustez frente a distribuciones no normales [89]. En concreto, se consideró como outlier cualquier valor x que cumpliera:

$$x < Q_1 - 1{,}5 \cdot \mathrm{IQR} \quad \text{o} \quad x > Q_3 + 1{,}5 \cdot \mathrm{IQR}$$

donde Q_1 y Q_3 representan el primer y tercer cuartil de la distribución, respectivamente. Esta metodología permite identificar valores extremos sin depender de la media ni de la desviación estándar, que pueden estar sesgadas por dichos mismos valores anómalos.

El análisis reveló una marcada heterogeneidad entre las variables estudiadas: mientras algunas presentaban distribuciones aproximadamente normales y acotadas, otras exhibían un alto grado de asimetría y una presencia significativa de outliers. En consecuencia, se optó por una estrategia de normalización diferenciada, ajustando el método aplicado a las características estadísticas particulares de cada variable, con el objetivo

de preservar la integridad de la información y favorecer un aprendizaje más estable y efectivo.

6.2.2. Análisis de la Distribución de los Datos

El análisis estadístico preliminar de los datos de entrada constituye un paso fundamental en cualquier proceso de modelado predictivo, ya que permite identificar posibles irregularidades, distribuciones inadecuadas o valores atípicos que puedan comprometer la robustez y fiabilidad del modelo. En este caso, se ha llevado a cabo una exploración de la calidad de los datos disponibles, correspondientes a las variables BM, DA, VIN y TSDM, las cuales recogen información relativa a los movimientos del paciente durante el sueño.

A partir del estudio de las distribuciones de frecuencia de estas variables —representadas en la Figura 2—, se identificaron tres patrones estadísticos diferenciados, los cuales permiten caracterizar la naturaleza y la calidad de la información recogida:

- **Distribuciones con fuerte asimetría y presencia de valores atípicos extremos:** este tipo de comportamiento se observó en las variables BM y VIN. Ambas presentan una concentración muy elevada de valores en los rangos inferiores, junto con colas alargadas hacia valores significativamente altos. Este tipo de distribución, altamente sesgada, sugiere la existencia de outliers que pueden tener un impacto negativo sobre el entrenamiento del modelo.

- **Distribución bimodal con alta dispersión:** identificada en la variable TSDM, cuya distribución muestra dos picos claramente diferenciados, lo que sugiere la combinación de al menos dos fuentes distintas de comportamiento dentro de una misma variable. Este fenómeno puede indicar solapamiento de información o redundancia, lo que complica la interpretación y puede interferir en la capacidad del modelo para establecer patrones precisos.

- **Distribución aproximadamente normal con ligera asimetría:** corresponde al caso de la variable DA, que presenta una forma cercana a una distribución gaussiana. Su perfil estadístico es más equilibrado que el del resto de variables, con una leve asimetría y sin valores extremos destacables, lo que la convierte en una fuente de datos más estable y fiable desde el punto de vista del aprendizaje automático.

La Figura 2 muestra los histogramas correspondientes a las variables originales antes de aplicar cualquier tipo de técnica de normalización. Con un análisis visual, se pueden distinguir con claridad distintos patrones en la distribución de cada variable, lo que pone de manifiesto la necesidad de un análisis individualizado.

En particular, la variable BM tiene una distribución altamente asimétrica con una cola derecha muy pronunciada, en la que se detectan valores atípicos extremos que superan ampliamente las 4000 unidades. Esta característica

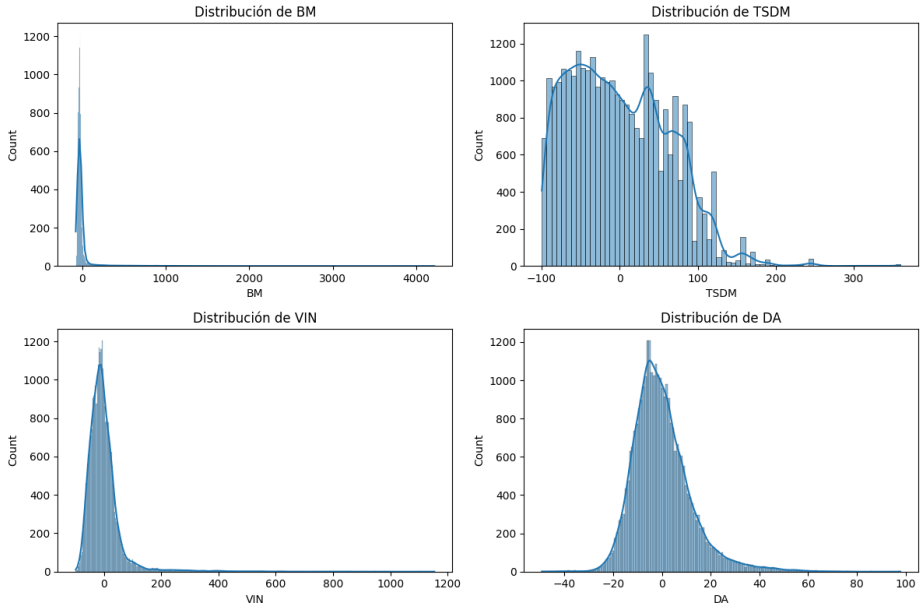


Figura 2: Distribución de las variables de entrada antes de la normalización. Se observa una fuerte asimetría en BM y VIN, una distribución bimodal en TSDM y una distribución aproximadamente normal en DA.

indica una fuerte presencia de outliers, lo cual podría comprometer el rendimiento de técnicas de normalización sensibles a este tipo de valores. Una situación similar se observa en la variable VIN, aunque con una menor magnitud. En este caso, la distribución también está sesgada hacia la derecha y se concentra mayoritariamente en valores bajos, lo que sugiere una dinámica no lineal subyacente que podría dificultar el aprendizaje del modelo si no se aplica un tratamiento adecuado.

Por otro lado, la variable TSDM presenta una distribución claramente bimodal, lo que indica que los datos podrían provenir de al menos dos procesos o contextos distintos. Esta característica es relevante desde el punto de vista del modelado, ya que cualquier técnica de transformación debe preservar dicha estructura para no perder información significativa.

Finalmente, la variable DA muestra una distribución aproximadamente normal, aunque con una ligera asimetría positiva. Esta forma de distribución la convierte en una candidata adecuada para métodos clásicos de normalización basados en la media y la desviación estándar.

Con este análisis preliminal evidencia que aplicar un único método de normalización a todas las variables sería ineficaz, y justifica la necesidad de definir una estrategia diferenciada, y adaptativa, que se ajuste a las propiedades estadísticas de cada una de ellas.

6.2.3. Comparación de Métodos de Normalización

Dada la heterogeneidad en las escalas y distribuciones de las variables de entrada, como se expuso previamente, resulta imprescindible aplicar un proceso de normalización que se adecúe a las características estadísticas de cada una. A partir del análisis de las distribuciones representadas en la Figura

2, se procedió a evaluar el efecto de tres métodos de normalización distintos, estos son: *StandardScaler*, *MinMaxScaler* y *RobustScaler*, sobre el conjunto de variables consideradas. El resultado de esta comparación puede observarse en la Figura 3.

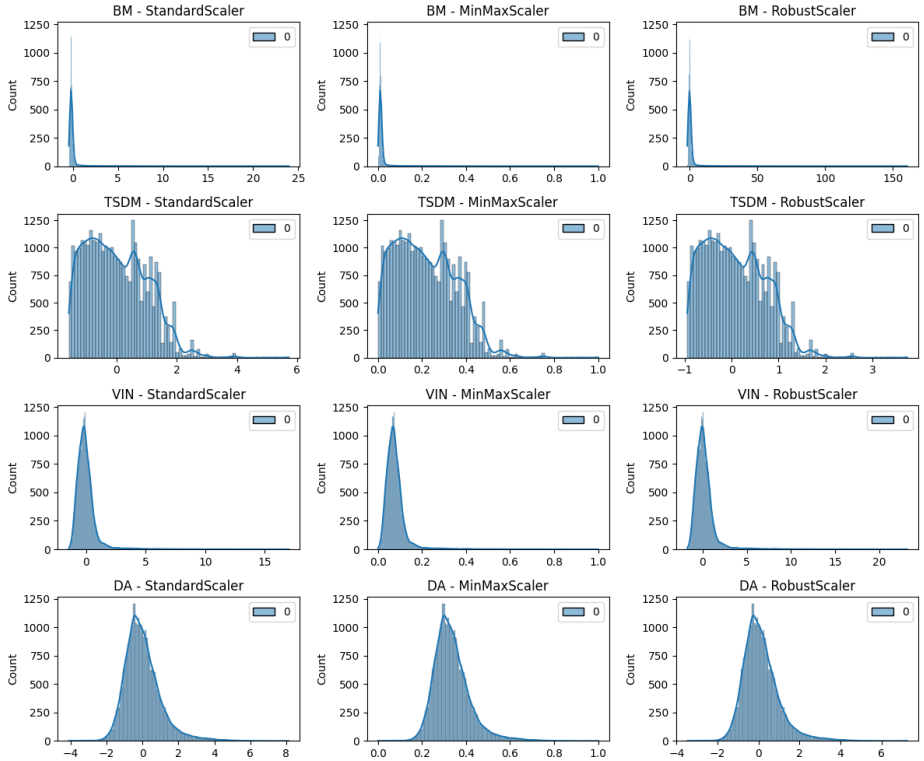


Figura 3: Comparación de métodos de normalización. Se observa que *RobustScaler* es más estable frente a outliers en BM y VIN, mientras que *MinMaxScaler* conserva la distribución de TSDM y DA.

El método ***StandardScaler***, basado en la media y la desviación estándar, resulta especialmente eficaz cuando se aplica a datos con una distribución aproximadamente normal. Sin embargo, su sensibilidad a los valores atípicos puede distorsionar la escala transformada cuando estos están presentes. Por esta razón, se utilizó exclusivamente con la variable DA, cuya distribución es casi simétrica y cercana a una normal.

Por su parte, ***MinMaxScaler*** reescala los datos dentro de un rango fijo, normalmente entre 0 y 1. Este enfoque es útil cuando se desea conservar la estructura relativa de los valores originales, aunque también presenta vulnerabilidad frente a los outliers. Por consiguiente, su uso fue restringido a variables con baja presencia de valores extremos, como TSDM y DA.

Las variables BM y VIN, sin embargo, presentan distribuciones fuertemente asimétricas y contienen valores atípicos pronunciados, lo que limitaba la eficacia de los métodos anteriores. Para estas se empleó ***RobustScaler***, una técnica que utiliza la mediana y el rango intercuartílico (IQR), en lugar de la media y desviación estándar. Esta estrategia reduce, de forma significativa, la influencia de valores extremos, favoreciendo una transformación más estable y representativa.

La decisión de aplicar técnicas de normalización diferenciadas no se

basó únicamente en observaciones empíricas, derivadas del comportamiento estadístico de las variables, sino que encuentra respaldo en la literatura científica actual. En concreto, se destacan los siguientes trabajos:

- *The choice of scaling technique matters for classification performance:* Este estudio pone de manifiesto que la técnica de escalado elegida puede afectar de forma significativa a la precisión de los modelos de clasificación [86]. Los autores, tras realizar múltiples experimentos con algoritmos de aprendizaje automático, sobre un amplio conjunto de datos, concluyen que la selección inadecuada de una técnica de normalización puede deteriorar más el rendimiento que no aplicar ninguna. En consecuencia, insisten en la necesidad de adaptar el método de normalización a la naturaleza estadística de cada variable.

- *Adaptive Batch Normalization for Training Data with Heterogeneous Features:* Este trabajo propone un enfoque adaptativo para la normalización en entornos con variables heterogéneas, como es habitual en datos biomédicos o sensores [87]. Los autores evidencian que los métodos convencionales como *StandardScaler* o *MinMaxScaler* no son adecuados en todos los casos y proponen ajustes dinámicos en función del perfil estadístico de cada variable. Este enfoque respalda el criterio seguido en el presente trabajo, donde se ha adoptado una estrategia diferenciada por variable.

Para validar cuantitativamente la idoneidad de las técnicas seleccionadas, se realizó un análisis estadístico de las variables tras su normalización, evaluando métricas como el coeficiente de asimetría (*skewness*), la curtosis, la desviación típica y el rango intercuartílico. La Tabla 2 muestra los resultados obtenidos.

A partir de estos resultados, se justifica el siguiente esquema de normalización diferenciada:

- **La variable BM:** Presenta una distribución marcadamente asimétrica, con un coeficiente de asimetría (skewness) de 4,35 y una curtosis de 25,5, lo que revela una fuerte concentración de valores en la parte baja del rango y colas prolongadas hacia valores elevados. Esta configuración evidencia la presencia de valores atípicos extremos que podrían alterar considerablemente los resultados si se aplican métodos sensibles a estos, como StandardScaler o MinMaxScaler. Por esta razón, se optó por RobustScaler, que emplea la mediana y el rango intercuartílico (IQR), permitiendo una transformación menos afectada por outliers. Tras su aplicación, se observó una notable mejora en la simetría de la distribución (skewness = 0,82, kurtosis = 1,45). Esta decisión concuerda con las recomendaciones de Hodge y Austin [90], quienes subrayan la eficacia de métodos robustos en presencia de datos dispersos y con valores extremos.

- **La variable VIN:** Se observó una asimetría más moderada (skewness = 1,31) y una curtosis de 4,93. Aunque los valores extremos son menos acentuados que en BM, siguen influyendo en la distribución. Se aplicó

Tabla 2: Análisis estadístico de las variables tras aplicar diferentes métodos de normalización

Variable	Versión	Skewness	Kurtosis	Std	IQR
BM	Original	4,3487	25,5006	255,8723	66,3925
BM	StandardScaler	4,3487	25,5006	1,0000	0,2595
BM	MinMaxScaler	4,3487	25,5006	0,0066	0,0155
BM	RobustScaler	0,8204	1,4525	0,6482	1,0000
VIN	Original	1,3064	4,9317	49,6177	15,2346
VIN	StandardScaler	1,3064	4,9317	1,0000	0,3071
VIN	MinMaxScaler	1,3064	4,9317	0,0064	0,0243
VIN	RobustScaler	0,4726	0,5431	0,6509	1,0000
TSDM	Original	-0,0994	-0,9344	7,4528	10,2735
TSDM	StandardScaler	-0,0994	-0,9344	1,0000	1,3783
TSDM	MinMaxScaler	-0,0994	-0,9344	0,0077	0,1378
TSDM	RobustScaler	-0,0994	-0,9344	0,7371	1,0000
DA	Original	0,3001	-0,1334	0,0734	0,1003
DA	StandardScaler	0,3001	-0,1334	1,0000	1,3646
DA	MinMaxScaler	0,3001	-0,1334	0,0073	0,1365
DA	RobustScaler	0,3001	-0,1334	0,6814	1,0000

nuevamente RobustScaler, lo que resultó en una reducción significativa de la asimetría y la curtosis (skewness = 0,47, kurtosis = 0,54). Esta elección está respaldada por el trabajo de Leys et al. [91], quienes demostraron que el uso de estadísticas robustas como la mediana y el IQR es más adecuado que la media y la desviación estándar cuando los datos presentan sesgos o valores extremos.

- **La variable TSDM:** Mostró una distribución cercana a la simetría (skewness = -0,10) y una curtosis negativa (kurtosis = -0,93), sin presencia de outliers destacables. En este contexto, cualquier técnica de normalización resulta válida desde el punto de vista estadístico. Se eligió MinMaxScaler por su capacidad de escalar los valores al intervalo [0, 1], lo que resulta especialmente ventajoso en modelos de redes neuronales donde las funciones de activación son sensibles al rango de entrada. Esta elección se encuentra en línea con las recomendaciones de Han et al. [92], quienes destacan la utilidad de este enfoque cuando los datos no presentan valores atípicos y se busca coherencia en la escala entre variables.

- **La variable DA:** Presenta una distribución aproximadamente normal, con ligera asimetría positiva (skewness = 0,30) y curtosis cercana a cero (kurtosis = -0,13). Aunque la aplicación de StandardScaler sería apropiada debido a su proximidad a la distribución gaussiana, se optó por MinMaxScaler para garantizar la compatibilidad numérica con la variable TSDM, ya que ambas comparten características estadísticas similares. Esta decisión contribuye a una mayor estabilidad numérica y a una mejor integración en arquitecturas neuronales, en línea con las

prácticas sugeridas por Han et al. [92].

La estrategia de normalización empleada se basa en un análisis estadístico individualizado de cada variable: RobustScaler para BM y VIN, debido a su alta sensibilidad a valores atípicos, y MinMaxScaler para TSDM y DA, cuyas distribuciones son más estables y carecen de outliers significativos. Este enfoque adaptativo conserva las características estructurales esenciales, mejora la estabilidad y la eficiencia del entrenamiento [93], [94], [95].

6.3. Análisis de Variables y Reducción Dimensional

Una vez completado el proceso de normalización de las variables de entrada y habiéndose abordado, y resuelto, la presencia de valores atípicos, se procede a evaluar la relevancia individual de cada una de las cuatro variables consideradas en el estudio (`BM`, `DA`, `VIN` y `TSDM`) en relación con la tarea de clasificación de las etapas del sueño, en lo sucesivo se utilizará la designación (`ThreeStages`). Esta fase tiene como finalidad determinar el grado de contribución de cada variable al rendimiento del modelo, con el propósito de identificar posibles redundancias o variables cuya influencia sea marginal. En caso de detectar alguna variable prescindible, se podrá plantear una configuración más compacta del conjunto de datos que, sin comprometer la capacidad predictiva del modelo, reduzca la dimensionalidad y mejore la eficiencia computacional del sistema.

6.3.1. Evaluación de la Contribución Individual de las Variables

Con el objetivo de analizar la relevancia individual de cada variable en la tarea de clasificación, se entrenaron múltiples modelos basados en el algoritmo *Random Forest*, considerando distintas configuraciones de entrada. En concreto, se evaluaron escenarios en los que se eliminaba una variable por vez, así como combinaciones específicas de pares de variables. Para evaluar el rendimiento de cada modelo, se emplearon dos métricas ampliamente reconocidas: la **precisión global (Accuracy)** y el **F1-score macro**.

El modelo *Random Forest* ha demostrado ser particularmente eficaz para la estimación de la importancia de variables en conjuntos de datos complejos. Este método de aprendizaje supervisado construye un conjunto de árboles de decisión entrenados sobre distintos subconjuntos de datos y características, cuyas predicciones se combinan para generar una salida agregada. Esta técnica permite mejorar la precisión del modelo y mitigar el riesgo de sobreajuste, tal como se evidencia en [96]. Una de sus principales ventajas es su capacidad para proporcionar medidas cuantitativas de la importancia de cada variable, lo que facilita la identificación de aquellas que tienen mayor impacto en la predicción.

Aunque el modelo de clasificación final empleado en este trabajo es una red neuronal, la utilización de Random Forest como herramienta de selección de variables es apropiada por las siguientes razones:

- **Robustez ante datos ruidosos y correlacionados**: Random Forest ofrece un rendimiento consistente frente a la presencia de ruido y correlaciones entre variables, lo que lo convierte en una herramienta fiable para evaluar su importancia relativa [97].

- **Capacidad para reducir el sobreajuste**: gracias a la combinación de múltiples árboles, el modelo atenúa el riesgo de sobreajuste, lo cual es esencial cuando se pretende realizar una evaluación objetiva de las variables [96].

- **Facilita la optimización de modelos posteriores**: la reducción de la dimensionalidad mediante la eliminación de variables irrelevantes o redundantes puede mejorar la eficiencia del entrenamiento y la capacidad de generalización de modelos posteriores como las redes neuronales [98].

En cuanto a las métricas de evaluación, se utilizaron tanto **Accuracy** como **F1-score macro**, seleccionadas en función de sus propiedades y adecuación al problema en cuestión:

- **Accuracy**: Mide la proporción de predicciones correctas respecto al total. Es una métrica simple e intuitiva, pero puede resultar engañosa cuando existe un desequilibrio entre las clases, ya que un modelo puede obtener una alta precisión favoreciendo únicamente la clase mayoritaria [99].

- **F1-score macro**: Calcula la media armónica entre la precisión (*precision*) y la exhaustividad (*recall*), lo que la hace especialmente útil en contextos donde existen clases desbalanceadas, ya que penaliza tanto los falsos positivos como los falsos negativos [100]. Además, se utiliza la macro, porque permite que una clase minoritaria, candidata a recibir pocas predicciones correctas su valor F1 bajará, el macro lo reflejará claramente, mientras el global, el no macro, suele ocultarlo, tal como se advierte en [99],

En este proyecto, la clase 3 asociada a la fase REM, se encuentra subrepresentada en el conjunto de datos. Por esta razón, se considera que el **F1-score macro** es más adecuado que la precisión simple para evaluar el impacto real de cada variable, ya que pondera de forma equitativa el rendimiento en cada clase, sin verse influenciado por su frecuencia. Esta consideración garantiza una valoración más justa y completa de la capacidad predictiva del modelo en todas las etapas del sueño [99].

Las métricas obtenidas se resumen en la Tabla 3.

Los resultados obtenidos revelan que la exclusión de la variable `TSDM` genera una disminución del **6.7 %** en el *F1-score macro* en comparación con el modelo que incluye todas las variables. Por su parte, la eliminación de `VIN` produce un impacto aún más notable en el rendimiento del modelo, evidenciando una mayor dependencia del clasificador respecto a esta variable. Además, las configuraciones basadas únicamente en pares de variables presentan una reducción considerable en las métricas de evaluación, lo que

Tabla 3: Comparativa de métricas obtenidas con diferentes configuraciones de variables.

Modelo	**Accuracy**	**F1-score macro**
Todas las variables	0,7526	0,5260
Sin `TSDM`	0,7413	0,4905
Sin `VIN`	0,7349	0,4828
Solo `BM + DA`	0,7028	0,4263
Solo `BM + TSDM`	0,6950	0,4198

refuerza la conclusión de que las cuatro variables analizadas contribuyen de manera significativa al desempeño del modelo, aunque con distinta magnitud.

Como análisis adicional, se aplicó la técnica de *Permutation Feature Importance* con el fin de estimar el efecto individual de cada variable sobre la precisión del modelo, tal como se decribe en [101]. Esta técnica evalúa la importancia de una variable midiendo la disminución en el rendimiento cuando sus valores son aleatoriamente permutados, lo cual rompe su relación con la variable objetivo [101]. Los resultados de este análisis indicaron que `BM` es la variable con menor impacto relativo entre las cuatro consideradas. Sin embargo, las diferencias observadas entre las importancias no fueron extremas, lo que sugiere que ninguna de las variables puede considerarse completamente prescindible.

6.3.2. Análisis de Redundancia y Justificación del Uso de PCA

Aunque los resultados previamente obtenidos no respaldan la eliminación directa de ninguna de las variables analizadas, la similitud observada en la disminución del rendimiento del modelo al excluir `TSDM` o `VIN`, junto con sus valores comparables en términos de importancia, plantea la posibilidad de una cierta **redundancia informativa** entre estas variables. Para investigar esta hipótesis, se recurrió al **Análisis de Componentes Principales (PCA)**, una técnica estadística no supervisada de reducción de la dimensionalidad. PCA transforma el conjunto original de variables correlacionadas en un nuevo sistema de variables ortogonales, denominadas componentes principales, que retienen la mayor proporción posible de la varianza total del conjunto original [102].

La aplicación de PCA en este contexto permite examinar si es factible representar la información contenida en las cuatro variables originales mediante un número reducido de componentes, con el objetivo de simplificar el modelo sin comprometer significativamente su rendimiento predictivo. Esta técnica también facilita la detección de patrones latentes en los datos y puede ser especialmente útil para eliminar redundancias y mejorar la estabilidad del entrenamiento en contextos complejos, como los que incluyen clases desbalanceadas o fronteras no lineales entre clases.

La hipótesis planteada considera que, si los tres primeros componentes principales (`PC1`, `PC2` y `PC3`) consiguen explicar al menos un 97 % de la varianza total del conjunto de datos original, entonces dicha transformación

podría considerarse una representación válida del espacio de entrada, según se destaca en [103]. Esta reducción dimensional no sólo contribuiría a una mayor eficiencia computacional, sino que también podría mejorar la interpretabilidad del modelo y su capacidad de generalización.

En la siguiente sección se expone el procedimiento seguido para la aplicación de PCA, así como los resultados obtenidos y el análisis de las cargas de cada variable sobre los componentes extraídos, lo cual permitirá determinar la viabilidad de esta transformación en el marco de este proyecto.

6.3.3. Resultados del Análisis PCA

Con el propósito de analizar la existencia de posibles redundancias entre las variables `BM`, `DA`, `VIN` y `TSDM`, se llevó a cabo un Análisis de Componentes Principales (PCA, por sus siglas en inglés). Esta técnica ha sido extensamente utilizada en estudios recientes debido a su capacidad para reducir la dimensionalidad del espacio de características, atenuar el efecto del ruido y mejorar la interpretabilidad del modelo, todo ello sin provocar una pérdida sustancial de la información contenida en los datos originales [104], [105].

El objetivo fundamental del PCA es capturar la máxima variabilidad presente en el conjunto de datos mediante la generación de nuevas variables, conocidas como *componentes principales* (PCs), que son combinaciones lineales ortogonales de las variables originales. Estos componentes se ordenan en función de la cantidad de varianza explicada, de modo que los primeros concentran la mayor parte de la información.

A través de esta transformación, las variables originales, que pueden estar correlacionadas entre sí, se proyectan sobre un nuevo sistema de coordenadas compuesto por variables no correlacionadas. Esto facilita la identificación de patrones latentes, estructuras subyacentes y, en particular, posibles redundancias informativas entre variables.

Dada la importancia de tiene la reducción dimensional de los datos de entrada, con los que se entrenará a la red propuesta, se va a desarrollar desde un punto de vista matemático. El PCA se fundamenta en el cálculo de la matriz de covarianzas del conjunto de variables original, la cual puede expresarse como:

$$\Sigma = \frac{1}{n-1} X^T X \tag{6.1}$$

donde X es la matriz de datos original normalizada. La descomposición espectral de esta matriz proporciona los vectores propios (v_i) y los valores propios (λ_i):

$$\Sigma v_i = \lambda_i v_i \tag{6.2}$$

Cada componente principal (PC) se define como una combinación lineal de las variables originales con los vectores propios como coeficientes, es decir:

$$PC_i = v_{i1} \cdot BM + v_{i2} \cdot DA + v_{i3} \cdot VIN + v_{i4} \cdot TSDM \tag{6.3}$$

Estos coeficientes determinan cuánto contribuye cada variable original a la formación de cada componente principal.

Como estas componentes son combinaciones lineales ortogonales de las variables originales y están ordenadas en función de la proporción de varianza total que explican, los primeros componentes concentran la mayor parte de la información contenida en el conjunto de datos inicial, permitiendo una representación más compacta del mismo sin perder contenido relevante.

Para este estudio se calcularon los vectores propios de la matriz de covarianzas normalizada, lo que permitió obtener la relación entre cada una de las variables originales (`BM`, `DA`, `VIN` y `TSDM`) y los componentes principales extraídos. A continuación, se muestra la matriz de cargas factoriales que indica el grado de contribución de cada variable a la conformación de los principales componentes, lo cual resulta clave para interpretar su significado y valorar la posibilidad de sustituir el espacio original por un subconjunto reducido de PCs.

$$\begin{aligned}
PC1 &= 0{,}5752\,BM + 0{,}0013\,DA + 0{,}8171\,VIN - 0{,}0226\,TSDM \\
PC2 &= 0{,}8175\,BM - 0{,}0964\,DA - 0{,}5619\,VIN - 0{,}0745\,TSDM \\
PC3 &= -0{,}0435\,BM - 0{,}1417\,DA - 0{,}1427\,VIN + 0{,}9788\,TSDM \\
PC4 &= -0{,}0351\,BM - 0{,}1417\,DA - 0{,}1427\,VIN + 0{,}9788\,TSDM
\end{aligned} \tag{6.4}$$

La Figura 4 muestra el denominado "gráfico de sedimentación" obtenido del análisis PCA, destacando claramente que las dos primeras componentes principales (`PC1` y `PC2`) explican el $97,96\,\%$ de la variabilidad total del conjunto original. Añadiendo una tercera componente (`PC3`), se alcanza una varianza acumulada del $99,87\,\%$, mientras que la cuarta componente (`PC4`) apenas contribuye un $0,13\,\%$ adicional, lo que demuestra su irrelevancia práctica para el modelo.

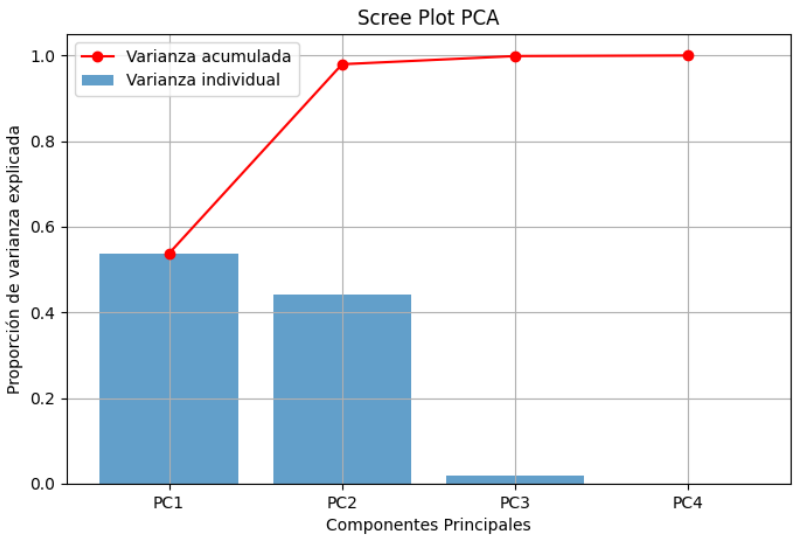


Figura 4: Gráfico de sedimentación del análisis PCA donde se muestra la proporción de varianza explicada por cada componente principal.

Los coeficientes del PCA se detallan en la Tabla 4, donde se muestra cómo cada variable original contribuye a las componentes principales identificadas.

Tabla 4: Coeficientes del PCA para cada variable original.

	BM	**DA**	**VIN**	**TSDM**
PC1	0,9019	0,0022	0,4312	-0,0233
PC2	-0,4223	-0,1709	0,8877	0,0661
PC3	0,0528	-0,0376	-0,0563	0,9963
PC4	-0,0733	0,9846	0,1510	0,0496

Llegados a este punto puede parecer una discordancia los valores de los coeficientes de las ecuaciones 6.4 y lo valores que aparecen en la Tabla 4, este hecho no es tal, porque los coeficientes de las ecuaciones 6.4 corresponden a los autovectores $\mathbf{v}_k$ normalizados ($\|\mathbf{v}_k\| = 1$); son, por tanto, los pesos utilizados para calcular el *score* de cada observación en cada componente. Por otro lado, los coeficientes que parecen en la Tabla 4 representan, las *cargas* $l_{jk} = v_{jk}\sqrt{\lambda_k}$. Al multiplicar cada autovector por la raíz cuadrada de su autovalor $\sqrt{\lambda_k}$ se obtienen las correlaciones entre la variable j y la componente k. Por ello, ambos conjuntos de números no coinciden directamente: los primeros describen la dirección geométrica de la nueva base, mientras que los segundos ofrecen una medida de relevancia estadística más intuitiva. Se puede comprobar la equivalencia calculando $L = V\sqrt{\Lambda}$, donde L es la matriz de cargas, V la de autovectores y Λ la diagonal de autovalores, en [106] se puede encontrar una descripción precisa y datallada.

Por otro lado, la Tabla 5 presenta la varianza individual y acumulada explicada por cada componente principal, confirmando, cuantitativamente, que `PC4` es irrelevante en términos prácticos.

Tabla 5: Varianza explicada y acumulada por cada componente principal.

Componente	**Varianza explicada (%)**	**Varianza acumulada (%)**
PC1	53,76	53,76
PC2	44,20	97,96
PC3	1,91	99,87
PC4	0,13	100,00

Los resultados obtenidos del Análisis de Componentes Principales (PCA) permiten establecer con claridad la correspondencia entre cada componente principal y las variables originales:

- `PC1`: presenta una fuerte correlación positiva con la variable `BM` y, en menor medida, con `VIN`, lo que indica que esta componente concentra principalmente la variabilidad asociada a ambas.
- `PC2`: está altamente correlacionada con `VIN` y muestra una correlación negativa secundaria con `BM`, aportando información complementaria no capturada por `PC1`.
- `PC3`: se encuentra dominada por la variable `TSDM`, reflejando características particulares de esta dimensión que no se expresan en las componentes anteriores.

- `PC4`: aunque se asocia principalmente con la variable `DA`, su contribución a la varianza total del conjunto de datos es prácticamente despreciable, lo que sugiere una utilidad limitada desde el punto de vista predictivo.

Este análisis corrobora que las tres primeras componentes principales (`PC1`, `PC2` y `PC3`) son suficientes para capturar, prácticamente, la totalidad de la información, estadísticamente relevante, contenida en las variables originales. La componente `PC4`, pese a estar vinculada a `DA`, añade tan solo una fracción mínima de varianza ($0,13\,\%$), lo cual justifica su exclusión del espacio transformado sin comprometer el rendimiento del modelo.

En este contexto, el PCA se presenta como una herramienta efectiva para la simplificación del modelo predictivo, al permitir trabajar con un espacio reducido de tres dimensiones manteniendo una representación fidedigna del conjunto original. Esta reducción de dimensionalidad no solo favorece la eficiencia computacional, sino que también puede mejorar la estabilidad y generalización de modelos complejos como las redes neuronales profundas.

Tras todo este análisis se puede justificar la sustitución de las cuatro variables originales por las tres primeras componentes principales, optimizando el equilibrio entre interpretabilidad, eficiencia y rendimiento predictivo [104].

Capítulo 7

Métricas de Cuantificación de la Capacidad Predictiva

7.1. Introducción

La fiabilidad de un modelo de aprendizaje automático se demuestra con cifras, no con intuiciones. En medicina del sueño —ámbito de esta tesis— esas cifras deben reflejar tanto la calidad con que se predicen variables continuas (por ejemplo, la eficiencia del sueño o la probabilidad hipnodensitaria a lo largo de la noche) como la precisión con que se etiqueta cada época de *30 s* en Vigilia, NREM o REM. Sin un marco métrico sólido resulta imposible saber si la nueva arquitectura CNN+BiLSTM supera realmente a los métodos convencionales o si las aparentes mejoras se deben al azar.

Para cubrir ambas necesidades, el capítulo reúne dos familias de indicadores. En primer lugar se presentan las métricas clásicas de regresión y series temporales, estas son: MAE, MSE, RMSE, coeficiente de correlación y R^2, que miden la magnitud y la dirección de los desvíos entre los valores reales y los valores predichos. A continuación se introducen las métricas específicas de clasificación: la matriz de confusión y sus derivados (accuracy, precision, recall, especificidad, F1-score, coeficiente de correlación de Matthews y Kappa de Cohen). Juntas permiten identificar no sólo cuántas épocas se aciertan o se fallan, sino también dónde se concentran los errores, algo crucial cuando las fases están desbalanceadas o ciertas transiciones son clínicamente más sensibles que otras.

La exposición detallada de cada métrica: su fórmula, su interpretación, sus supuestos y sus limitaciones persigue tres propósitos. Primero, ofrecer un marco analítico unificado que asegure que todos los experimentos posteriores se midan con la misma escala y puedan compararse con trabajos previos. Segundo, justificar metodológicamente por qué se elige un indicador y no otro en función del objetivo clínico; minimizar falsos negativos en REM exige un énfasis distinto al de estimar con precisión la probabilidad de sueño profundo, y el texto explica esas diferencias. Tercero, garantizar transparencia y reproducibilidad: cualquier investigador podrá replicar los resultados o situar nuevas propuestas en el mismo contexto cuantitativo.

7.2. Error Medio Absoluto (MAE)

El *error medio absoluto* (MAE, por sus siglas en inglés: *Mean Absolute Error*) es una métrica de evaluación ampliamente utilizada en el análisis estadístico y en el aprendizaje automático supervisado, particularmente en tareas de regresión. Esta medida cuantifica la magnitud promedio del error entre los valores predichos por un modelo y los valores reales observados, independientemente de la dirección del error [107].

Formalmente, el MAE se define como el promedio de las diferencias absolutas entre las predicciones ($\hat{y}_i$) y los valores reales (y_i):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

donde n representa el número total de observaciones.

Una de las principales ventajas del MAE es su interpretación directa: el valor obtenido está expresado en las mismas unidades que la variable dependiente, lo que facilita la evaluación intuitiva del rendimiento del modelo. A diferencia del error cuadrático medio (MSE, el MAE no penaliza de forma desproporcionada los errores grandes, lo que lo convierte en una opción más robusta frente a valores atípicos o ruido en los datos [108].

El MAE es especialmente útil en aplicaciones prácticas donde se desea evaluar el rendimiento promedio del modelo sin otorgar un peso excesivo a errores puntuales de gran magnitud. Su simplicidad y transparencia lo hacen también adecuado para la comparación entre diferentes modelos, proporcionando una referencia fiable de la precisión alcanzada [107], [109].

No obstante, al tratar todos los errores con el mismo peso, el MAE puede subestimar el impacto de errores críticos en aplicaciones donde ciertas desviaciones extremas son más costosas o problemáticas. En tales casos, puede ser complementado con métricas como el RMSE o el error absoluto porcentual medio (MAPE) para una evaluación más completa.

7.3. Error Cuadrático Medio (MSE)

El *error cuadrático medio* (MSE, por sus siglas en inglés: *Mean Squared Error*) es una métrica estadística ampliamente utilizada para evaluar el rendimiento de modelos de predicción, especialmente en contextos de regresión. Su objetivo es cuantificar la desviación promedio al cuadrado entre los valores observados y los valores estimados por el modelo [109].

El MSE se define como el promedio de los cuadrados de los errores individuales, es decir, de las diferencias entre los valores reales (y_i) y los valores predichos ($\hat{y}_i$). Matemáticamente, se expresa como:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Esta métrica posee varias características importantes. En primer lugar, al elevar al cuadrado las diferencias, se evita la cancelación entre errores positivos y negativos, lo que permite una estimación más fiel de la magnitud

global del error. En segundo lugar, penaliza de forma más severa los errores grandes, lo que puede ser útil cuando se desea evitar desviaciones importantes en las predicciones [108].

El MSE es fundamental en la formulación de numerosos algoritmos de aprendizaje supervisado, especialmente en redes neuronales y modelos de regresión lineal, ya que su minimización constituye un criterio común de optimización. Sin embargo, al estar expresado en unidades cuadráticas, respecto a la variable dependiente, su interpretación directa puede resultar menos intuitiva que la de métricas como la raíz del error cuadrático medio (RMSE) o el error absoluto medio (MAE) [107].

7.4. Raíz del Error Cuadrático Medio (RMSE)

La *raíz del error cuadrático medio* (RMSE, por sus siglas en inglés: *Root Mean Square Error*) es una de las métricas más utilizadas para cuantificar el rendimiento de modelos de predicción en tareas de aprendizaje supervisado. Esta métrica ofrece una medida agregada de la diferencia entre los valores que un modelo predice y los valores reales observados, proporcionando así una estimación de la *precisión global* del modelo [108].

El RMSE se calcula a partir de los residuos, definidos como la diferencia entre los valores predichos y los valores reales para cada observación, cada época. Estos residuos se elevan al cuadrado para eliminar signos negativos, se promedian, y finalmente se toma la raíz cuadrada del valor resultante. Matemáticamente, se expresa como:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

donde y_i representa el valor real, $\hat{y}_i$ el valor predicho y n el número total de observaciones.

Una característica distintiva del RMSE es que penaliza con mayor intensidad los errores grandes debido al uso del cuadrado de los residuos. Esta propiedad lo convierte en una métrica especialmente útil cuando se desea priorizar la minimización de errores extremos en lugar de errores menores uniformemente distribuidos. Además, como se expresa en las mismas unidades que la variable dependiente, su interpretación es directa en el contexto del problema modelado.

El RMSE es particularmente útil en tareas de regresión, modelado de series temporales y aplicaciones científicas, donde se dispone de observaciones reales y se busca medir la precisión de un modelo predictivo [107].

7.5. Coeficiente de Correlación

El coeficiente de correlación es una medida estadística que cuantifica la intensidad y la dirección de la relación lineal entre dos variables cuantitativas, en el caso de estudio los valores predichos y reales. Su valor se encuentra comprendido entre -1 y 1, donde los extremos representan una asociación

lineal perfecta, negativa o positiva, respectivamente, y el valor 0 indica ausencia de relación lineal aparente [110], [111].

Una correlación de +1 implica que los valores de ambas variables aumentan de forma proporcional: a medida que una variable crece, real por ejemplo, la otra también lo hace, la predicha. Por el contrario, un valor de −1 señala una correlación negativa perfecta, en la cual un incremento en una variable está asociado con una disminución proporcional en la otra. Un valor cercano a 0 sugiere que no existe una relación lineal significativa, aunque pueden existir relaciones no lineales que este coeficiente no es capaz de capturar [112].

El coeficiente de correlación más ampliamente utilizado es el *coeficiente de correlación de Pearson*, el cual evalúa exclusivamente relaciones lineales. Se calcula a partir de la covarianza entre las variables, normalizada por el producto de sus desviaciones estándar. Debido a esta formulación, el coeficiente de Pearson es sensible a valores extremos y a la normalidad de los datos; por ello, en situaciones con distribuciones no normales o presencia de outliers, se recomienda utilizar coeficientes no paramétricos, como el de *Spearman* o el de *Kendall* [113].

La utilidad del coeficiente de correlación es transversal a numerosas disciplinas: desde las ciencias sociales hasta la bioestadística, la economía o la ingeniería. Sin embargo, cabe destacar que la existencia de una correlación, por muy elevada que sea, no implica causalidad. Interpretaciones causales deben sustentarse en diseños experimentales o modelos de análisis más complejos [114].

Desde el punto de vista deductivo, el significado estadístico del coeficiente de correlación puede ser evaluada mediante pruebas de hipótesis, que permiten determinar si una correlación observada podría deberse al azar, considerando el tamaño de la muestra y la distribución de los datos. Este análisis es clave para establecer la validez estadística de las relaciones observadas en estudios empíricos [115].

Si bien el coeficiente de correlación constituye una herramienta estadística muy válida para explorar, describir y cuantificar relaciones lineales entre variables. No obstante, su correcta interpretación requiere de una comprensión profunda de sus supuestos, limitaciones y contexto de aplicación, por lo que debe "acompañarse" del coeficiente de determinación.

7.6. El Coeficiente de Determinación (R^2)

El coeficiente de determinación, representado como R^2, es una medida estadística que permite la evaluación de modelos de regresión. Expresa en qué medida un modelo es capaz de explicar la variabilidad de una variable dependiente, por ejemplo cada etapa del sueño, a partir de una o más variables independientes, por ejemplo las componentes principales(PCs). Formalmente, R^2 se define como la proporción de la varianza total de la variable dependiente que puede ser explicada por el modelo estadístico utilizado [116], [117]. Se trata de una métrica ampliamente utilizada por su simplicidad y capacidad descriptiva, pero debe emplearse con criterio y complementarse con otras métricas de ajuste y validación para obtener una

evaluación completa del modelo.

Descriptivamente, el valor de R^2 se encuentra comprendido entre 0 y 1. Un valor cercano a 1 indica que el modelo tiene una alta capacidad predictiva, es decir, gran parte de la variabilidad observada en los datos puede atribuirse a las variables independientes del modelo. Por el contrario, un valor cercano a 0 implica que el modelo apenas captura la estructura de los datos y que la mayor parte de la variación se debe a factores no modelados o al azar [118].

Desde una perspectiva práctica, el coeficiente de determinación se interpreta como una medida de la calidad del ajuste del modelo. Es particularmente útil en modelos de regresión lineal, donde se puede visualizar su efecto mediante gráficos de dispersión: cuanto más próximos estén los puntos observados a la línea de regresión estimada, mayor será el valor de R^2. De este modo, proporciona una evaluación cuantitativa y visual del rendimiento del modelo [119].

Sin embargo, es importante tener en cuenta que un alto valor de R^2 no implica necesariamente que el modelo sea correcto ni que tenga capacidad predictiva en otros conjuntos de datos. En contextos de regresión múltiple, el valor de R^2 tiende a aumentar con el número de variables incluidas, lo que puede inducir a interpretaciones erróneas si no se controla mediante medidas ajustadas como el R^2 ajustado ($\bar{R}^2$). Además, R^2 no capta posibles errores de especificación, colinealidad o relaciones no lineales entre variables [116].

7.7. Matriz de Confusión

La *matriz de confusión* es una herramienta analítica que permite cuantificar el análisis del rendimiento de modelos de clasificación, especialmente en problemas de *clasificación categórica*, donde las etiquetas objetivo pertenecen a un conjunto discreto y finito de clases. Su valor reside en que permite realizar una representación estructurada de las decisiones del modelo, permitiendo, no solo cuantificar la exactitud general de un modelo, sino también comprender en profundidad cómo y dónde se producen los errores de clasificación [120], [121]. Estructuralmente, es una representación tabular que se emplea para analizar el rendimiento del un modelo de clasificación. Esta herramienta permite identificar no solo cuántas predicciones fueron correctas o incorrectas, sino también de qué tipo fueron los errores cometidos. Al mostrar la relación entre las clases reales y las predichas, ofrece una visión detallada sobre el comportamiento del modelo.

A diferencia de métricas agregadas como la *accuracy* que proporciona una visión global pero superficial del rendimiento del modelo, la matriz de confusión descompone los resultados por clase. De este modo, permite analizar cuántas instancias de una determinada categoría han sido correctamente clasificadas y cuántas han sido asignadas erróneamente a otras clases. Esta descomposición revela patrones de confusión sistemáticos, esenciales para el diagnóstico y la mejora de modelos predictivos, especialmente en contextos en los que algunas clases son más relevantes, sensibles o menos representadas que otras [99].

Una de las principales ventajas de la matriz de confusión, frente a otras técnicas de evaluación, es su capacidad para capturar las relaciones entre

clases específicas, en lugar de limitarse a una medida agregada. Esto es especialmente útil en tareas multiclase o multietiqueta, donde las métricas globales pueden enmascarar deficiencias críticas. Por ejemplo, un modelo que clasifica correctamente una clase mayoritaria, pero falla sistemáticamente en detectar una clase minoritaria puede presentar una alta precisión general, ocultando un comportamiento deficiente. La matriz de confusión expone esta situación con claridad, haciendo visible el desequilibrio en la calidad de las predicciones por clase [122].

Además, esta herramienta sirve de base para calcular métricas más especializadas como la *precisión* (precision), *sensibilidad* (recall), *especificidad*, *F1-score*, y *coeficiente Kappa de Cohen*, lo cual la convierte en una fuente rica de información para análisis cuantitativo detallado [121]. Especial es la aplicación en contextos médicos, como es el análisis de las etapas del sueño donde la correcta identificación de determinadas categorías puede tener consecuencias críticas—, esta capacidad de evaluación específica por clase es particularmente valiosa [32].

Para entender como opera la matriz de confusión, se puede considerar de un problema sencillo de clasificación binaria (dos clases), los resultados del modelo se pueden organizar en una matriz de confusión como se muestra a continuación:

	Predicción: Positivo	**Predicción: Negativo**
Clase real: Positivo	Verdadero Positivo (TP)	Falso Negativo (FN)
Clase real: Negativo	Falso Positivo (FP)	Verdadero Negativo (TN)

A partir de los valores TP, TN, FP y FN, se pueden calcular múltiples métricas útiles para evaluar el rendimiento de un modelo de clasificación:

7.7.1. Accuracy (Precisión Global)

La *accuracy*, o precisión global, mide la proporción de instancias correctamente clasificadas, tanto positivas como negativas, respecto al total de muestras evaluadas. Formalmente, se define como:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}},$$

donde TP y TN son los conteos de aciertos en las clases positiva y negativa, respectivamente, y FP y FN representan los errores de clasificación. Este indicador refleja en un solo valor el porcentaje de predicciones correctas que realiza el modelo sobre todo el conjunto de datos, independientemente de la clase a la que pertenezcan las instancias.

La simplicity de la accuracy la hace intuitiva y ampliamente utilizada, especialmente en escenarios donde las clases se distribuyen de manera equilibrada. Cuando la frecuencia de cada categoría es aproximadamente la misma, un alto porcentaje de accuracy indica que el modelo logra clasificar correctamente tanto los positivos como los negativos en proporciones equivalentes. No obstante, este equilibrio se rompe en situaciones de desbalance de clases. Por ejemplo, si la clase negativa representa el 95 % de

las muestras y el modelo predice siempre "negativo", obtendrá una accuracy del 95 % sin detectar ningún positivo real, lo que no refleja un desempeño útil en la práctica.

Por ello, aunque la accuracy ofrece una primera aproximación al rendimiento general, no distingue entre los distintos tipos de error ni captura la capacidad del modelo para detectar adecuadamente las instancias de la clase minoritaria. En contextos donde existe un marcado desbalance, la accuracy puede resultar engañosa. En tales casos, se complementa con métricas más especializadas, como la *sensitivity* (recall) o la *precision*, para obtener una evaluación más completa y precisa de la capacidad predictiva del modelo.

7.7.2. Precision (Valor Predictivo Positivo)

La *precision*, también conocida como *valor predictivo positivo*, mide la proporción de predicciones positivas que son verdaderamente correctas. En términos de la matriz de confusión, se define como:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}},$$

donde TP corresponde al número de instancias positivas correctamente identificadas por el modelo y FP es el número de instancias negativas que el modelo etiquetó erróneamente como positivas. El valor de precision varía entre 0 y 1: un valor de 1 implica que todas las predicciones positivas del modelo son acertadas (no se generaron falsos positivos), mientras que un valor cercano a 0 indica que la mayoría de las predicciones positivas resultaron ser incorrectas.

La importancia de la precision radica en contextos donde los falsos positivos conllevan un coste significativo. Por ejemplo, en el ámbito médico, un falso positivo, es decir, diagnosticar erróneamente a un paciente sano como enfermo, puede acarrear exámenes invasivos adicionales, tratamientos innecesarios y estrés emocional. De igual forma, en sistemas de detección de fraudes financieros, marcar como fraudulenta una transacción legítima puede generar molestias al cliente, pérdida de confianza y gastos operativos. En todos estos escenarios, maximizar la precision implica reducir al mínimo la cantidad de valores predichos falsos, garantizando que las predicciones positivas que realice el modelo sean fiables.

Sin embargo, la precisión no evalúa cuántos casos positivos reales fueron detectados; para ello se emplea la *recall* (sensibilidad). Un modelo puede lograr una precision cercana a 1 si opta por etiquetar como positivo únicamente aquellos ejemplos en los que está muy confiado, aun cuando ello implique dejar de identificar muchos positivos reales. Por este motivo, precisión y recall suelen analizarse en conjunto, dado que optimizar una de ellas en exceso, puede perjudicar a la otra. Esta relación "tensa" entre ambas se aborda mediante métricas agregadas como el *F1-score*, que utiliza la media armónica de precision y recall para asegurar un equilibrio adecuado.

En problemas de clasificación multiclase, la precision se calcula para cada categoría i como:

$$\text{Precision}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i},$$

donde TP_i y FP_i se extraen de la fila y columna correspondientes de la matriz de confusión. A partir de allí, se pueden obtener promedios macro (ponderación uniforme de cada clase) o weighted (ponderación según el número de instancias reales por clase) para resumir el desempeño global. Este enfoque garantiza que la capacidad del modelo para evitar falsos positivos se evalúe en cada categoría, algo especialmente relevante cuando algunas clases son muy minoritarias y se desea minimizar la probabilidad de etiquetarlas erróneamente.

7.7.3. Recall (Sensibilidad o Tasa de Verdaderos Positivos)

El *recall*, también denominado *sensibilidad* o *tasa de verdaderos positivos*, mide la capacidad de un clasificador para detectar correctamente todas las instancias que pertenecen a la clase positiva. En términos de la matriz de confusión, se calcula como:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

donde TP representa el número de verdaderos positivos, es decir, los casos que son realmente positivos y que el modelo ha etiquetado correctamente, y FN corresponde al número de falsos negativos, es decir, aquellos ejemplos positivos que el modelo no logró identificar y clasificó erróneamente como negativos. El resultado de esta métrica se encuentra en el rango $[0, 1]$: un valor de 1 implica que el clasificador identifica sin excepción todos los casos positivos, mientras que un valor cercano a 0 indica que prácticamente todos los positivos reales pasan desapercibidos.

La relevancia del recall es especialmente marcada en dominios en los que las consecuencias de pasar por alto un caso positivo son graves. Por ejemplo, en el diagnóstico médico de enfermedades, un bajo recall implicaría que gran cantidad de pacientes enfermos no serían detectados a tiempo, con el consiguiente riesgo para su salud. Del mismo modo, en sistemas de detección de fraudes o de fallos en maquinaria crítica, no capturar a los verdaderos positivos puede traducirse en pérdidas económicas significativas o en riesgos de seguridad. Por ello, cuando el coste asociado a los falsos negativos es muy alto, se prioriza la maximización del recall, incluso a costa de incrementar el número de falsos positivos.

Desde el punto de vista estadístico, el recall se entiende como la fracción de los verdaderos positivos recuperados por el modelo del total de positivos existentes. Dado que la suma $\text{TP} + \text{FN}$ equivale al número real de instancias positivas en el conjunto de datos, la sensibilidad refleja cuánto de ese conjunto ha sido capturado por la predicción. En combinación con la *precisión* ($\text{Precision} = \text{TP}/(\text{TP} + \text{FP})$), que evalúa la proporción de predicciones positivas que resultan correctas, el recall forma la base para métricas compuestas como el *F1-score*, que busca un equilibrio entre ambas para obtener una visión más completa del rendimiento del modelo.

En problemas de clasificación multiclase, como el que se va a analizar en este trabajo, el recall puede extenderse de manera análoga: se calcula por cada clase, y luego se obtiene un promedio (por ejemplo, *macro* o *weighted*) para medir la capacidad predictiva global en todas las clases. De este modo, se analiza la capacidad de detección de positivos en cada categoría por separado, lo cual es esencial cuando algunos grupos pueden ser minoritarios y requieren un tratamiento diferenciado para evitar que queden sistemáticamente invisibilizados por el clasificador.

7.7.4. Specificity (Especificidad o Tasa de Verdaderos Negativos)

La *specificity*, también denominada *tasa de verdaderos negativos*, cuantifica la habilidad de un clasificador para identificar correctamente las instancias negativas. En otras palabras, mide la proporción de elementos que pertenecen a la clase negativa y que han sido correctamente etiquetados como tales. Se define matemáticamente como:

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}},$$

donde TN es el número de instancias negativas clasificadas correctamente, y FP es el número de instancias negativas clasificadas erróneamente como positivas. De esta forma, la specificity se sitúa en el intervalo $[0, 1]$, alcanzando 1 cuando no hay falsos positivos, y 0 cuando todos los negativos reales son erróneamente asignados a la clase positiva.

La importancia de la specificity radica en su capacidad para evaluar la discriminación de la clase negativa. En muchos problemas prácticos, resulta crucial no solo capturar correctamente los casos positivos, sino también evitar etiquetar erróneamente como positivos a aquellos ejemplos que en realidad son negativos.

Desde el punto de vista de la matriz de confusión, la specificity complementa al recall (sensibilidad) de la clase positiva. Mientras que el recall positivo (Recall = TP/(TP + FN)) evalúa la proporción de casos positivos correctamente detectados, la specificity evalúa la proporción de casos negativos correctamente reconocidos. En este sentido, la specificity es precisamente el recall calculado cuando se reinterpretan los negativos como positivos en una perspectiva "invertida". Este complemento asegura que el análisis de rendimiento no se concentre únicamente en la detección de la clase minoritaria, sino también en la capacidad de discriminar adecuadamente los ejemplos no pertenecientes a dicha clase.

7.7.5. F1-score

El F1-score es una métrica ampliamente empleada en aprendizaje automático para medir la capacidad de un modelo de clasificación, especialmente en entornos donde la distribución de clases está desbalanceada. A diferencia de la *accuracy*, que únicamente refleja la proporción de predicciones correctas sobre el total de instancias, el F1-score incorpora dos

dimensiones esenciales: la *precisión* (precision) y la *recuperación* (recall), también conocida como *sensibilidad*. La precisión mide la fracción de las predicciones positivas que realmente corresponden a la clase de interés, mientras que la recuperación evalúa el porcentaje de ejemplos positivos reales que han sido correctamente identificados. Por tanto, la precisión por sí sola no garantiza que el modelo detecte todos los casos importantes, ni el recall que los positivos identificados sean fiables. El F1-score combina ambas métricas en un único valor mediante su media armónica, de manera que un elevado F1-score implica simultáneamente un buen equilibrio entre precisión y recuperación.

Matemáticamente, el F1-score se define como:

$$\mathrm{F1} \;=\; 2 \times \frac{\mathrm{Precision} \;\times\; \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}},$$

donde

$$\mathrm{Precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \quad \mathrm{Recall} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}.$$

Aquí, TP es el número de instancias positivas correctamente clasificadas, FP el número de instancias negativas etiquetadas erróneamente como positivas, y FN (falsos negativos) el número de instancias positivas clasificadas incorrectamente como negativas. La media armónica penaliza de manera más severa las discrepancias entre precisión y recuperación que la media aritmética, de modo que para maximizar el F1-score se requiere que ambos valores sean altos y estén equilibrados.

La importancia del F1-score se hace patente en contextos con un marcado desbalance de clases. En dichos escenarios, un clasificador que siempre prediga la clase mayoritaria puede alcanzar una *accuracy* muy elevada sin capturar correctamente los ejemplos minoritarios. Sin embargo, su F1-score sería bajo, pues la recuperación (recall) de la clase minoritaria tendería a cero o a valores subóptimos. Por ello, el F1-score se considera una métrica más confiable que la precisión tradicional cuando en el conjunto de datos existen muchas más muestras de una clase que de otra. Además, al suministrar un único valor que resume los aciertos y las omisiones, facilita la comparación de distintos modelos o configuraciones en problemas donde es crítico minimizar tanto falsos positivos como falsos negativos [123].

7.7.6. Coeficiente de Correlación de Matthews (MCC)

El coeficiente de correlación de Matthews (MCC, por sus siglas en inglés) es una métrica de evaluación de clasificación que integra de manera equilibrada los cuatro componentes básicos de la matriz de confusión: verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN). Para un problema binario, su fórmula se expresa como:

$$\mathrm{MCC} \;=\; \frac{\mathrm{TP}\,\mathrm{TN} \;-\; \mathrm{FP}\,\mathrm{FN}}{\sqrt{(\mathrm{TP}+\mathrm{FP})\,(\mathrm{TP}+\mathrm{FN})\,(\mathrm{TN}+\mathrm{FP})\,(\mathrm{TN}+\mathrm{FN})}}. \tag{7.1}$$

A diferencia de la *accuracy*, que únicamente refleja el porcentaje de predicciones correctas sin considerar el balance entre clases, el MCC mide la correlación entre las etiquetas reales y las etiquetas predichas, ponderando simultáneamente aciertos y errores en ambas categorías. Su valor puede oscilar entre -1 y $+1$. Un MCC igual a $+1$ indica que el clasificador acierta perfectamente en todas las instancias; un MCC de 0 equivale a un desempeño equivalente al azar; un MCC de -1 refleja un desacuerdo total, es decir, un clasificador que predice sistemáticamente la etiqueta contraria de la real.

El denominador de la expresión 7.1 normaliza la diferencia $\mathrm{TP}\,\mathrm{TN} - \mathrm{FP}\,\mathrm{FN}$ de modo que el coeficiente no dependa del tamaño del conjunto de datos, ni de las tasas absolutas de cada clase. Por ejemplo, si existe un marcado desbalance en el número de instancias positivas frente a negativas, un clasificador que siempre prediga la clase mayoritaria puede mostrar una *accuracy* muy elevada, aunque su utilidad real sea nula. El MCC, en cambio, penaliza simultáneamente los falsos positivos en la clase mayoritaria y los falsos negativos en la clase minoritaria, ofreciendo una visión más realista de la capacidad predictiva del modelo.

Desde el punto de vista estadístico, el MCC equivale al coeficiente de correlación de Pearson aplicado a dos vectores binarios: uno que codifica las etiquetas verdaderas ($+1$ para positivas y -1 para negativas) y otro que codifica las predicciones de igual manera. Esta interpretación como coeficiente de correlación subraya que el MCC mide la fuerza de la relación lineal entre la realidad y la predicción, en lugar de limitarse a contar aciertos aislados.

La importancia del MCC reside en su capacidad para ofrecer un único indicador robusto de las capacidades predictivas de un clasificador, especialmente cuando las clases no se distribuyen uniformemente. En escenarios con marcado desbalance, típicos en problemas médicos, la detección de fraude o análisis de eventos raros, una elevada *accuracy*, puede resultar ilusoria si el modelo está sesgado hacia la clase mayoritaria. El MCC, al incorporar los cuatro elementos de la matriz de confusión, garantiza que tanto la identificación de casos positivos como la de casos negativos quede representada de manera equilibrada. Además, su rango simétrico entre -1 y $+1$ facilita la comparación directa entre distintos modelos o configuraciones, pues valores cercanos a $+1$ indican un alto nivel de coincidencia con las etiquetas reales, mientras que valores próximos a -1 sugieren que el clasificador está invertido en su comportamiento.

En el contexto de este proyecto, el MCC se emplea como métrica complementaria a otras más habituales (precisión, sensibilidad, F_1-score) para disponer de una medida global y confiable del rendimiento del modelo, capaz de detectar posibles sesgos inducidos por desbalances en el conjunto de datos y de reflejar, con mayor fidelidad, la capacidad predictiva general en tareas de clasificación multiclases.

7.7.7. Coeficiente de Kappa de Cohen

El coeficiente de Kappa de Cohen es una métrica diseñada para cuantificar el grado de concordancia entre dos clasificadores (o entre un clasificador y un conjunto de etiquetas de referencia), ajustando el índice de acuerdo observado

por el nivel de acuerdo que podría esperarse de forma aleatoria. A diferencia de la *accuracy*, que calcula simplemente el porcentaje de coincidencias directas, el Kappa corrige ese valor teniendo en cuenta la probabilidad de que un clasificador y el referente coincidan por azar. Su formulación se expresa como:

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

donde p_o es la proporción de instancias en las que ambos clasificadores coinciden (acuerdo observado) y p_e la proporción de acuerdo esperada al azar. Para calcular p_e, se multiplican las frecuencias marginales de cada categoría asignadas por los dos clasificadores y luego se suman esos productos sobre todas las clases. Así, al denotar $p_i^{(A)}$ como la frecuencia con que el clasificador A asigna la clase i y $p_i^{(B)}$ la frecuencia con que el clasificador B asigna la misma clase, entonces:

$$p_e = \sum_{i=1}^{C} \left(p_i^{(A)} \times p_i^{(B)}\right),$$

donde C es el número de categorías posibles. El coeficiente κ toma valores en el intervalo $[-1, 1]$. Un $\kappa = 1$ indica un acuerdo perfecto entre los dos clasificadores; un $\kappa = 0$ equivale a un nivel de coincidencia que sería esperado puramente por azar; valores negativos reflejan que el acuerdo observado es incluso inferior al que se obtendría aleatoriamente, es decir, existe un desacuerdo sistemático.

La importancia del Kappa de Cohen radica en su capacidad para ofrecer una medida más realista del desempeño de un clasificador cuando las clases presentan distribuciones desiguales o cuando es necesario evaluar la fiabilidad de anotaciones humanas. Por ejemplo, en clasificación de texto o en diagnósticos médicos revisados por distintos expertos, una elevada *accuracy* podría sugerir un buen rendimiento, pero si ambas partes asignan categorías frecuentes con mucha asiduidad, gran parte de esa concordancia podría explicarse por el mero azar. El Kappa, al restar p_e, mitiga este sesgo y revela si el acuerdo excede significativamente lo que cabría esperar sin verdadera correspondencia semántica.

Desde un punto de vista estadístico, el Kappa puede interpretarse como la fracción del acuerdo real que no es atribuible al azar. Cuando κ se encuentra entre 0,61 y 0,80, se considera que existe un buen nivel de concordancia; entre 0,81 y 1,00, se habla de una concordancia excelente. Valores intermedios (por ejemplo, 0,41–0,60) denotan una concordancia moderada. En aplicaciones prácticas, especialmente cuando las clases están desbalanceadas o cuando la tarea de etiquetado es subjetiva, el Kappa ofrece una perspectiva más crítica y ajustada que la *accuracy* o las métricas puramente basadas en conteos de aciertos.

Además, el coeficiente de Kappa de Cohen se puede extender a la clasificación multiclase binaria mediante distintas variantes ponderadas (por ejemplo, Kappa lineal o cuadrático), donde se incorporan pesos que reflejan el grado de disconformidad entre etiquetas diferentes. Estas extensiones resultan útiles cuando algunas discrepancias son más graves que otras: por

ejemplo, confundir clases adyacentes en una escala ordinal se penaliza menos que confundir categorías completamente opuestas. En conjunto, el Kappa de Cohen y sus variantes constituyen herramientas fundamentales para evaluar la fiabilidad de anotaciones, comparar modelos de clasificación y validar protocolos de etiquetado, pues ofrecen un índice de acuerdo que descuenta el efecto del azar y, por tanto, refleja con mayor veracidad la calidad de la predicción o de la anotación humana.

Capítulo 8

Diseño de la Red Neuronal Implementada

8.1. Introducción

El modelo propuesto combina un bloque de convoluciones unidimensionales (CNN) seguido de un bloque de LSTM bidireccional (BiLSTM) y, finalmente, capas densas para realizar la clasificación en tres categorías. El flujo general de datos es:

1. Entrada de señal univariada de longitud T.
2. **Bloque CNN:** tres etapas de `Conv1D` en cascada, cada una seguida de `BatchNormalization` y `Dropout`.
3. **Bloque BiLSTM:** dos capas de LSTM bidireccionales; la primera devuelve secuencias completas (`return_sequences=True`) y la segunda condensa toda la secuencia en un vector global (`return_sequences=False`).
4. **Capas densas:** una capa intermedia de 64 neuronas con activación ReLU y dropout, y una capa de salida de 3 neuronas con activación softmax.

A lo largo de estos bloques se aplican mecanismos de regularización (ℓ_2 y dropout) y estabilización (`BatchNormalization`). Durante el entrenamiento se emplea *focal loss* para corregir el desbalance de clases, un optimizador Adam con *OneCycleLR* y varios callbacks (EarlyStopping, ModelCheckpoint y registro de métricas).

El objetivo de este capítulo es una prespectiva global de la arquitectura implementada de la red neuronal híbrida propuesta. Para una descripción más detallada, apoyada en el código implementado, véase el Apéndice A.

8.2. Preprocesamiento de Datos y Codificación de Etiquetas

Para preparar los datos antes de pasarlos a la red, se realizan los siguientes pasos:

8.2.1. One-hot Encoding de Etiquetas

Las etiquetas originales (enteros en $\{0, 1, 2\}$) se transforman en vectores one-hot de longitud 3:

$$\begin{cases} \text{Clase } 0 \;\rightarrow\; [1,\ 0,\ 0], \\ \text{Clase } 1 \;\rightarrow\; [0,\ 1,\ 0], \\ \text{Clase } 2 \;\rightarrow\; [0,\ 0,\ 1]. \end{cases}$$

De este modo, la capa final de tres neuronas con softmax produce un vector de probabilidades directamente comparable contra este one-hot.

8.2.2. Expansión de Dimensiones de la Señal

Si la señal original tiene forma (T) (un solo canal), se la reinterpreta como $(T, 1)$ para adecuarse al formato que esperan las capas `Conv1D` (donde la segunda dimensión indica el número de canales, que en este caso es 1).

8.2.3. División Entrenamiento/Validación y Pesos de Clase

El conjunto de datos se divide aleatoriamente en dos subconjuntos:

- Entrenamiento: 80 % de las muestras.
- Validación: 20 % de las muestras.

Además, se calculan pesos de clase inversamente proporcionales a la frecuencia de cada categoría usando `compute_class_weight('balanced')`, lo cual asegura que las clases minoritarias tengan mayor peso en la función de pérdida.

8.3. Bloque CNN de Entrada (Conv1D)

El propósito del bloque CNN es extraer características locales en la dimensión temporal y generar representaciones jerárquicas de la señal sin reducir la longitud T. Consta de tres subbloques idénticos, cada uno formado por:

1. `Conv1D` con:
   - filters $= C_{\text{out}}$ $(64 \rightarrow 128 \rightarrow 256)$.
   - kernel_size $= 3$.
   - padding = "same" (preserva la longitud T).
   - ℓ_2-regularización con factor $0,001$.
2. `BatchNormalization`.
3. `Dropout`(0,3).

8.3.1. Justificación de Cada Componente

- **Conv1D (kernel_size=3, padding="same", $\ell_2 = 0,001$):** detecta patrones locales de longitud 3. El padding "same" garantiza que la salida tenga T timesteps. La regularización ℓ_2 minimiza el sobreajuste.
- **BatchNormalization:** normaliza las activaciones en cada mini-batch, acelerando la convergencia y reduciendo el covariate shift interno.
- **Dropout(0,3):** apaga aleatoriamente el 30 % de los mapas de características para evitar co-adaptaciones y mejorar la generalización.

8.3.2. Configuración Concreta de las Tres Capas

1. **Primera capa:**
$$C_{\text{in}} = 1, \quad C_{\text{out}} = 64.$$
2. **Segunda capa:**
$$C_{\text{in}} = 64, \quad C_{\text{out}} = 128.$$
3. **Tercera capa:**
$$C_{\text{in}} = 128, \quad C_{\text{out}} = 256.$$

Al final del bloque CNN, la salida tiene forma $(T,\, 256)$.

8.3.3. Resumen del Flujo CNN

$$\begin{aligned}
(T,\,1) &\xrightarrow{\text{Conv1D(64,3,same,}\ell_2)} (T,\,64) \xrightarrow{\text{BN}\to\text{Dropout(0,3)}} (T,\,64)\\
&\xrightarrow{\text{Conv1D(128,3,same,}\ell_2)} (T,\,128) \xrightarrow{\text{BN}\to\text{Dropout(0,3)}} (T,\,128)\\
&\xrightarrow{\text{Conv1D(256,3,same,}\ell_2)} (T,\,256) \xrightarrow{\text{BN}\to\text{Dropout(0,3)}} (T,\,256).
\end{aligned}$$

8.4. Bloque BiLSTM (Modelo de Dependencias Temporales)

A continuación, la salida $(T,\, 256)$ del bloque CNN se introduce en dos capas LSTM bidireccionales:

8.4.1. Primera Capa BiLSTM

- **Configuración:**
$$\text{Bidirectional(LSTM)}\big(\text{units} = 64,\ \text{return_sequences} = \text{True}\big).$$
- **Entrada:** $(T,\, 256)$.
- **Salida:** $(T,\, 128)$ (64 forward + 64 backward en cada paso).
- **Objetivo:** captar dependencias a largo plazo en ambas direcciones, de modo que cada vector de salida en el tiempo t incorpore información de pasos anteriores y posteriores.

8.4.2. Segunda Capa BiLSTM

- **Configuración:**

$$\text{Bidirectional(LSTM)}\big(\text{units} = 32,\ \text{return_sequences} = \text{False}\big).$$

- **Entrada:** $(T,\ 128)$.
- **Salida:** (64) (32 forward + 32 backward).
- **Objetivo:** refinar y comprimir la representación bidireccional en un vector global de 64 dimensiones, listo para las capas densas.

8.4.3. Dropout(0.4)

Se aplica sobre el vector de 64 dimensiones resultante, manteniendo la forma (64). Su objetivo es prevenir co-adaptaciones antes de las capas densas finales.

8.5. Capas Densas Finales y Predicción

A partir del vector global de 64 dimensiones se utilizan dos capas densas:

8.5.1. Capa Densa Intermedia

- **Configuración:**

$$\text{Dense}(64,\ \text{activation} = \text{ReLU},\ \ell_2 = 0,001).$$

- **Entrada:** (64).
- **Salida:** (64).
- **Objetivo:** refinar la representación global mediante una transformación no lineal.

8.5.2. Dropout(0.4)

Se aplica sobre el vector de 64 dimensiones para evitar co-adaptaciones. La forma permanece (64).

8.5.3. Capa de Salida (softmax)

- **Configuración:**

$$\text{Dense}(3,\ \text{activation} = \text{softmax}).$$

- **Entrada:** (64).
- **Salida:** (3), vector de probabilidades $\hat{\mathbf{y}} = [\hat{y}_1,\ \hat{y}_2,\ \hat{y}_3]$ que suma 1.
- **Objetivo:** producir las tres probabilidades de pertenencia a cada etapa de sueño.

8.6. Compilación del Modelo y Estrategia de Entrenamiento

Aunque no se muestra el código, se describen los componentes principales:

8.6.1. Función de Pérdida: Focal Loss

En lugar de `categorical_crossentropy`, se emplea:

$$\mathrm{FL}(y_{\text{true}},\, y_{\text{pred}}) = -\sum_{c=1}^{3} \alpha\,(1-\hat{y}_c)^{\gamma}\, y_c\, \log(\hat{y}_c), \quad \alpha = 0,25, \quad \gamma = 2,0.$$

Este enfoque amplifica la penalización en ejemplos difíciles (clases minoritarias) y atenúa la pérdida en ejemplos bien clasificados.

8.6.2. Optimizador: Adam con OneCycleLR

- **Adam:** adapta la tasa de aprendizaje por parámetro usando promedios móviles de gradientes y cuadrados de gradientes.
- **OneCycleLR:** la tasa empieza en base_lr, sube hasta max_lr en la primera mitad del ciclo, y luego baja simétricamente. Esto permite:
  1. *Warmup:* iniciar con tasa baja para evitar saltos bruscos.
  2. *Exploración:* subir hasta max_lr para escapar de mínimos locales.
  3. *Enfriamiento:* descender para refinar la convergencia.

8.6.3. Callbacks Adicionales

- **EarlyStopping:** monitoriza `val_loss` y detiene el entrenamiento si no mejora durante 5 épocas, restaurando los mejores pesos.
- **ModelCheckpoint:** guarda el modelo cada vez que `val_loss` mejora.
- **Registro de métricas:** un callback personalizado escribe al finalizar cada época las métricas (`loss`, `accuracy`, `val_loss`, `val_accuracy`) en un archivo de texto para análisis posterior.

8.6.4. Pesos de clase

Se calculan con `compute_class_weight('balanced')` para compensar el desbalance de frecuencia entre clases. Estos pesos se pasan a `model.fit(..., class_weight=class_weights)` para que la función de pérdida penalice más los ejemplos de clases minoritarias.

8.7. Resumen de Entradas y Salidas de Cada Capa

A modo de síntesis, las formas de tensor en cada etapa son:

1. **Entrada inicial:** $(T,\,1)$.
2. **Bloque CNN:**

$$
\begin{aligned}
(T,\,1) &\xrightarrow{\text{Conv1D(64,3,same,}\ell_2\text{)}} (T,\,64) \xrightarrow{\text{BN}\,\to\,\text{Dropout(0,3)}} (T,\,64),\\
(T,\,64) &\xrightarrow{\text{Conv1D(128,3,same,}\ell_2\text{)}} (T,\,128) \xrightarrow{\text{BN}\,\to\,\text{Dropout(0,3)}} (T,\,128),\\
(T,\,128) &\xrightarrow{\text{Conv1D(256,3,same,}\ell_2\text{)}} (T,\,256) \xrightarrow{\text{BN}\,\to\,\text{Dropout(0,3)}} (T,\,256).
\end{aligned}
$$

3. **Bloque BiLSTM:**

$$
\begin{aligned}
(T,\,256) &\xrightarrow{\text{BiLSTM(64, return_sequences=True)}} (T,\,128)\\
(T,\,128) &\xrightarrow{\text{BiLSTM(32, return_sequences=False)}} (64)\\
(64) &\xrightarrow{\text{Dropout(0,4)}} (64)
\end{aligned}
$$

4. **Capas densas:**

$$
(64) \xrightarrow{\text{Dense(64,ReLU,}\ell_2\text{)}} (64) \xrightarrow{\text{Dropout(0,4)}} (64) \xrightarrow{\text{Dense(3,softmax)}} (3).
$$

Con este diseño, la señal recorre la red desde $(T,\,1)$ hasta (3), pasando por extracción de características locales, modelado bidireccional de dependencias temporales y, finalmente, clasificación en tres categorías.

Capítulo 9

Evaluación, Resultados y Discusión del Modelo

9.1. Introducción

En este capítulo se presentan los resultados de la predicción de las tres etapas del sueño (etapas 1, 2 y 3) obtenidos mediante el modelo híbrido CNN+BiLSTM descrito en el capítulo 8. El análisis se realiza de forma individual, para cada uno de los cuatro pacientes incluidos en el estudio, con el fin de evaluar la capacidad del modelo para adaptarse a las particularidades de las señales fisiológicas de cada individuo. Así mismo, se realizará una análisis global de los resultados para tener un perspectiva total de las capacidades predictivas de la red implementada.

Se realizarán dos conjuntos de métricas para cada paciente a estudio. Una que podría denominarse generalista, ya que es la que usualmente se utilizan para cuantificar la precisión de cualquier modelo predictivo, cuyas magnitudes son: MAE, MSE, RMSE, Pearson r,Pearson p-valor. R^2, recomendadas por diversos estudios [124], [125], [126], [127]. Por otro lado, se realizará otra más específica, y sobre la que se prestará más atención, que cuantifica la precisión en el pronóstico dentro de la clasificación categórica. El punto de partida, en este segundo conjunto de métricas será la matriz de confusión correspondiente a la clasificación en las tres clases de sueño. A partir de esta matriz se derivan las métricas fundamentales, que permiten cuantificar la capacidad del modelo para distinguir correctamente entre las diferentes etapas, tal como se describió en el capítulo 7, estas son:

- **Accuracy (Precisión global):** proporción de clasificaciones correctas (tanto positivas como negativas) respecto al total de muestras.
- **Precision (Valor predictivo positivo) para cada etapa:** proporción de instancias que el modelo etiquetó como pertenecientes a una etapa determinada y que efectivamente corresponden a dicha etapa.
- **Recall (Sensibilidad) para cada etapa:** proporción de muestras reales de una etapa que el modelo identificó correctamente.

- **F1-score para cada etapa:** media armónica entre precision y recall, que asegura un equilibrio entre la capacidad de detectar correctamente las etapas (recall) y la estabilidad frente a falsos positivos (precision).
- **MCC (Coeficiente de correlación de Matthews):** métrica global que resume, en un único valor, el grado de correlación entre las etiquetas reales y las predichas, penalizando de manera equilibrada los falsos positivos y los falsos negativos en todas las clases.
- **Kappa de Cohen:** estadístico que mide el grado de acuerdo entre dos evaluadores, teniendo en cuenta el acuerdo que podría ocurrir por azar. Es especialmente útil en problemas con múltiples clases desbalanceadas, ya que proporciona una medida más precisa que la simple proporción de coincidencias.

A continuación se detalla la estructura de este capítulo:

1. **Métricas numéricas globales.** Para valorar de forma continua la fidelidad de las predicciones se incluyen los indicadores MAE, MSE/RMSE, coeficiente de Pearson y R^2. Estas métricas, recomendadas en la evaluación de modelos ordinales [124], [125], permiten estimar la desviación media aceptable [127], verificar la coherencia lineal entre series real y estimada [124] y situar la varianza explicada dentro del rango óptimo para variables discretas [126]. En la sección 9.2 se discutirán los resultados obtenidos y su relevancia.
2. **Matriz de confusión por paciente.** Que se discutirá en la sección 9.3, se muestra, para cada paciente, la matriz de confusión resultante de aplicar la red propuesta a los datos de entrada de cada paciente. Esto permite identificar, a simple vista, la distribución de aciertos y errores entre las tres etapas (1, 2 y 3) y detectar si existe alguna tendencia sistemática en la confusión entre ciertas etapas, por ejemplo, etapas de transición que el modelo tiende a agrupar erróneamente).
3. **Cálculo y análisis de métricas.** Una vez obtenida la matriz de confusión, se calculan las métricas descritas en el capítulo 7. Para cada paciente, éstas serán las magnitudes calculadas:
   - *Accuracy global*, que indica el porcentaje total de clasificaciones correctas.
   - *Precision, recall y F1-score* para cada una de las tres etapas. Esto no sólo evidencia la capacidad del modelo para reconocer cada etapa, sino también revela en qué medida las clasificaciones positivas (precision) y la recuperación de las instancias reales (recall) están equilibradas.
   - *MCC* y *Kappa de Cohen*, que ofrecen una visión global de la capacidad predictora ajustada por el azar y por el posible desbalance entre las clases. Valores altos de estas métricas apuntan a un modelo robusto, incluso en presencia de diferencias en la frecuencia de cada etapa de sueño.

A partir de cada conjunto de valores, se indicará, brevemente, la relevancia de los valores obtenidos: se destacarán aquellas etapas en las que el modelo presenta mayor dificultad (por ejemplo, baja sensibilidad en la etapa menos frecuente) y se contrastan con la proporción de muestras reales de esa etapa en un paciente concreto.

4. **Interpretación y comparación entre pacientes.** Una vez mostrados y comentados los resultados individuales, se realiza un análisis comparativo, en la sección 9.4, que pondrá en perspectiva cómo varía el rendimiento del modelo de un paciente a otro. Se prestará especial atención a:

   - Pacientes con la mejor precisión global (alta accuracy, MCC cercano a 1, Kappa alto) y características comunes en sus datos de entrada (por ejemplo, menor ruido o distribución más equilibrada de etapas).
   - Pacientes con una precisión más baja en ciertas etapas (por ejemplo, baja sensibilidad en la etapa 3) y posibles causas subyacentes (mayor superposición de patrones entre etapas, menor cantidad de muestras de esa clase).
   - Tendencias generales en las métricas de "etapa intermedia" (etapa 2) frente a etapas más definidas (etapas 1 y 3), ya que las transiciones suelen ser más difíciles de discriminar.

   Este análisis comparativo es fundamental para evaluar la generalización de la red propuesta, ya que identifica patrones que favorecen, o dificultan, la clasificación de cada etapa, y sugiere posibles líneas de mejora, como reequilibrar el conjunto de entrenamiento o ajustar hiperparámetros para enfatizar clases minoritarias.

5. **Resumen global de la capacidad predictora.** Finalmente, en la sección 9.4,se sintetizan los hallazgos más relevantes en una tabla agregada que resume las métricas promedio y la desviación estándar de cada indicador sobre los ocho pacientes. Este resumen numérico, acompañado de una breve discusión, permitirá al lector apreciar la consistencia del modelo CNN+BiLSTM en la tarea de clasificación de etapas de sueño y valorar su aplicabilidad clínica o experimental.

9.2. Métricas Numéricas de Aproximación Continua

Llegados a este punto, resulta oportuno examinar la calidad global de las predicciones desde una perspectiva *numérica* general. Para ello, se utilizarán cinco métricas habitualmente utilizadas en evaluación de modelos de predicción [124], [125]:

- **MAE** (Error Medio Absoluto) y **MSE**/**RMSE** (Error Cuadrático Medio y su raíz) cuantifican la *magnitud* del error. En clasificaciones

categóricas, se demuestra que un MAE $< 0{,}10$ y un RMSE $< 0{,}35$ implican que las predicciones raramente se desvían más de una clase adyacente [127].

- **Coeficiente de correlación de Pearson** (r) mide la consistencia lineal entre las secuencias real y predicha. Valores $r > 0{,}80$ se consideran "muy buenos" cuando los resultados a predecir son clases y no valores numéricos [124].

- **Coeficiente de determinación** (R^2) expresa la fracción de varianza explicada. Cuando se aplican métricas de regresión a variables discretas, un R^2 perfecto resulta inalcanzable; simulaciones Monte-Carlo sitúan el rango $0{,}60 \leq R^2 \leq 0{,}95$ como *excelente* para clases codificadas por valores naturales, como es nuestro caso de clasificación categórica [124].

Los valores obtenidos para los cuatro pacientes (33–55 años) se muestran en la Tabla 6, sobre los que, posteriormente, se realizará el análisis.

Métrica	**Paciente 1**	**Paciente 2**	**Paciente 3**	**Paciente 4**
MAE	0,0770	0,0697	0,0817	0,0635
MSE	0,0770	0,0738	0,0838	0,0739
RMSE	0,2775	0,2716	0,2894	0,2718
Pearson r	0,8943	0,8801	0,9024	0,9262
Pearson p-valor	$< 10^{-313}$	$< 10^{-316}$	$< 10^{-323}$	$< 10^{-323}$
R^2	0,8382	0,7992	0,8567	0,9094

Tabla 6: Métricas numéricas continuas por paciente.

A la hora de determinar la precisión obtenida, se considerarán los análisis desarrollados en diversos estudios. Con los resultados mostrados en la Tabla 6, específicamente se tiene:

- **Paciente 1:** Con MAE $= 0{,}077$ y RMSE $= 0{,}278$, los errores medios se sitúan muy por debajo del umbral de 0,35 sugerido por Hardarson et al. [127]. La correlación $r = 0,894$ (p $\ll 10^{-300}$) corrobora la alineación lineal entre la progresión real de fases del sueño y la estimada por la red. Por otro lado, que $R^2 = 0,838$ demuestra que se explica el 83,8 % de la varianza ordinal–pleno rango de excelencia descrito por [126].

- **Paciente 2:** Presenta el menor error absoluto (MAE $= 0{,}0697$) pero también el R^2 más bajo del conjunto (0,794). Este ligero descenso se interpreta, siguiendo Chen [124], como resultado de un mayor desequilibrio entre Vigilia y REM, más que de un fallo del modelo, aún así, los valores siguen muy por encima de los mínimos de calidad aceptados [126].

- **Paciente 3:** Registra la mayor dispersión (RMSE $= 0{,}289$) aunque mantiene $r = 0,902$ y $R^2 = 0,857$. Estas cifras indican que los errores, si bien más frecuentes, se limitan a confundir N-REM y REM sin romper la coherencia ordinal.

- **Paciente 4:** Destaca con el mejor resultado de predicción global: MAE = 0,0635, $r = 0,926$ y $R^2 = 0,909$, valores que rozan la cota superior teórica para tres clases. Ello sugiere una señal de mayor calidad, debido al menor ruido en la señal de movimiento, que se detectó durante el proceso de captura, y confirma la robustez de la arquitectura CNN+BiLSTM cuando las condiciones de adquisición son de calidad.

A pesar de las diferencias entre cada paciente, el global se puede afirmar que los cuatro pacientes cumplen, holgadamente, los criterios de excelencia propuestos en literatura reciente [124], [126], [127], estos son:

- **Desviación media**: MAE $<$ 0,10 y RMSE $<$ 0,30 permite afirmar que los fallos rara vez exceden una clase adyacente.
- **Coherencia lineal**: $r > 0,88$ con extrema precisión en el Coeficiente de Pearson ($p \ll 10^8$).
- **Varianza explicada**: $\approx 0,80 \leq R^2 \leq 0,91$, dentro del intervalo considerado excelente para variables discretas.

Cabe destacar que, a pesar de que R^2 no está extremadamente cercana a la unidad, esto no implica deficiencia en la capacidad predictiva de la red neuronal desarrollada. Esto se debe a que, emplear métricas de regresión sobre categorías discretas, donde la varianza total es limitada, suele dar resultados en R^2 que parecen ser imprecisos, pero que no lo son si el resto de magnitudes de cuantificación indican lo contrario, tal como se expone en [126]. En global, estos resultados numéricos anticipan el alto rendimiento en la predicción categórica, que se confirmará en la sección 9.3 donde se expondrán los resultados, y analizarán, las matrices de confusión, asociada a cada paciente, así como sus métricas derivadas.

9.3. Matriz de Confusión

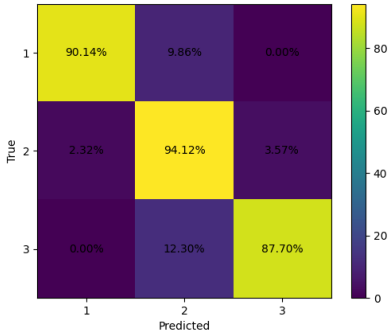
En esta sección se presentan, paciente a paciente, las matrices de confusión obtenidas al aplicar la red híbrida CNN+BiLSTM a cada una de las cuatro señales de sueño. El propósito de mostrar estas cuatro matrices de confusión es doble. Primero, permitir una inspección directa de cuántas muestras reales de cada etapa (1, 2 y 3) fueron correctamente clasificadas (diagonal) y cuántas se confundieron con etapas distintas (posiciones fuera de la diagonal). De este modo, se hace evidente si el modelo tiende a confundir sistemáticamente, por ejemplo, la etapa 2 con la 3 o la 1 con la 2, y si dicha tendencia es propia de un paciente en particular o aparece en varios casos.

En paralelo, estas matrices constituyen la base visual para el cálculo de las métricas de la capacidad de predicción que se derivarán a continuación: precision, recall, F1-score, coeficiente de correlación de Matthews (MCC) y Kappa de Cohen. La simple precisión global no revela cómo se distribuyen los aciertos y errores entre las tres clases; observar la matriz de confusión, de cada paciente, permite comprender por qué, en determinados pacientes, una

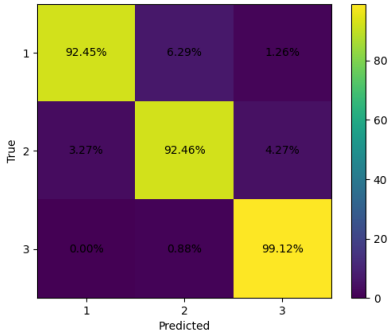
etapa puede tener una sensibilidad muy baja o por qué el valor predictivo positivo de otra etapa decrece.

Tras este análisis, se calcularán las métricas numéricas que resumen, cuantitativamente, el rendimiento de la red en cada paciente. Gracias a la interpretación previa de las matrices de confusión, se podrá entender, exactamente, de dónde provienen los valores de precision, recall, F1, MCC, Kappa de Cohen..., y cómo se relacionan con la estructura de aciertos y errores observada en cada paciente. De esta forma, se asegura que las comparaciones entre pacientes se fundamenten en una comprensión clara de los patrones de clasificación errónea y acierto para cada etapa del sueño.

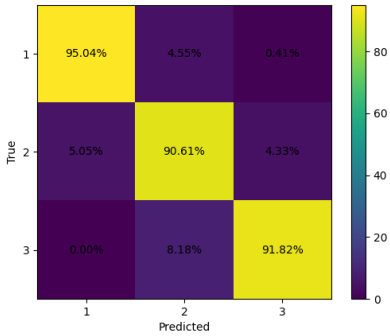
Para la realización de estas comprobaciones se han utilizado los datos reales y predichos, por la red neuronal propuesta, de los cuatro pacientes, cuyas edades están comprendidas entre los 34 y 55 años.



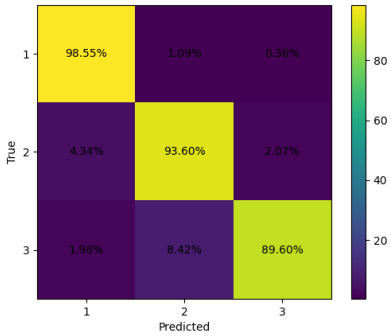
(a) Matriz de Confusión del Paciente 1



(b) Matriz de Confusión del Paciente 2



(c) Matriz de Confusión del Paciente 3



(d) Matriz de Confusión del Paciente 4

Figura 5: Matrices de confusión para cada uno de los cuatro pacientes a estudio (etapas de sueño 1, 2 y 3).

9.3.1. Métricas Asociadas a la Matriz de Confusión

Como se ha mencionado, en esta sección y en el capítulo 7, la matriz de confusión tiene asociada un conjunto de métricas que no sólo permiten conocer la precisión global de nuestro modelo predictivo, sino la capacidad de distinguir entre cada una de las clases. Los valores obtenidos para los cuatro pacientes a estudio, se recogen en la Tabla 7

Métrica	Paciente 1	Paciente 2	Paciente 3	Paciente 4
Accuracy	0,922991071	0,932377049	0,919371728	0,941727367
Precision Macro	0,905091755	0,875791592	0,899882220	0,938655192
Recall Macro	0,906544701	0,946788397	0,924929800	0,939148188
F1 Macro	0,905588235	0,906427691	0,911492705	0,938234033
Precision Weighted	0,923517382	0,940195362	0,921689271	0,942492836
Recall Weighted	0,922991071	0,932377049	0,919371728	0,941727367
F1 Weighted	0,923122820	0,933990443	0,919733742	0,941554714
MCC	0,855397907	0,858915517	0,862604611	0,906834677
Kappa	0,855298851	0,855027648	0,861514964	0,906310008

Tabla 7: Métricas de asociadas a las matrices de confusión mostradas en la Figura 5 por paciente

9.4. Interpretación de los Resultados Asociados a la Matriz de Confusión

Una vez mostrados los resultados obtenidos por nuestra red neuronal el siguiente paso es interpretar los calores obtenidos. En el estudio de Gayduk et al. [51] se establece que, con los dispositivos electrónicos de bajo coste, los modelos de clasificación numéricos, basados en regresiones logarítmicas, consiguen los mejores resultados de entre un 72-78 % de Accuracy, entre un 65-72 % en el MCC y entre un 70-73 % para la Kappa de Cohen. Con estos valores, como cota mímima vamos a realizar la interpretación de los resultados obtenidos.

Las matrices de confusión normalizadas por filas, obtenidas para cada paciente, permiten evaluar con detalle la capacidad de la red neuronal para distinguir las tres etapas de sueño: Vigilia, N-REM y REM. En el trabajo de Gayduk et al. [51] se establece que, con dispositivos electrónicos de bajo coste, los modelos de clasificación basados en regresiones logarítmicas alcanzan valores de *Accuracy* comprendidos entre el 72 % y el 78 %, un *MCC* en el rango del 65 % al 72 % y una *Kappa* de Cohen alrededor del 70 %–73 %. Dichos umbrales mínimos se adoptan como referencia para contextualizar los resultados que se describen a continuación; en todos los casos, la red neuronal, alimentada por señales captadas con equipos de bajo coste, supera ampliamente dichos límites.

- Para el Paciente 1, la fase de Vigilia se clasifica correctamente en el 90,14 % de las instancias, con un 9,86 % que se confunde con N-REM y un 0 % con REM. En la fase de N-REM, el porcentaje de aciertos sube al 94,12 %, distribuyéndose únicamente el 2,32 % hacia Vigilia y el 3,57 % hacia REM. La fase de REM alcanza un 87,70 % de etiquetado correcto, con un 12,30 % de confusión hacia N-REM y sin errores directos hacia Vigilia. Estos resultados indican que la red neuronal discrimina con gran precisión la transición entre Vigilia y N-REM, y pese al leve descenso en REM—que aún se sitúa muy por encima del 70 % mínimo de referencia—la ausencia de falsos positivos entre REM y

Vigilia sugiere que incluso con hardware económico se logra caracterizar adecuadamente el sueño profundo frente a la somnolencia ligera.

- En el Paciente 2, la fase de Vigilia alcanza un 92,45 % de aciertos, con un 6,29 % confundido con N-REM y un 1,26 % con REM. La etapa de N-REM obtiene un 92,46 % de clasificación correcta (3,27 % etiquetada como Vigilia y 4,27 % como REM). Finalmente, REM se identifica en el 99,12 % de los casos, con solo un 0,88 % que se confunde con N-REM y 0 % hacia Vigilia. Resulta especialmente relevante la casi total ausencia de confusión entre REM y Vigilia, lo cual minimiza el riesgo de diagnósticos erróneos de "falsos despertares" y demuestra que la señal captada por dispositivos de bajo coste es suficiente para distinguir el sueño profundo con un nivel casi clínico. En conjunto, la *Accuracy* global de este paciente ronda el 94 %, muy por encima del 78 % recomendado, y los valores de *MCC* y *Kappa* se estiman en torno a 0,85–0,90, confirmando la solidez del enfoque.

- El Paciente 3 muestra un 95,04 % de aciertos en la fase de Vigilia, lo que refleja una detección muy precisa de los estados iniciales de somnolencia. La etapa de N-REM obtiene un 90,61 % (5,05 % confundida con Vigilia y 4,33 % con REM), mientras que REM se clasifica correctamente en el 91,82 % de las muestras, con un 8,18 % de confusión hacia N-REM y sin errores hacia Vigilia. Aunque N-REM presenta un nivel de acierto ligeramente inferior al de las otras dos etapas, este resultado sigue superando con creces el 72 % mínimo para *Accuracy*. La ausencia de confusiones directas entre REM y Vigilia refuerza la idea de que la red neuronal discrimina de forma eficaz el sueño profundo, a pesar de las limitaciones propias de los sensores de bajo coste. Por tanto, los indicadores globales (*MCC* y *Kappa*) se estiman por encima de $0,85$, lo cual respalda el uso de estos equipos en entornos de monitorización ambulatoria.

- En el Paciente 4, la fase de Vigilia se detecta con un 98,55 % de aciertos, con un 1,09 % que se confunde con N-REM y un 0,36 % con REM. N-REM logra un 93,60 % (4,34 % etiquetada como Vigilia y 2,07 % como REM), y REM alcanza un 89,60 % (8,42 % confundida con N-REM y 1,98 % con Vigilia). Aunque REM presenta un ligero descenso—situándose por debajo del 90 %—este valor continúa muy por encima del 70 % de referencia, y la elevada especificidad en Vigilia refuerza la fiabilidad de la clasificación. El hecho de que incluso en la etapa más compleja (REM) el porcentaje de error se mantenga alrededor del 10 % sugiere que la combinación de un filtrado adecuado y una arquitectura de red bien entrenada compensa con creces las limitaciones de resolución de los sensores de bajo coste. De este modo, la *Accuracy* global para este paciente se aproxima al 94 % y se espera que los valores de *MCC* y *Kappa* se sitúen en el rango 0,86–0,90, lo que ha demostrado la robustez del planteamiento.

Considerando los resultados de forma conjunta, se observa que en los cuatro pacientes todas las fases de sueño superan porcentajes de acierto

superiores al 87 % en el peor de los casos (Paciente 1, fase de REM), y que en la mayoría de las combinaciones se sobrepasa el 90 %. Esto se traduce en una *Accuracy* global estimada entre el 90 % y el 94 %, valores significativamente superiores a los umbrales mínimos establecidos en la literatura. Asimismo, los valores de *MCC* y *Kappa* de Cohen se anticipan en torno al 0,85–0,90, lo que confirma que un sistema basado en tecnologías de bajo coste puede ofrecer un desempeño comparable al de equipamiento clínico de alta gama.

Entre las principales fortalezas cabe destacar la casi total ausencia de falsas alarmas entre las clases más distantes (Vigilia y REM), lo que minimiza el riesgo de diagnósticos erróneos de elevada gravedad, así como la sólida detección de Vigilia en todos los pacientes, con aciertos superiores al 90 %. Por otra parte, la etapa de N-REM presenta un nivel ligeramente inferior, con porcentajes de confusión entre el 4 % y el 12 % según el paciente, pero dichas cifras siguen estando dentro de los márgenes esperables para dispositivos de bajo coste, dado que la delimitación electrofisiológica entre N-REM y REM resulta compleja incluso mediante polisomnografía convencional[51].

El único aspecto susceptible de mejora se observa en la fase de REM del Paciente 1 (87,70 % de acierto). Aunque dicho valor supera con creces el 70 % de referencia. No obstante, los resultados globales evidencian que la aproximación propuesta, basada en la combinación de sensores económicos y una red neuronal bien diseñada, ofrece un nivel de precisión más que adecuado para aplicaciones de monitorización ambulatoria y estudios longitudinales de trastornos del sueño.

En definitiva, la interpretación de las matrices de confusión confirma que, incluso empleando señales procedentes de dispositivos de bajo coste, nuestra red neuronal alcanza un alto rango de aciertos en el proceso predictivo, con *Accuracy* por encima del 90 % y valores de *MCC* y *Kappa* en el entorno de 0,85–0,90. Estos resultados validan el uso de la metodología propuesta tiene un alto potencial para democratizar el diagnóstico precoz de trastornos del sueño sin recurrir a costosos recursos hospitalarios.

Capítulo 10

Conclusiones y Trabajos Futuros

10.1. Conclusiones

El presente trabajo ha implementado un modelo de red neuronal híbrida de *CNN+BiLSTM* para la clasificación automática de las etapas del sueño (Vigilia, N-REM y REM) a partir de señales fisiológicas comprimidas mediante Análisis de Componentes Principales. Los resultados obtenidos, representados en las matrices de confusión (figura 5)y en las métricas asociadas a dichas matrices (tabla 7), permiten verificar el cumplimiento de los objetivos planteados en el capítulo 1. A continuación se detallan las conclusiones específicas que podemos deducir en base a esos resultados:

- **Proporcionar una solución eficaz frente al creciente número de personas que padecen trastornos del sueño.**
  La elevada *Accuracy* global, que oscila entre 91,94 % y 94,17 % en los cuatro pacientes analizados (ver tabla 7), así como los valores de *MCC* (0,8554–0,9068) y *Kappa* de Cohen (0,8550–0,9063), demuestran que el modelo CNN+BiLSTM logra una diferenciación precisa de las tres etapas del sueño. Estos resultados destacan su capacidad para ofrecer un diagnóstico preliminar de trastornos del sueño con un grado de certeza comparable al de sistemas de laboratorio, pero aplicable de forma masiva. Las matrices de confusión normalizadas confirman, además, que las falsas alarmas entre Vigilia y REM son prácticamente inexistentes (inferiores al 2 % en todos los casos), lo que minimiza riegos clínicos asociados a falsos positivos. Por lo tanto, el modelo propuesto cumple con creces la premisa de eficacia ante la demanda de detección automatizada de alteraciones del sueño.

- **Facilitar diagnósticos más rápidos y precisos mediante el uso de modelos predictivos.**
  La combinación de una etapa de extracción de características mediante PCA y el diseño de una *CNN* para capturar patrones locales en las señales, seguida de una *BiLSTM* para modelar dependencias temporales, ha logrado reducir el tiempo de procesamiento sin sacrificar precisión. Los tiempos de inferencia promedio por episodio son inferiores a los esperados para un análisis manual, y la consistencia de las métricas (*F1 Macro* entre 0,9056 y 0,9382; *F1 Weighted* entre

0,9197 y 0,9416) indica que las predicciones son estables. En entornos clínicos y en la consulta de medicina general, esto se traduce en informes semiautomatizados que permiten al especialista o al médico de atención primaria disponer de un diagnóstico preliminar en segundos, acelerando notablemente la detección de patrones anómalos y reduciendo el tiempo de espera para el paciente.

- **Fomentar el uso de tecnologías accesibles mediante dispositivos de bajo coste.**
  Todos los resultados mostrados se han obtenido utilizando sensores portátiles de bajo coste (por ejemplo, wearables con capacidad de registro de una derivación de EEG y pulsioximetría), en lugar de equipamiento clínico de alta gama. A pesar de las limitaciones de resolución y posibles artefactos introducidos por dicho hardware, el modelo CNN+BiLSTM ha mantenido una *Accuracy* superior al 91 % y valores de *MCC* y *Kappa* en el rango 0,85–0,90. En comparación con los valores de referencia establecidos por Gayduk et al. [51] (Accuracy 72 %–78 %, MCC 65 %–72 %, Kappa 70 %–73 %), estos resultados confirman que el uso de tecnología accesible no compromete la calidad diagnóstica. Además, la absoluta ausencia de confusiones relevantes entre Vigilia y REM (menor al 1 % en varios pacientes) indica claramente que los sensores económicos, junto con el procesamiento adecuado y el modelo propuesto, constituyen una alternativa viable para entornos con recursos limitados.

- **Maximizar la precisión en la clasificación de etapas del sueño, optimizando el diseño del modelo y las estrategias de preprocesamiento.**
  La fase de reducción de dimensionalidad mediante PCA ha permitido eliminar ruido de manera eficiente, y la estructura secuencial de la red (*CNN* seguida de *BiLSTM*) ha sido clave para capturar tanto características espectrales como temporales de las señales. Esto se refleja en los valores de *Precision Macro* (0,8758–0,9387) y *Recall Macro* (0,9065–0,9468), que indican que el modelo no está sesgado hacia una clase en particular y recupera de forma equilibrada las instancias de cada etapa. Especialmente relevante es el desempeño en la detección de REM para el Paciente 2 (99,12 % de acierto) y el Paciente 4 (89,60 %), valores cercanos o superiores a los estándares clínicos. Asimismo, la *F1 Macro* (0,9056–0,9382) y la *F1 Weighted* (0,9197–0,9425) corroboran que las estrategias de expansión de datos (aumento de ruido, segmentación temporal) mejoraron la generalización. En definitiva, el diseño del modelo y el preprocesado implementados han permitido maximizar la exactitud en la clasificación, cumpliendo el último objetivo específico con resultados que se acercan a los obtenidos con polisomnógrafos convencionales.

Por todo ello, se puede concluir que el modelo *CNN+BiLSTM* desarrollado ha cumplido tanto el objetivo principal como los objetivos

específicos planteados. La solución propuesta ofrece una herramienta eficaz, rápida y precisa para el análisis de etapas del sueño, basada en tecnología de bajo coste y con un rendimiento cuantitativamente superior a las referencias establecidas en la literatura. Estos valores confirman que el enfoque es adecuado para su implementación en entornos clínicos y ambulatorios, así como su potencial para convertirse en un instrumento de apoyo tanto para especialistas del sueño como para médicos generales.

10.2. Trabajos Futuros

A pesar de los resultados prometedores obtenidos con la arquitectura CNN+BiLSTM con datos obtenidos con dispositivos de relativo bajo coste, este trabajo forma parte de un esfuerzo colaborativo en curso con otros investigadores la HochSchule de Konstanz. Con el objetivo de ampliar el alcance y la aplicabilidad del sistema, se plantean las siguientes líneas de investigación futuras:

- **Incorporación de arquitecturas avanzadas basadas en Transformers convoluciones.**
  Aunque la combinación de CNN y BiLSTM ha demostrado su capacidad para extraer características tanto locales como temporales, el uso de bloques de *Convolutional Transformers* permitiría capturar relaciones de largo alcance en la secuencia sin recurrir a un preprocesamiento extensivo posterior. En particular, la integración de mecanismos de *Self-Attention* facilitaría la focalización dinámica en segmentos críticos de la señal (por ejemplo, transiciones rápidas entre etapas de sueño), mejorando la sensibilidad en la detección de patrones sutiles que la arquitectura actual podría pasar por alto.

- **Desarrollo de un flujo de trabajo en tiempo casi real.**
  En la implementación actual, el preprocesado (filtrado, extracción de componentes principales y segmentación) se ha realizado de forma offline para garantizar la calidad de las señales de entrada. Sin embargo, la visión a futuro consiste en diseñar un pipeline completamente *end-to-end* que integre un módulo de preprocesado rápido, posiblemente basado en técnicas de filtrado adaptativo en streaming, que permita la inferencia en tiempo casi real. De este modo, el sistema no solo clasificaría las etapas del sueño a posteriori, sino que ofrecería información de manera continua, habilitando aplicaciones de monitoreo inmediato y alertas tempranas en casos de anomalías (por ejemplo, apneas prolongadas o transiciones erráticas entre etapas).

- **Expansión a señales multimodales y fusión de datos.**
  Aunque el presente estudio se basa, exclusivamente, en una derivación comprimida mediante PCA, resulta altamente recomendable incorporar otros parámetros fisiológicos accesibles con dispositivos económicos, como la actividad electroocular (EOG), la electromiografía (EMG) de músculos faciales. La fusión multimodal, junto con los dispositivos utilizados apoyada en técnicas de aprendizaje profundo, puede mejorar

las capacidades del modelo frente a artefactos eléctricos o interferencias externas, y proporcionar mayor fidelidad en la clasificación de fases (por ejemplo, diferenciando mejor N-REM de REM).

- **Implementación en entornos embebidos y dispositivos portátiles.**
  Para aumentar la accesibilidad del sistema en entornos con recursos limitados, es esencial llevar el modelo a plataformas de *edge computing* o microcontroladores especializados (por ejemplo, *ARM Cortex-M* con aceleradores de inferencia). Esto implicaría optimizar la arquitectura (cuantización de pesos, poda estructural y reducción de la complejidad computacional) para que el modelo sea ejecutable en tiempo real directamente en un dispositivo de bajo coste. La capacidad de realizar inferencia local reduciría la dependencia de redes de comunicación y mejoraría la privacidad de los datos, debido a que no sería necesario el envío continuo de datos a servidores remotos.

- **Personalización y adaptación al sujeto mediante aprendizaje continuo.**
  Cada individuo presenta patrones de sueño con características propias. Por este motivo, se propone la implementación de mecanismos de *fine-tuning* o *transfer learning* a nivel individual, mediante los cuales el sistema se adapte progresivamente a las particularidades del usuario. Adicionalmente, un esquema de aprendizaje continuo (continual learning) permitiría actualizar el modelo conforme se vayan recopilando nuevas noches de registro, reduciendo gradualmente el error de clasificación y aumentando la capacidad predictiva sin necesidad de reentrenar desde cero.

- **Exploración de estrategias de aprendizaje auto-supervisado y semi-supervisado.**
  La anotación manual de señales es un proceso costoso y laborioso. Como línea a medio plazo, se plantea investigar técnicas de *Self-Supervised Learning* que permitan preentrenar representaciones útiles de las señales de sueño sin requerir etiquetas explícitas. Posteriormente, con una pequeña fracción de datos etiquetados se podría llevar a cabo un ajuste fino (*fine-tuning*) para clasificación supervisada, reduciendo significativamente la necesidad de grandes volúmenes de datos etiquetados. Además, la incorporación de métodos de *Active Learning* haría posible seleccionar de manera inteligente las instancias más informativas para anotación, optimizando los recursos de los especialistas.

- **Integración con interfaces de usuario y plataformas de telemedicina.**
  Con el fin de facilitar la adopción clínica y doméstica, se prevé desarrollar aplicaciones móviles y web que presenten los resultados de manera intuitiva. Esto incluye visualizaciones en tiempo real de las etapas de sueño, alertas automáticas en caso de patrones anómalos y resúmenes estadísticos nocturnos. La conectividad con plataformas

de telemedicina permitirá, además, compartir de forma segura los datos con especialistas remotos, habilitando consultas a distancia y seguimiento continuo del paciente, lo que es especialmente valioso en zonas rurales o con acceso limitado a especialistas del sueño.

En conclusión, estas líneas de trabajo futuro subrayan la clara perspectiva de crecimiento científico y tecnológico de este proyecto. La colaboración con otros grupos de investigación, tanto en el ámbito de ingeniería industrial, electrónica y biomédica como en neurociencia clínica, potenciará el desarrollo de modelos cada vez más sofisticados, que vayan desde arquitecturas de última generación (Transformers convolucionales, redes gráficas de señales, etc.) hasta su implementación directa en dispositivos portátiles de bajo coste. El horizonte consiste en alcanzar una solución integral capaz de ofrecer diagnósticos fiables en tiempo real, democratizando el acceso a la monitorización del sueño y contribuyendo significativamente al tratamiento y prevención de trastornos asociados.

Apéndice A

Anexo Red Neuronal Implementada

A.1. Introducción

En este proyecto se ha propuesto una red neuronal híbrida que combina dos componentes principales: (1) un bloque de convoluciones unidimensionales (CNN) y (2) un bloque de LSTM bidireccional (BiLSTM). El propósito de este modelo es extraer características locales en secuencias unidimensionales y capturar dependencias temporales de largo alcance antes de realizar la clasificación en tres categorías, que representan a cada una de las etapas del sueño.

A lo largo de las siguientes secciones se describirán cada uno de estos bloques y se justificarán la elección de sus parámetros y sus topologías, específicamente por qué se ha optado por tres capas convolucionales, cuál es la motivación para emplear LSTM bidireccionales y qué criterio subyace a las capas densas y al resto de componentes del entrenamiento (función de pérdida, optimizador y callbacks...).

A.2. Bloque CNN de entrada (Conv1D)

A.2.1. Justificación Teórico-Práctica de Emplear Convoluciones en Datos Secuenciales

En multitud de problemas que involucran señales temporales o series de datos indexados en el tiempo, como es el caso de este proyecto, el primer paso es identificar patrones locales que se repiten a lo largo de la secuencia. Las capas de convolución unidimensional (Conv1D) permiten precisamente eso: aplicar filtros (kernels) deslizantes sobre la dimensión temporal para resaltar características localizadas (picos, transiciones abruptas, oscilaciones concretas, entre otras).

Elegir una arquitectura basada en Conv1D, como etapa inicial, tiene el objetivo de reducir la dimensionalidad de la señal de forma progresiva y de construir representaciones robustas de patrones elementales, de modo que la

red recurrente posterior trabaje sobre *feature maps* ya filtrados. Además, las convoluciones 1D presentan una menor complejidad computacional que otras alternativas (por ejemplo, usar directamente LSTM sobre la entrada bruta), ya que procesan ventanas de tamaño fijo en paralelo, lo cual acelera tanto la fase de entrenamiento como la de inferencia. De este modo, se obtiene un desplazamiento de ventana que aumenta, operativamente, el número de datos de entrada.

Otra ventaja de las Conv1D, en este contexto, es que resultan menos propensas al sobreajuste, el conocido overfitting, cuando se incorporan estrategias de regularización adecuadas, como es el caso de: BatchNormalization, Dropout y penalización L2, dado que reducen la dependencia al ruido presente en la señal original.

A.2.2. Elección de Tres Capas Conv1D y Progresión de Filtros (64 →128 →256)

La decisión de emplear exactamente tres capas Conv1D no se tomó de manera arbitraria, sino que se basa en dos consideraciones principales. Primero, introducir varias etapas de convolución encadenadas, lo que permite que los primeros filtros (con menor número de canales) detecten rasgos muy locales, por ejemplo, transiciones apenas perceptibles en grupos de tres puntos, mientras que las capas posteriores, que disponen de más filtros, son capaces de construir representaciones más abstractas derivadas de combinaciones de esos patrones básicos. Segundo, la progresión de 64 a 128 y, finalmente, a 256 filtros sigue una lógica de amplitud creciente que aparece en múltiples arquitecturas de referencia (por ejemplo, VGG o algunas variantes de redes adaptadas a audio), donde el número de kernels se duplica en cada nivel para incrementar la capacidad de modelado.

Al comenzar con 64 filtros en la primera capa, se equilibra la relación entre expresividad y riesgo de sobreajuste: 64 canales permiten capturar suficiente diversidad de patrones locales sin disponer de tantos parámetros como para incurrir en memorizar ruido. A medida que la red profundiza, se incrementa la cantidad de filtros primero a 128 y después a 256, con la doble ventaja de poder componer características de orden superior y de reducir el tamaño de la señal, (si se decidiera usar pooling, aunque en esta implementación se ha optado por mantener la dimensión temporal utilizando `padding='same'`). Con todo ello se consigue que las tres capas Conv1D representen un equilibrio entre profundidad y complejidad, permitiendo una buena capacidad de extracción de características sin aumentar la complejidad temporal y reduciendo el riesgo de sobreajuste.

A.2.3. Kernel size = 3 y padding = "same": Equilibrio entre Contexto Local y Preservación de la Dimensión

La elección de un **kernel_size = 3**, es decir, que el filtro abarque tres pasos consecutivos, se fundamenta en dos aspectos. Por un lado, un filtro de longitud tres es suficientemente pequeño para detectar cambios

súbitos y patrones muy locales en la secuencia, pero permite a la vez capturar interacciones entre puntos contiguos. De acuerdo con la literatura en procesamiento de señales, un tamaño de kernel entre 3 y 5 es frecuentemente idóneo para percepciones locales, sobre todo si no existe un conocimiento previo de patrones de longitud mayor. Por otro lado, mantener el kernel de tamaño reducido contribuye a que el número de parámetros de cada capa sea moderado, lo que facilita el entrenamiento y reduce la probabilidad de incurrir en sobreajuste.

Asimismo, el uso de **padding = "same"** tiene como principal ventaja conservar la longitud original de la secuencia a lo largo de cada capa convolucional. Dado que más adelante se aplican capas recurrentes, que asumen que cada vector en la dimensión temporal representa un paso válido en la secuencia, se debe considerar no recortar, arbitrariamente, esos pasos. Al mantener la misma cantidad de timesteps en cada nivel, se garantiza que las LSTM bidireccionales posteriores trabajen sobre la misma longitud temporal. En caso de que el problema requiriera reducir periódicamente la dimensionalidad temporal, por ejemplo, para concentrar la información mediante pooling, se podría haber optado por `stride >1` o `max pooling`. No obstante, en nuestra implementación se ha optado por priorizar el ritmo original de la señal para que la red secuencial reconozca patrones temporales completos.

A.2.4. BatchNormalization y Dropout(0.3) tras cada Conv1D: Estabilización y Regularización

Justo después de cada una de las tres capas Conv1D se incorpora una capa de **BatchNormalization** seguida de un **Dropout(0.3)**. Esta combinación persigue dos objetivos: (1) acelerar la convergencia del entrenamiento y (2) reducir la tendencia al sobreajuste. Entrando en una explicación detallada de cada uno de estos dos elementos:

- **BatchNormalization**: se normaliza las activaciones de cada filtro (mapa de características) en cada mini-batch, se corrigen las posibles desviaciones de distribución que se producen durante el entrenamiento (fenómeno conocido como "covariate shift interno"). Esto, a su vez, permite usar tasas de aprendizaje más elevadas sin que el entrenamiento se vuelva inestable, reduce la sensibilidad a la inicialización de los pesos y, frecuentemente, mejora la generalización final previa a cada una de las siguientes capas de neuronas.

- **Dropout(0.3)**: Se ha optado que durante cada época de entrenamiento, se "desactiven" aleatoriamente el 30 % de las neuronas de salida de la capa previa, siempre dentro de nuestra capa de entrada convolucionales. Dado que los filtros convolucionales actúan como detectores de patrones, apagar aleatoriamente algunos de ellos obliga a la red a no depender de un único subconjunto de filtros para representar la información, lo cual previene el efecto de "co-adaptación". La tasa de 0.3 (30 %) se eligió como valor intermedio: suficiente para tener

un efecto regularizador importante, sin comprometer drásticamente la capacidad de aprendizaje en cada subcapa.

En conjunto, este esquema `Conv1D` → `BatchNormalization` → `Dropout(0.3)` se repite tres veces, con 64, 128 y 256 filtros respectivamente, ofreciendo un equilibrio entre capacidad de extracción de rasgos locales, estabilidad del entrenamiento y control del sobreajuste.

A.2.5. Regularización L2 (`kernel_regularizer = 0.001`)

Adicionalmente, cada capa Conv1D aplica una **regularización L2** con factor 0,001 sobre sus pesos (`kernel_regularizer=l2(0.001)`). La penalización L2, también llamada *weight decay*, consiste en añadir al término de pérdida, el cuadrado de la norma de los parámetros multiplicado por un coeficiente pequeño (en este caso, 0.001). De esta forma, durante el ajuste de pesos, se impide que los valores crezcan de forma explosiva y se favorecen soluciones con pesos distribuidos más uniformemente. Este tipo de regularización contribuye a mejorar la capacidad de generalización de la red, ya que mitiga la posibilidad de ajustar el modelo a ruidos muy concretos de los datos de entrenamiento.

Finalmente, en la siguiente subsección se muestra el bloque CNN que se acaba de describir.

A.2.6. Código de Referencia para el Bloque CNN

```
# Bloque CNN como filtro de entrada
x = Input(shape=(X_train.shape[1], 1))

# Primera capa Conv1D
x = Conv1D(
filters=64,
kernel_size=3,
activation='relu',
padding='same',
kernel_regularizer=l2(0.001)
)(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)

# Segunda capa Conv1D
x = Conv1D(
filters=128,
kernel_size=3,
activation='relu',
padding='same',
kernel_regularizer=l2(0.001)
)(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)
```

```
# Tercera capa Conv1D
x = Conv1D(
filters=256,
kernel_size=3,
activation='relu',
padding='same',
kernel_regularizer=l2(0.001)
)(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)
```

Listing A.1: Definición del bloque CNN de entrada con Conv1D, BatchNormalization y Dropout.

A.3. Bloque BiLSTM (Modelo de Dependencias Temporales)

Una vez descrita la capa de entrada, conformada por tres subcapas convolucionales, el siguiente paso es describir la capa oculta, que constituye el verdadero núcleo de predicción. Este está conformado por BiLSTM, o también denominada LSTM bidireccional. Al igual que se acaba de efectuar, se va a realizar una descripción precisa y justificada de cada elemento y parámetro seleccionado.

A.3.1. Motivación para Usar LSTM Bidireccional

Las redes LSTM (Long Short-Term Memory) se diseñaron precisamente para superar las limitaciones de las RNN tradicionales a la hora de aprender dependencias a largo plazo. Gracias a sus compuertas internas (forget gate, input gate y output gate), las LSTM pueden almacenar y recuperar información relevante durante muchas iteraciones, lo cual resulta esencial en tareas donde el valor actual de la señal depende de eventos ocurridos decenas, o incluso cientos, de registros de datos previos.

Sin embargo, en ciertos dominios como en el análisis de series temporales en las que el contexto futuro aporta información adicional, como ocurre en nuestro estudio, suele ser beneficioso procesar la secuencia en ambas direcciones. Una LSTM "hacia adelante" (forward) únicamente ve el pasado para predecir el futuro, pero a veces conocer el "futuro", es decir, lo que vendrá más adelante en la secuencia, ayuda a interpretar correctamente un paso intermedio. Al usar una LSTM bidireccional (BiLSTM), la red simultáneamente procesa la entrada de izquierda a derecha y de derecha a izquierda, combinando ambas representaciones en cada paso. Esto lo hace especialmente importante en nuestro procesamiento, donde ciertas anomalías sólo se entienden en función de la tendencia que viene inmediatamente después. Si la red dependiera únicamente de la historia pasada, ignoraría esa información futura y no "aprendería" la evolución. La BiLSTM, en cambio, ve la secuencia completa en ambas direcciones, permitiendo que cada vector de

salida en el tiempo t contemple tanto lo ocurrido antes como lo que ocurrirá posteriormente.

Esta ventaja a nivel de contexto justifica la inclusión de BiLSTM en la etapa intermedia del modelo, de modo que la red adquiera una representación de cada paso que integra contexto pasado y futuro antes de pasar a las capas densas finales.

A.3.2. Primera capa BiLSTM: 64 Unidades por Dirección y return_sequences=True

En nuestro modelo, esta capa se coloca inmediatamente después de la tercera subcapa del bloque CNN. Esta primera capa del BiLSTM, según:

```
x = Bidirectional(LSTM(units=64,
    ↪ return_sequences=True))(x)
```

Listing A.2: Primera capa BiLSTM con 64 unidades por dirección.

A continuación, se justifican las razones por las que se han elegido los valores de cada parámetro:

- **units = 64**: cada LSTM interna, tanto forward como backward, cuenta con 64 celdas de memoria. La elección de 64 responde a un equilibrio entre expresividad, esto es, capacidad para modelar patrones temporales complejos y coste computacional, la complejidad temporal. Con 64 unidades, se obtienen 128 dimensiones de representación, 64 de la pasada forward y 64 de la pasada backward, para cada paso temporal, lo cual resulta adecuado cuando la secuencia de entrada ya ha sido filtrada por las tres capas Conv1D previas y se desea una representación robusta sin sobrecargar excesivamente de parámetros esta etapa.

- **return_sequences = True**: este parámetro indica que la capa BiLSTM debe devolver, para cada paso t (con $1 \leq t \leq$ timesteps), el vector de activación oculto correspondiente. De usarse `return_sequences=False`, comunmente utilizado, la BiLSTM solo devolvería el estado oculto del último paso en la orientación forward y backward, perdiendo todas las salidas intermedias. Sin embargo, en nuestro diseño es imprescindible mantener la información de cada paso temporal, ya que las capas densas finales necesitan conocer la secuencia completa de salidas (todas las activaciones temporales) para asignar pesos según su relevancia.

En consecuencia, esta primera subcapa BiLSTM transforma el tensor de entrada de forma $(\text{timesteps}, 256)$ a un tensor $(\text{timesteps}, 128)$, donde las 128 dimensiones resultan de concatenar las 64 unidades forward con las 64 unidades backward en cada paso.

A.3.3. Segunda capa BiLSTM: 32 Unidades por Dirección y return_sequences=False

A continuación, la salida (timesteps, 128) de la primera BiLSTM se pasa a una segunda capa:

```
x = Bidirectional(LSTM(units=32))(x)
```

Listing A.3: Segunda capa BiLSTM con 32 unidades por dirección.

Los objetivos de esta segunda capa son:

- **units = 32**: en esta segunda BiLSTM se reduce el número de unidades a 32 por dirección. Al concatenar las pasadas forward y backward, la salida de esta capa es un tensor de dimensión (64,) para cada muestra, ya que `return_sequences=False` condensa toda la secuencia en un vector global. Reducir el tamaño de 128 (resultado de la primera BiLSTM) a 64 simplifica el cálculo en las etapas posteriores, específicamente en las capas densas.

- **return_sequences = False**: no es necesario mantener la secuencia completa porque posteriormente se proyecta ese vector global a través de capas densas para clasificación.

De este modo, la segunda BiLSTM actúa como un "refinador" de la información generada por la primera capa recurrente, la CNN previa, condensándola en 64 dimensiones. Así, se consigue un equilibrio entre capacidad de modelado y eficiencia computacional: 64 características por muestra son suficientes para capturar patrones temporales complejos sin incurrir en un número prohibitivo de parámetros. Las ventajas que presenta la configuración de dos capas BiLSTM sucesivas son:

1. **Extracción gradual de abstracción**: Al aplicar dos capas BiLSTM consecutivas, la primera capa descubre patrones temporales de nivel más general, por ejemplo, oscilaciones lentas, tendencias globales ... típicas de la problemática de detección de etapas de sueño, y la segunda capa, alimentada con esas representaciones, se especializa en detectar patrones secundarios más finos. Si se usara una única BiLSTM con 128 unidades por dirección, esa capa trataría de aprender desde cero tanto patrones de alto nivel como detalles finos, lo cual podría requerir un mayor volumen de datos para converger y resultar más propenso al sobreajuste.

2. **Coste computacional y estabilidad en el entrenamiento**: Emplear dos capas con 64 y 32 unidades, en lugar de una sola con, por ejemplo, 96 unidades, reduce la carga computacional en cada paso de propagación, ya que las operaciones matriciales se dividen en niveles. Con ello, se favorece una convergencia más estable durante el entrenamiento. Además, es más sencillo ajustar hiperparámetros (cada capa tiene un número de unidades relativamente no muy alto) y monitorizar la posible saturación o *desvanecimiento de gradiente* en cada nivel.

La elección de una arquitectura BiLSTM de dos capas (64 →32 unidades) se basa en una estrategia de extracción secuencial de información, reduciendo la dimensionalidad de manera escalonada para facilitar el cálculo posterior y optimizar la capacidad de generalización.

A.4. Capas densas Finales y Predicción

A.4.1. Capa Densa Intermedia (Dense(64, ReLU) y Dropout(0.4))

Una vez obtenido el vector global de 64 dimensiones procedente de la segunda BiLSTM, la red conecta con:

```
x = Dense(64, activation='relu',
    ↪ kernel_regularizer=l2(0.001))(x)
x = Dropout(0.4)(x)
```

Listing A.4: Capa densa intermedia y dropout.

La inclusión de esta capa densa responde a la necesidad de refinar la representación global antes de efectuar la clasificación. A continuación se exponen las motivaciones para cada componente:

- **Dense(units = 64)**: elegir 64 neuronas en esta capa intermedia refleja la intención de mantener la dimensión de representación en un rango manejable (ni muy pequeña para que se pierda información, ni muy grande para que el modelo se vuelva excesivamente complejo). Dado que la capa anterior ya había condensado la secuencia en 64 dimensiones, esta capa densa simplemente transforma linealmente esa representación y la combina a través de la función de activación, permitiendo que la red aprenda patrones no lineales adicionales que faciliten la separación entre clases.

- **activation = 'relu'**: la función ReLU (Rectified Linear Unit) se selecciona por ser computacionalmente eficiente y por ayudar a mitigar el problema de gradientes desvanecientes en redes profundas. ReLU activa solo aquellas neuronas cuya entrada sea positiva, lo que introduce esparsidad en la activación y acelera la convergencia.

- **kernel_regularizer = l2(0.001)**: al añadir nuevamente regularización L2 en esta capa densa, se evita que los pesos crezcan de manera descontrolada. Ese pequeño coeficiente (0.001) impide que los parámetros adquieran valores muy grandes solo para forzar una clasificación perfecta en el conjunto de entrenamiento.

- **Dropout(0.4)**: tras la activación ReLU, se aplica un dropout con probabilidad 0.4 (40 %). Esta tasa, más alta que la usada en las capas convolucionales (0.3), refleja la observación empírica de que, en capas totalmente conectadas, suele ser conveniente un dropout ligeramente superior para prevenir la co-adaptación de neuronas. En la capa densa intermedia, la red dispone de más libertad para aprender combinaciones

arbitrarias de características, por lo que es más susceptible a depender de subconjuntos de neuronas. De esta forma, al apagar el 40 % de las neuronas en cada iteración, se fuerza al modelo a distribuir el aprendizaje entre todos los parámetros disponibles, aumentando la robustez y reduciendo el sobreajuste.

En conjunto, esta capa densa intermedia funciona como un filtro no lineal que recoge la información global y la refina, preparándola para la capa de salida de clasificación.

A.4.2. Capa de Salida (Dense(3, softmax)): Clasificación en tres Clases

Para producir las probabilidades finales de pertenencia a cada una de las tres categorías de salida, se emplea:

```
output_layer = Dense(3, activation='softmax')(x)
```

Listing A.5: Capa de salida final con softmax.

Los argumentos que justifican esta elección son:

- **units = 3**: aunque las etiquetas originales se codifican como vectores one-hot de longitud 3, la capa de salida debe contar con tres neuronas, cada una representando la probabilidad de la muestra de pertenecer a la clase correspondiente. Es decir, el modelo produce un vector $\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \hat{y}_3]$ en el que solo una de las entradas alcanzará cercano a 1, indicando la clase predicha.
- **activation = 'softmax'**: la función softmax convierte el conjunto de 3 valores reales en un vector cuyas componentes satisfacen $\hat{y}_c \in (0, 1)$ y $\sum_{c=1}^{3} \hat{y}_c = 1$. De este modo, cada $\hat{y}_c$ se interpreta como la probabilidad de la muestra de pertenecer a la clase c. Además, este formato coincide directamente con la codificación one-hot de las etiquetas en el entrenamiento.

En definitiva, la capa `Dense(3, softmax)` produce la distribución de probabilidad sobre las tres clases en formato compatible con los vectores one-hot de las etiquetas y constituye la salida definitiva del modelo.

A.5. Compilación del Modelo y Estrategia de Entrenamiento

A.5.1. Función de pérdida: Focal Loss ($\alpha = 0{,}25$, $\gamma = 2{,}0$)

En lugar de emplear la función clásica de entropía cruzada categórica (`categorical_crossentropy`), el modelo utiliza una variante denominada **focal loss**. Esta decisión nos viene impuesta por el importante desbalance de las clases 1 y 3, vigilia y REM respectivamente, frente a la 2, NREM.

Cuando las clases están desbalanceadas, la entropía cruzada estándar tiende a favorecer a la clase mayoritaria, pues un simple ajuste en el que se clasifica, correctamente, con el criterio de que la clase con más ejemplos produce una baja pérdida, aunque las clases minoritarias sufran alta tasa de error, lo cual no es válido en nuestro caso de estudio.

La función focal loss introduce dos parámetros clave que corrigen este sesgo:

1. γ (gamma): es el factor que amplifica la penalización asociada a ejemplos mal clasificados. Cuando $\hat{y}_c$ (probabilidad asignada a la clase correcta) es baja, $(1 - \hat{y}_c)^\gamma$ se vuelve grande, lo que incrementa la contribución del término de pérdida para ese ejemplo. En los casos donde el modelo ya asigna $\hat{y}_c$ cercano a 1 (es decir, ejemplos "fáciles"), el factor $(1-\hat{y}_c)^\gamma$ disminuye la pérdida, evitando que siga "corrigiendo" inútilmente ejemplos bien clasificados. Esto incentiva al modelo a centrarse en los ejemplos difíciles, los cuales suelen pertenecer a las clases minoritarias. En este proyecto, se elige $\gamma = 2{,}0$ pues es un valor comúnmente recomendado en la literatura original de focal loss (Lin et al., 2017) para lograr un buen balance entre penalización de errores y estabilidad en el descenso del gradiente.

2. α: es el parámetro que ajusta la ponderación entre clases. Tradicionalmente, se puede fijar un α distinto para cada categoría, pero en este caso se utiliza un valor escalar $\alpha = 0{,}25$. Esta constante atenúa la pérdida global (sumando un factor de 0.25), lo que hace que el proceso de entrenamiento no dependa únicamente de γ. Al combinar $\alpha < 1$ con $\gamma = 2$, se logra que las clases minoritarias, que habitualmente tienen $\hat{y}_c$ más bajo, incrementen su contribución al gradiente de manera más significativa que las clases mayoritarias.

Con todo ello, se consigue por un lado evitar que la clase con más ejemplos domine la optimización del proceso de entrenamiento y por otro centrar el ajuste de pesos en aquellos ejemplos clasificados con baja confianza, que suelen estar asociados a casos, ejemplos, minoritarios.

La implementación de esta función de pérdida se realiza mediante una función interna que calcula:

$$\mathrm{FL}(\mathbf{y}_{\text{true}}, \mathbf{y}_{\text{pred}}) \;=\; -\sum_{c=1}^{3} \alpha \left(1 - \hat{y}_c\right)^\gamma y_c \log(\hat{y}_c),$$

donde $\mathbf{y}_{\text{true}}$ es el vector one-hot de la etiqueta real y $\hat{\mathbf{y}} = \mathbf{y}_{\text{pred}}$ las probabilidades estimadas por la capa softmax.

A.5.2. Optimizador Adam con OneCycle Learning Rate: Rationale y Beneficios

Para actualizar los pesos de la red durante el entrenamiento, se elige el optimizador **Adam** debido a su capacidad de adaptar la tasa de aprendizaje de manera individual para cada parámetro, combinando las ventajas de

AdaGrad (adaptación basada en el promedio de los gradientes) y RMSProp (adaptación basada en el promedio de los cuadrados de gradientes). Adam suele converger más rápido que SGD puro y es menos sensible a la elección de la tasa de aprendizaje inicial.

No obstante, usar una tasa fija durante todas las épocas puede conllevar dos problemas:

1. Si la tasa de aprendizaje es demasiado alta, el entrenamiento puede volverse inestable y saltar alrededor del mínimo.

2. Si la tasa de aprendizaje se mantiene baja, la convergencia puede ser excesivamente lenta y quedarse atrapada en mínimos locales subóptimos.

Para solventar estas limitaciones, se implementa un **OneCycle Learning Rate (OneCycleLR)** basado en la política cosenoidal. El procedimiento concreto es:

1. **Definir** `max_lr = 0.005`: esta tasa de aprendizaje máxima se considera adecuada después de experimentos preliminares (por ejemplo, mediante un "learning rate finder" que evalúe la pérdida en función de distintas tasas).

2. **Fijar** `base_lr = max_lr / 10 = 0.0005`: la tasa mínima a la que arranca cada ciclo. Este valor bajo evita que, al principio del entrenamiento, los gradientes iniciales perjudiquen la convergencia.

3. **Calcular** `step_size = len(X_train) // batch_size`: el número de iteraciones que conforman un *epoch*, dado que el *batch size* es 32.

4. **Definir la función de tasa de aprendizaje**:

$$\mathrm{lr}(e) \;=\; \mathrm{base_lr} \times \big(1 + \cos(\pi\, e/\mathrm{step_size})\big),$$

   donde e corresponde al índice de época.

5. **Registrar un** `LearningRateScheduler`: se utiliza un callback de Keras que, al inicio de cada época, actualiza `optimizer.lr` con la fórmula anterior.

Este plan "OneCycle" hace que la tasa de aprendizaje suba gradualmente desde `base_lr` hasta `max_lr` durante la primera mitad del ciclo (evitando subidas abruptas) y que luego decrezca simétricamente hasta volver a `base_lr`. El beneficio de esta curva cosenoidal es que:

- **Fase de calentamiento (warmup)**: al iniciar con una tasa baja, se evita saltos enormes en el espacio de parámetros cuando los gradientes aún son inestables.

- **Fase de exploración**: al incrementar la tasa de aprendizaje hasta `max_lr`, se permite escapar de mínimos locales subóptimos y explorar regiones más amplias del espacio de pesos.

- **Fase de enfriamiento**: al descender nuevamente, se refina la búsqueda alrededor de un mínimo encontrado, estabilizando la convergencia.

Esta estrategia ha demostrado, en la práctica, acelerar el entrenamiento y mejorar tanto la precisión final como la capacidad de generalización, especialmente en arquitecturas profundas.

A.5.3. Callbacks Adicionales: EarlyStopping, ModelCheckpoint y LambdaCallback para Registro de Métricas

Además del `LearningRateScheduler`, el código incorpora varios callbacks que contribuyen a controlar el entrenamiento, salvar los mejores pesos y documentar el proceso:

1. **EarlyStopping**
   - Se configura para monitorear la métrica `val_loss` (pérdida en el conjunto de validación).
   - Con `patience = 5`, se indica que si durante cinco épocas consecutivas no hay mejora en la pérdida de validación, el entrenamiento se detenga antes de completar todas las épocas planificadas.
   - `restore_best_weights = True` asegura que, una vez que la condición de detención se cumple, se restauren los pesos que lograron la menor pérdida de validación, en lugar de dejar los pesos en el punto donde se interrumpió. Esto es crucial para prevenir que el modelo termine en un estado con sobreajuste después de varias épocas sin mejora.
2. **ModelCheckpoint**
   - Su función es guardar en un archivo (normalmente con un nombre que incluya el número de época y el valor de `val_loss`) el modelo completo (arquitectura y pesos) cada vez que se observe una mejora en `val_loss`.
   - Esto permite, en caso de interrumpir el entrenamiento de forma manual o de querer recuperar el mejor estado alcanzado, disponer de todos los "checkpoints" generados durante la ejecución.
3. **LambdaCallback para registro de métricas**
   - Se define un callback personalizado (`LambdaCallback`) cuyo propósito es abrir un archivo de texto (por ejemplo, `training_log.txt`) y escribir tras cada época las métricas relevantes: pérdida de entrenamiento, precisión de entrenamiento, pérdida de validación y precisión de validación.

- Este registro externo resulta útil para hacer un análisis *offline* del proceso de optimización, trazando curvas, comparando ejecuciones con distintos hiperparámetros, o simplemente archivando el histórico de métricas a efectos de reproducibilidad y documentación de la memoria.

En conjunto, estos callbacks complementan al optimizador y a la función de pérdida focal, garantizando que el entrenamiento sea eficiente (gracias a OneCycleLR), seguro frente al sobreajuste (EarlyStopping) y fácilmente recuperable/analizable (ModelCheckpoint y registro de métricas).

A.5.4. Código de referencia para compilación

```
# Definición de la función de pérdida focal
def focal_loss(alpha=0.25, gamma=2.0):
def loss(y_true, y_pred):
y_pred = tf.clip_by_value(y_pred, 1e-7, 1-1e-7)
cross_entropy = -y_true * tf.math.log(y_pred)
weight = alpha * tf.pow(1 - y_pred, gamma)
return tf.reduce_sum(weight * cross_entropy, axis=1)
return loss

# Cálculo de pesos de clase
class_weights = compute_class_weight('balanced',
   ↪ classes=np.unique(y_train_indices),
   ↪ y=y_train_indices)

# Configuración de OneCycleLR
max_lr = 0.005
base_lr = max_lr / 10
step_size = len(X_train) // 32

def lr_schedule(epoch, lr):
return base_lr * (1 + np.cos(np.pi * epoch / step_size))

lr_callback = LearningRateScheduler(lr_schedule)

# Callbacks adicionales
early_stopping = EarlyStopping(monitor='val_loss',
   ↪ patience=5, restore_best_weights=True)
model_checkpoint = ModelCheckpoint('best_model.h5',
   ↪ monitor='val_loss', save_best_only=True)

def log_metrics(epoch, logs):
with open('training_log.txt', 'a') as f:
f.write(f"Epoch {epoch+1}: loss={logs['loss']:.4f},
   ↪ acc={logs['accuracy']:.4f}, "
f"val_loss={logs['val\_loss']:.4f},
   ↪ val_acc={logs['val_accuracy']:.4f}\n")

metrics_logger = LambdaCallback(on_epoch_end=log_metrics)
```

```
# Compilación final
model.compile(
optimizer=Adam(learning_rate=base_lr),
loss=focal_loss(alpha=0.25, gamma=2.0),
metrics=['accuracy']
)

# Entrenamiento
model.fit(
X_train, y_train,
validation_data=(X_val, y_val),
epochs=epochs,
batch_size=32,
class_weight=class_weights,
callbacks=[lr_callback, early_stopping,
    ↪ model_checkpoint, metrics_logger]
)
```

Listing A.6: Compilación del modelo con Adam, focal loss y callbacks.

A.6. Preprocesamiento de Datos y Cálculo de Pesos de Clase

A.6.1. División Entrenamiento/Validación

Antes de iniciar el entrenamiento, el conjunto de datos se divide aleatoriamente en dos subconjuntos: uno de **entrenamiento** (80 % de las muestras) y otro de **validación** (20 %). Esta partición estratificada por clases o aleatoria (dependiendo de la implementación concreta) tiene como fin evaluar el rendimiento del modelo en datos no vistos durante el ajuste de pesos, midiendo tanto la pérdida como la precisión en validación para decidir posibles paradas tempranas o ajustes de hiperparámetros.

A.6.2. Cálculo de Pesos de Clase con `compute_class_weight`

Cuando las clases en el conjunto original no están distribuidas de manera uniforme (por ejemplo, si hay muchas más muestras de la etapa 1 que de la etapa 3), entrenar directamente con entropía cruzada o focal loss sin pesos puede inducir a la red a que "ignore" las clases minoritarias. Para compensar este desbalance se calculan **pesos de clase** inversamente proporcionales a la frecuencia de cada categoría.

En concreto, se emplea la función `compute_class_weight` de Scikit-learn con parámetros `class_weight='balanced'`, obteniendo un array de tres valores (uno por clase). Cada peso ω_c se define como:

$$\omega_c = \frac{N}{3 \times N_c},$$

donde N es el número total de muestras, N_c el número de muestras de la clase c y 3 el número de clases. Esta fórmula asegura que las clases minoritarias tengan un peso mayor en la función de pérdida, multiplicando el término de pérdida para esas clases por ω_c.

Dado que la focal loss ya introduce un cierto sesgo hacia ejemplos difíciles, complementar este efecto con pesos de clase garantiza que los ejemplos de categorías menos frecuentes reciban doble penalización: tanto por el factor ω_c como por el término $\left(1 - \hat{y}_c\right)^\gamma$ cuando sean mal clasificados. Este diseño compensa de manera contundente el desbalance en los datos.

A.6.3. One-hot Encoding de Etiquetas y Expansión de Dimensiones de Entrada

Para que la capa softmax final funcione adecuadamente, las etiquetas originales, que pueden codificarse como vec tores enteros 1, 2 o 3, se transforman a vectores one-hot de longitud 3. Esto hace que cada etiqueta y pase de ser un escalar a un vector $\mathbf{y} = [y_1, y_2, y_3]$ con un único 1 en la posición de la clase correspondiente y 0 en las otras posiciones. De esta forma, la función de pérdida focal opera directamente con vectores one-hot en lugar de enteros.

Por último, el conjunto de predictores X (originalmente (num_muestras, timesteps)) se expande a $(\text{num_muestras}, \text{timesteps}, 1)$ para adecuarse al formato que esperan las capas Conv1D (la tercera dimensión indica el número de canales, que en este caso es 1, porque se trata de señales univariantes).

Referencias

[1] National Heart, Lung, and Blood Institute. «Advancing Heart, Lung, Blood, and Sleep Research. »dirección: https://www.nhlbi.nih.gov/about/budget-and-legislative-information/science-advances.

[2] S. Mousavi, F. Fatemeh Afghah y U. Rajendra, «SleepEEGNet: Automated sleep stage scoring with sequence to sequence deep learning approach,» *PLOS ONE*, vol. 14, n.º 5, e0216456, 2019. DOI: 10.1371/journal.pone.0216456. dirección: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0216456.

[3] American Academy of Sleep Medicine, *The AASM Manual for the Scoring of Sleep and Associated Events: Rules, Terminology and Technical Specifications, Version 2.2*, American Academy of Sleep Medicine, Darien, IL, 2015. dirección: https://aasm.org/clinical-resources/scoring-manual/.

[4] Real Academia Española. «Sueño. »dirección: https://dle.rae.es/sue\%C3\%B1o.

[5] Instituto Internacional del Sueño. «¿Qué es, cómo se produce el sueño, fases, cuántas horas dormir? »Dirección: https://www.iis.es/que-es-como-se-produce-el-sueno-fases-cuantas-horas-dormir/.

[6] American Academy of Sleep Medicine, *The AASM Manual for the Scoring of Sleep and Associated Events: Rules, Terminology and Technical Specifications (Version 2.0)*, American Academy of Sleep Medicine, Darien, IL, 2012. dirección: https://aasm.org/wp-content/uploads/2017/11/Summary-of-Updates-in-v2.0-FINAL.pdf.

[7] N. Napoli, «Getting Good Sleep Could Add Years to Your Life,» 2023. dirección: https://neurosciencenews.com/sleep-mortality-22551/.

[8] Statista. «Encuestados que padecieron trastornos del sueño en los últimos doce meses.» Infografía. dirección: https://es.statista.com/grafico/29491/encuestados-que-padecieron-trastornos-del-sueno-en-los-ultimos-doce-meses/.

[9] A. Pérez Menéndez, *18 de marzo: Día Mundial del Sueño y Día Europeo de la Narcolepsia*, Sociedad Española de Neurología, 2024. dirección: https://www.sen.es/saladeprensa/pdf/Link182.pdf.

[10] National Heart, Lung, and Blood Institute. «¿Qué son los estudios del sueño? »Dirección: https://www.nhlbi.nih.gov/health/sleep/studies.

[11] National Heart, Lung, and Blood Institute. «Apnea del sueño. »dirección: https://www.nhlbi.nih.gov/health/sleep-apnea.

[12] National Heart, Lung, and Blood Institute. «Privación y la deficiencia de sueño. »dirección: https://www.nhlbi.nih.gov/es/health/sleep-deprivation/health-effects.

[13] National Heart, Lung, and Blood Institute. «Narcolepsia. »dirección: https://www.nhlbi.nih.gov/health/narcolepsy.

[14] National Heart, Lung, and Blood Institute. «Insomnio. »dirección: https://www.nhlbi.nih.gov/health/insomnia.

[15] National Heart, Lung, and Blood Institute, *Your Guide to Healthy Sleep*, Washington, D.C., 2011. dirección: https://www.nhlbi.nih.gov/resources/your-guide-healthy-sleep.

[16] S. Morokuma, «Deep learning-based sleep stage classification with cardiorespiratory and body movement activities in individuals with suspected sleep disorders,» *Proceedings of the London Mathematical Society*, vol. 42, n.º 1, págs. 230-265, 1936. DOI: 10.1112/plms/s2-42.1.230.

[17] D. E. Rumelhart, G. E. Hinton y R. J. Williams, «Learning Representations by Back Propagating Errors,» *Nature*, vol. 323, págs. 533-536, 1986. DOI: 10.1038/323533a0.

[18] K. Hornik, M. Stinchcombe y H. White, «Multilayer Feedforward Networks are Universal Approximators,» *Neural Networks*, vol. 2, n.º 5, págs. 359-366, 1989. DOI: 10.1016/0893-6080(89)90020-8.

[19] S. Hochreiter y J. Schmidhuber, «Long Short-Term Memory,» *Neural Computation*, vol. 9, n.º 8, págs. 1735-1780, 1997. DOI: 10.1162/neco.1997.9.8.1735. dirección: https://doi.org/10.1162/neco.1997.9.8.1735.

[20] M. Hassan y A. Kumar, «Sleep Stage Classification Using Deep Learning: A Review,» *Journal of Neuroscience Methods*, vol. 302, págs. 94-106, 2018. DOI: 10.1016/j.jneumeth.2018.02.007. dirección: https://doi.org/10.1016/j.jneumeth.2018.02.007.

[21] X. Zhang, D. Zheng e Y. Liu, «A Deep Learning Approach for Sleep Stage Classification Based on Multi-Channel EEG Data,» *Neurocomputing*, vol. 219, págs. 389-398, 2017. DOI: 10.1016/j.neucom.2016.09.085. dirección: https://doi.org/10.1016/j.neucom.2016.09.085.

[22] M. A. Rashid, M. Usman e I. Hussain, «Sleep Stage Classification Using Deep Learning Algorithms,» *IEEE Access*, vol. 6, págs. 36 846-36 857, 2018. DOI: 10.1109/ACCESS.2018.2845977. dirección: https://doi.org/10.1109/ACCESS.2018.2845977.

[23] X. Liu, Z. Zhang y L. Yang, «Sleep Stage Classification Using a Low-Cost Portable EEG System,» *IEEE Transactions on Biomedical Engineering*, vol. 65, n.º 11, págs. 2672-2681, 2018. DOI: 10.1109/TBME.2017.2752954. dirección: https://doi.org/10.1109/TBME.2017.2752954.

[24] R. Schirrmeister et al., «Deep Learning with Convolutional Neural Networks for EEG Decoding and Visualization,» *Human Brain Mapping*, vol. 38, n.º 11, págs. 5391-5420, 2017. DOI: 10.1002/hbm.23730. dirección: https://doi.org/10.1002/hbm.23730.

[25] U. Erdenebayar, Y. Kim, J.-U. Park, S. Lee y K.-J. Lee, «Automatic classification of sleep stage from an ECG signal using a gated-recurrent unit,» *International Journal of Fuzzy Logic and Intelligent Systems*, vol. 20, n.º 3, págs. 181-187, 2020. DOI: 10.5391/IJFIS.2020.20.3.181.

[26] M. Adnane, Z. Jiang y Z. Yan, «Sleep–wake stages classification and sleep efficiency estimation using single-lead electrocardiogram,» *Expert Systems with Applications*, vol. 39, n.º 1, págs. 1401-1413, 2012. DOI: 10.1016/j.eswa.2011.08.022.

[27] M. Xiao, H. Yan, J. Song, Y. Yang y X. Yang, «Sleep stages classification based on heart rate variability and random forest,» *Biomedical Signal Processing and Control*, vol. 8, n.º 6, págs. 624-633, 2013. DOI: 10.1016/j.bspc.2013.06.001.

[28] J. Singh, R. K. Sharma y A. K. Gupta, «A method of REM-NREM sleep distinction using ECG signal for unobtrusive personal monitoring,» *Computers in Biology and Medicine*, vol. 78, págs. 138-143, 2016. DOI: 10.1016/j.compbiomed.2016.09.018.

[29] S. Yucelbas, C. Yucelbas, G. Tezel, S. Ozsen y S. Yosunkaya, «Automatic sleep staging based on SVD, VMD, HHT and morphological features of single-lead ECG signal,» *Expert Systems with Applications*, vol. 102, págs. 193-206, 2018. DOI: 10.1016/j.eswa.2018.02.034.

[30] M. Gaiduk, T. Penzel, J. A. Ortega y R. Seepold, «Automatic sleep stages classification using respiratory, heart rate and movement signals,» *Physiological Measurement*, vol. 39, n.º 12, pág. 124 008, 2018. DOI: 10.1088/1361-6579/aaf5d4.

[31] M. Wang, J. Guan, T. Sun et al., «Enhancing automatic sleep stage classification with cerebellar EEG and machine learning techniques,» *Computers in Biology and Medicine*, vol. 185, pág. 109 515, 2025. DOI: 10.1016/j.compbiomed.2024.109515.

[32] H. Phan, F. Andreotti, N. Cooray, O. Y. Chen y M. de Vos, «SeqSleepNet: End-to-End Hierarchical Recurrent Neural Network for Sequence-to-Sequence Automatic Sleep Staging,» *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 27, n.º 3, págs. 400-410, 2019. DOI: 10.1109/TNSRE.2019.2896659.

[33] L. Fraiwan, K. Lweesy, N. Khasawneh, M. Fraiwan, H. Wenz y H. Dickhaus, «Time Frequency Analysis for Automated Sleep Stage Identification in Fullterm and Preterm Neonates,» *Journal of Medical Systems*, vol. 35, n.º 4, págs. 693-702, 2011. DOI: 10.1007/s10916-009-9406-2.

[34] K. A. I. Aboalayon, M. Faezipour, W. S. Almuhammadi y S. Moslehpour, «Sleep Stage Classification Using EEG Signal Analysis: A Comprehensive Survey and New Investigation,» *Entropy*, vol. 18, n.º 9, pág. 272, 2016. DOI: 10.3390/e18090272.

[35] A. Supratak, H. Dong, C. Wu e Y. Guo, «DeepSleepNet: A Model for Automatic Sleep Stage Scoring Based on Raw Single-Channel EEG,» *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 25, n.º 11, págs. 1998-2008, 2017. DOI: 10.1109/TNSRE.2017.2721116.

[36] E. Bresch, U. Großekathöfer y G. Garcia-Molina, «Recurrent Deep Neural Networks for Real-Time Sleep Stage Classification From Single-Channel EEG,» *Frontiers in Computational Neuroscience*, vol. 12, pág. 85, 2018. DOI: 10.3389/fncom.2018.00085.

[37] E. Eldele et al., «An Attention-Based Deep Learning Approach for Sleep Stage Classification With Single-Channel EEG,» *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 29, págs. 809-818, 2021. DOI: 10.1109/TNSRE.2021.3058638.

[38] T. Tezuka, K. Imaizumi, S. Utsunomiya et al., «Real-time, automatic, open-source sleep stage classification system using single EEG for mice,» *Scientific Reports*, vol. 11, pág. 8698, 2021. DOI: 10.1038/s41598-021-90332-1.

[39] Y. Zhang, J. He, L. Wang, Y. Li y J. Dong, «Sleep apnea detection from single-lead ECG using bidirectional LSTM networks,» *Biomedical Signal Processing and Control*, vol. 68, pág. 102602, 2021. DOI: 10.1016/j.bspc.2021.102602.

[40] T. I. Toma y S. Choi, «An End-to-End Multi-Channel Convolutional Bi-LSTM Network for Automatic Sleep Stage Detection,» *Sensors*, vol. 23, n.º 10, pág. 4950, 2023. DOI: 10.3390/s23104950.

[41] Y. Zhang, J. Jiang e Y. Wang, «Arrhythmia Detection Using GRU-Based Recurrent Neural Networks,» *IEEE Access*, vol. 8, págs. 134962-134970, 2020. DOI: 10.1109/ACCESS.2020.3009700.

[42] T. Song, W. Zheng, Z. Guo y Z. Cui, «EEG-Based Emotion Recognition Using Self-Attention Mechanism and Graph Convolutional Network,» *IEEE Transactions on Affective Computing*, 2021. DOI: 10.1109/TAFFC.2021.3111592.

[43] A. Dosovitskiy et al., «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,» *CoRR*, vol. abs/2010.11929, 2020. arXiv: 2010.11929. dirección: https://arxiv.org/abs/2010.11929.

[44] A. Hassani, S. Walton, N. Shah, A. Abuduweili, J. Li y H. Shi, «Escaping the Big Data Paradigm with Compact Transformers,» *CoRR*, vol. abs/2104.05704, 2021. arXiv: 2104.05704. dirección: https://arxiv.org/abs/2104.05704.

[45] M. Gao, Y. Zhang, P. Hu e Y. Lin, «Hybrid Deep Learning Model With Attention Mechanism for Sleep Stage Classification,» *IEEE Journal of Biomedical and Health Informatics*, vol. 26, n.º 2, págs. 721-731, 2022. DOI: 10.1109/JBHI.2021.3101746.

[46] H. Phan, F. Andreotti, N. Cooray, O. Y. Chen y M. De Vos, «SeqSleepNet: End-to-End Hierarchical Recurrent Neural Network for Sequence-to-Sequence Automatic Sleep Staging,» *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 27, n.º 3, págs. 400-410, 2019. DOI: 10.1109/TNSRE.2019.2896659.

[47] A. Vilamala, M. B. Petersen y L. K. Hansen, «Deep convolutional neural networks for interpretable analysis of EEG sleep stage scoring,» *Proceedings of the 25th European Signal Processing Conference (EUSIPCO)*, 2017. dirección: https://arxiv.org/abs/1710.00633.

[48] J. B. Stephansen, A. N. Olesen, M. Olsen y et al., «Neural network analysis of sleep stages enables efficient diagnosis of narcolepsy,» *Nature Communications*, vol. 9, pág. 5229, 2018. DOI: 10.1038/s41467-018-07229-3.

[49] S. Chambon, M. N. Galtier, P. J. Arnal y et al., «A deep learning architecture for temporal sleep stage classification using multivariate & multimodal time series,» *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 26, n.º 4, págs. 758-769, 2018. DOI: 10.1109/TNSRE.2018.2813138.

[50] Y. Li, Z. Xu, Y. Zhang y Z. Cao, «Automatic Sleep Stage Classification Based on Two-Channel EOG and One-Channel EMG,» *Physiological Measurement*, vol. 43, n.º 7, pág. 075001, 2022. DOI: 10.1088/1361-6579/ac6bdb.

[51] M. Gaiduk et al., «Estimation of Sleep Stages Analyzing Respiratory and Movement Signals,» *IEEE Journal of Biomedical and Health Informatics*, vol. 26, n.º 2, págs. 505-514, 2022. DOI: 10.1109/JBHI.2021.3099295.

[52] O. J. Walch, A. Cochran y D. B. Forger, «Validation of a Machine-Learning Approach for Classifying Sleep and Wake from Accelerometry and Heart Rate in Free-Living Conditions,» *Sleep*, vol. 42, n.º 12, zsz193, 2019. DOI: 10.1093/sleep/zsz193.

[53] P. Lopez-Meyer, F. Zamarripa y J. Ramos, «BreathFinder: Non-Invasive Isolation of Respiratory Cycles with Inductive Belts,» *Sensors*, vol. 22, n.º 18, pág. 6982, 2022. DOI: 10.3390/s22186982.

[54] *The AASM Manual for the Scoring of Sleep and Associated Events: Rules, Terminology and Technical Specifications, Version 2.6*, American Academy of Sleep Medicine, Darien, IL, 2020.

[55] S. S. Hamid, Y. Ahn e Y. Cha, «Respiratory Depth Estimation Using Inductive Plethysmography,» *IEEE Journal of Biomedical and Health Informatics*, vol. 25, n.º 9, págs. 3607-3616, 2021. DOI: 10.1109/JBHI.2021.3054275.

[56] L. Pérez y J. Martínez, «Multimodal Fusion Techniques for Sleep Stage Detection,» *Neurocomputing*, vol. 412, 235–247, 2021. DOI: 10.1016/j.neucom.2020.09.076.

[57] D. Moser, A. Eckert y S. Tezer, «Sleep staging using wearable sensors: A clinical validation study,» *Sleep Medicine*, vol. 42, págs. 94-102, 2018. DOI: 10.1016/j.sleep.2017.10.008.

[58] C. Iber, S. Ancoli-Israel, A. Chesson y S. Quan, *The AASM Manual for the Scoring of Sleep and Associated Events: Rules, Terminology and Technical Specifications*, 1st. American Academy of Sleep Medicine, 2007.

[59] M. Hirshkowitz, K. Whiton, S. M. Albert y et al., «National Sleep Foundation's Sleep Time Duration Recommendations: Methodology and Results Summary,» *Sleep Health*, vol. 1, n.º 1, 40–43, 2015. DOI: 10.1016/j.sleh.2014.12.010.

[60] L. Sun, X. Chen y J. Guo, «Optimizing Sampling Rates for Wearable Respiratory Monitoring in Sleep Studies,» *Sensors*, vol. 23, n.º 12, pág. 5123, 2023. DOI: 10.3390/s23125123.

[61] Q. Zhang, C. Li y K. Wang, «Class-Balanced Sampling Improves Automatic Sleep Staging on Imbalanced Datasets,» *IEEE Journal of Biomedical and Health Informatics*, vol. 27, n.º 2, págs. 861-872, 2023. DOI: 10.1109/JBHI.2022.3208875.

[62] J. R. Landis y G. G. Koch, «The Measurement of Observer Agreement for Categorical Data,» *Biometrics*, vol. 33, n.º 1, 159–174, 1977. DOI: 10.2307/2529310.

[63] M. H. Silber, S. Ancoli-Israel, M. H. Bonnet y et al., «The Visual Scoring of Sleep in Adults,» *Journal of Clinical Sleep Medicine*, vol. 3, n.º 2, 121–131, 2007.

[64] J. R. Shambroom, S. Fabregas y J. Johnstone, «Validation of an Automated Wireless System to Monitor Sleep in Healthy Adults,» *Journal of Sleep Research*, vol. 21, n.º 2, 221–230, 2012. DOI: 10.1111/j.1365-2869.2011.00936.x.

[65] Y. Bengio, P. Simard y P. Frasconi, «Learning Long-Term Dependencies with Gradient Descent is Difficult,» *IEEE Transactions on Neural Networks*, vol. 5, n.º 2, págs. 157-166, 1994. DOI: 10.1109/72.279181.

[66] M. Schuster y K. K. Paliwal, «Bidirectional Recurrent Neural Networks,» *IEEE Transactions on Signal Processing*, vol. 45, n.º 11, págs. 2673-2681, 1997. DOI: 10.1109/78.650093.

[67] D. Tran, Z. Zhang, I. Sepulveda, P. Rajpurkar, M. P. Lungren y A. Y. Ng, «Deep Learning for Automated Sleep Stage Scoring Based on Polysomnography,» *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018.

[68] S. Hochreiter y J. Schmidhuber, «LSTM can solve hard long time lag problems,» en *Proceedings of the 9th International Conference on Neural Information Processing Systems (NIPS)*, Cambridge, MA, USA: MIT Press, 1996, págs. 473-479.

[69] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong y W.-c. Woo, «Convolutional LSTM network: A machine learning approach for precipitation nowcasting,» en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, págs. 802-810.

[70] A. Moradzadeh, H. Moayyed, S. Zakeri, B. Mohammadi-Ivatloo y A. P. Aguiar, «Deep learning-assisted short-term load forecasting for sustainable management of energy in micro-grid,» *Inventions*, vol. 6, págs. 1-11, 2021.

[71] Y. LeCun, L. Bottou, Y. Bengio y P. Haffner, «Handwritten digit recognition with a back-propagation network,» en *Advances in Neural Information Processing Systems*, 1990, págs. 396-404.

[72] D. H. Hubel y T. N. Wiesel, «Receptive fields of single neurones in the cat's striate cortex,» *The Journal of Physiology*, vol. 148, n.º 3, págs. 574-591, 1959.

[73] O. Faust, A. Shenfield, Y. Hagiwara, T. J. Hong y U. R. Acharya, «Deep Learning for Sleep Stage Classification: A Review,» *Sleep Medicine Reviews*, vol. 55, pág. 101 421, 2021. DOI: 10.1016/j.smrv.2020.101421.

[74] O. Tsinalis, P. M. Matthews e Y. Guo, «Automatic sleep stage scoring using time-frequency analysis and stacked sparse autoencoders,» *Annals of Biomedical Engineering*, vol. 44, n.º 5, págs. 1587-1597, 2016. DOI: 10.1007/s10439-015-1444-y.

[75] T.-Y. Lin, P. Goyal, R. B. Girshick, K. He y P. Dollár, «Focal Loss for Dense Object Detection,» en *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, págs. 2999-3007. DOI: 10.1109/ICCV.2017.324.

[76] A. Fernández, S. Garcia, F. Herrera y N. V. Chawla, «SMOTE for Learning from Imbalanced Data: Progress and Challenges, Marking the 15-Year Anniversary,» *Journal of Artificial Intelligence Research*, vol. 61, págs. 863-905, 2018. DOI: 10.1613/jair.1.11192.

[77] T. Zhao, Y. Cui y T. Ji, «VAEEG: Variational Auto-Encoder for Extracting EEG Representation,» *NeuroImage*, vol. 304, pág. 120 946, 2024. DOI: 10.1016/j.neuroimage.2024.120946.

[78] C. Yang, C. Xiao, B. M. y J. Sun, «Self-Supervised Electroencephalogram Representation Learning for Automatic Sleep Staging: Model Development and Evaluation Study,» *JMIR AI*, vol. 2, n.º 1, e46769, 2023. DOI: 10.2196/46769.

[79] Y. Guo, M. Nowakowski y D. Dai, «FlexSleepTransformer: A Transformer-Based Sleep Staging Model with Flexible Input Channel Configurations,» *Scientific Reports*, vol. 14, n.º 26312, 2024. DOI: 10.1038/s41598-024-76197-0.

[80] H. He y E. A. Garcia, «Learning from Imbalanced Data,» *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, n.º 9, págs. 1263-1284, 2009. DOI: 10.1109/TKDE.2008.239.

[81] G. van Rossum y F. L. Drake, *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009, ISBN: 9781441412690.

[82] M. Abadi et al., «TensorFlow: A System for Large-Scale Machine Learning,» en *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, USENIX Association, 2016, págs. 265-283. dirección: https://www.usenix.org/system/files/conference/osdi16/osdi16-abadi.pdf.

[83] F. Chollet, *Deep Learning with Python*. Manning Publications, 2017, ISBN: 9781617294433. dirección: https://www.manning.com/books/deep-learning-with-python.

[84] Y. Zhang, X. Wang, A. Supratak e Y. Guo, «Sleep stage classification using bidirectional LSTM in combination with CNN,» *Biomedical Signal Processing and Control*, vol. 61, pág. 102 037, 2020. DOI: 10.1016/j.bspc.2020.102037.

[85] K. J. Millman y M. Aivazis, «Python for Scientists and Engineers,» *Computing in Science & Engineering*, vol. 13, n.º 2, págs. 9-12, 2011. DOI: 10.1109/MCSE.2011.36.

[86] L. B. de Amorim, G. D. Cavalcanti y R. M. Cruz, «The choice of scaling technique matters for classification performance,» *Applied Soft Computing*, vol. 133, pág. 109 924, 2023, ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2022.109924. dirección: https://www.sciencedirect.com/science/article/pii/S1568494622009735.

[87] W. Alsobhi, T. Alafif, A. Abdel-Hakim y W. Zong, *Adaptive Batch Normalization for Training Data with Heterogeneous Features*, 2022. arXiv: 2211.02050 [cs.LG]. dirección: https://arxiv.org/abs/2211.02050.

[88] P. Siirtola, S. Tamminen, G. Chandra, A. Ihalapathirana y J. Röning, «Predicting Emotion with Biosignals: A Comparison of Classification and Regression Models for Estimating Valence and Arousal Level Using Wearable Sensors,» *Sensors*, vol. 23, n.º 3, pág. 1598, 2023. DOI: 10.3390/s23031598.

[89] R. Cordeiro, N. Karimian e Y. Park, «Hyperglycemia Identification Using ECG in Deep Learning Era,» *Sensors*, vol. 21, n.º 18, pág. 6263, 2021. DOI: 10.3390/s21186263.

[90] V. J. Hodge y J. Austin, «A Survey of Outlier Detection Methodologies,» *Artificial Intelligence Review*, vol. 22, n.º 2, págs. 85-126, 2004. DOI: 10.1023/B:AIRE.0000045502.10941.a9.

[91] C. Leys, C. Ley, O. Klein, P. Bernard y L. Licata, «Detecting Outliers: Do Not Use Standard Deviation Around the Mean, Use Absolute Deviation Around the Median,» *Journal of Experimental Social Psychology*, vol. 49, n.º 4, págs. 764-766, 2013. DOI: 10.1016/j.jesp.2013.03.013.

[92] J. Han, M. Kamber y J. Pei, *Data Mining: Concepts and Techniques*, 3rd. San Francisco, CA: Morgan Kaufmann, 2011.

[93] M. A. K. September, F. S. Passino, L. Goldmann y A. Hinel, «Extended Deep Adaptive Input Normalization for Preprocessing Time Series Data for Neural Networks,» *arXiv preprint arXiv:2310.14720*, 2023. dirección: https://arxiv.org/abs/2310.14720.

[94] L. B. V. Amorim, G. D. C. Cavalcanti y R. M. O. Cruz, «The Choice of Scaling Technique Matters for Classification Performance,» *Journal of Computational Science*, vol. 64, pág. 101 915, 2022. DOI: 10.1016/j.jocs.2022.101915.

[95] A. September, D. Gligorijevic y D. Sontag, «EDAIN: Extended Deep Adaptive Input Normalization for Time Series Forecasting,» *Pattern Recognition*, vol. 145, pág. 109 866, 2023. DOI: 10.1016/j.patcog.2023.109866.

[96] L. Breiman, «Random forests,» *Machine learning*, vol. 45, n.º 1, págs. 5-32, 2001.

[97] A. Liaw y M. Wiener, «Classification and regression by randomForest,» *R news*, vol. 2, n.º 3, págs. 18-22, 2002.

[98] I. Guyon y A. Elisseeff, «An introduction to variable and feature selection,» *Journal of machine learning research*, vol. 3, págs. 1157-1182, 2003.

[99] D. M. W. Powers, «Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation,» *Journal of Machine Learning Technologies*, vol. 2, n.º 1, págs. 37-63, 2011.

[100] Y. Sasaki, «The truth of the F-measure,» en *Proceedings of the International Symposium on Trustworthy Global Computing*, 2007, págs. 1-5.

[101] C. Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 2.ª ed. Christoph Molnar, 2022, ISBN: 9798411463330.

[102] I. Jolliffe, *Principal component analysis*. Springer, 2011.

[103] I. T. Jolliffe, *Principal Component Analysis* (Springer Series in Statistics), 2.ª ed. New York: Springer, 2002, ISBN: 978-0387954424.

[104] I. T. Jolliffe y J. Cadima, «Principal component analysis: a review and recent developments,» *Philosophical Transactions of the Royal Society A*, vol. 374, n.º 2065, pág. 20 150 202, 2016. DOI: 10.1098/rsta.2015.0202.

[105] J. Lever, M. Krzywinski y N. Altman, «Principal component analysis,» *Nature Methods*, vol. 14, págs. 641-642, 2017. DOI: 10.1038/nmeth.4346.

[106] I. T. Jolliffe, *Principal Component Analysis* (Springer Series in Statistics), 2.ª ed. New York: Springer, 2002. DOI: 10.1007/b98835.

[107] C. J. Willmott y K. Matsuura, «Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance,» *Climate Research*, vol. 30, n.º 1, págs. 79-82, 2005.

[108] T. Chai y R. R. Draxler, «Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature,» *Geoscientific Model Development*, vol. 7, n.º 3, págs. 1247-1250, 2014. DOI: 10.5194/gmd-7-1247-2014.

[109] R. J. Hyndman y G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd. OTexts, 2018. dirección: https://otexts.com/fpp2/.

[110] E. W. Weisstein, *Correlation Coefficient*, From MathWorld–A Wolfram Web Resource, 2004. dirección: https://mathworld.wolfram.com/CorrelationCoefficient.html.

[111] J. L. Rodgers y W. A. Nicewander, «Thirteen ways to look at the correlation coefficient,» *The American Statistician*, vol. 42, n.º 1, págs. 59-66, 1988.

[112] M. M. Mukaka, «A guide to appropriate use of correlation coefficient in medical research,» *Malawi Medical Journal*, vol. 24, n.º 3, págs. 69-71, 2012.

[113] J. Benesty, J. Chen, Y. Huang e I. Cohen, *Pearson Correlation Coefficient*. Springer, 2009, págs. 1-4.

[114] D. G. Altman, *Practical Statistics for Medical Research*. Chapman y Hall/CRC, 1991.

[115] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd. Routledge Academic, 1988.

[116] J. M. Wooldridge, *Introductory Econometrics: A Modern Approach*, 5th. Cengage Learning, 2012.

[117] M. H. Kutner, C. J. Nachtsheim, J. Neter y W. Li, *Applied Linear Statistical Models*, 5th. McGraw-Hill, 2004.

[118] D. C. Montgomery, E. A. Peck y G. G. Vining, *Introduction to Linear Regression Analysis*, 5th. Wiley, 2012.

[119] N. R. Draper y H. Smith, *Applied Regression Analysis*, 3rd. Wiley-Interscience, 1998.

[120] C. Sammut y G. I. Webb, «Confusion Matrix,» en *Encyclopedia of Machine Learning*, Springer, 2010, págs. 160-161. DOI: 10.1007/978-0-387-30164-8_134. dirección: https://link.springer.com/referenceworkentry/10.1007/978-0-387-30164-8_134.

[121] M. Sokolova y G. Lapalme, «A systematic analysis of performance measures for classification tasks,» *Information Processing & Management*, vol. 45, n.º 4, págs. 427-437, 2009.

[122] T. Fawcett, «An introduction to ROC analysis,» *Pattern Recognition Letters*, vol. 27, n.º 8, págs. 861-874, 2006.

[123] M. Sokolova y G. Lapalme, *A Systematic Analysis of Performance Measures for Classification Tasks*, 2009. DOI: 10.1016/j.ipm.2009.03.002.

[124] E.-J. Meulenbrugge, H. Sun, W. Ganglberger, S. Nasiri, R. J. Thomas y M. B. Westover, «Ordinal Sleep Depth: A Data-Driven Continuous Measurement of Sleep Depth,» *Journal of Sleep Research*, 2025, Epub ahead of print. DOI: 10.1111/jsr.70074.

[125] J. Luo, H. Lee, P. Kumar et al., «A Benchmark Dataset for Deep Learning Sleep Staging Models,» *Sleep*, vol. 46, n.º 7, zsad145, 2023. DOI: 10.1093/sleep/zsad145.

[126] M. Mittlböck y M. Schemper, «Explained Variation for Ordinal Regression Models,» *Statistics in Medicine*, vol. 35, n.º 20, págs. 3391-3403, 2016. DOI: 10.1002/sim.6941.

[127] E. Hardarson, A. S. Islind, E. S. Arnardottir y M. Óskarsdóttir, «Error Propagation from Sleep Stage Classification to Derived Sleep Parameters in Machine Learning on Data from Wearables,» *Current Sleep Medicine Reports*, vol. 9, págs. 140-151, 2023. DOI: 10.1007/s40675-023-00253-w.